

**UNIVERSIDAD AUTÓNOMA
GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS
DE LA COMPUTACIÓN Y TELECOMUNICACIONES**



PRIMER PARCIAL

**Plataforma Colaborativa de Modelado UML con
Generación Inteligente de Backend Spring Boot y
Aplicación Móvil Flutter**

ESTUDIANTE:

Sanchez Paniagua Felix Santiago

MATERIA: INGENIERÍA DE SOFTWARE I - SC

DOCENTE: ING. ROLANDO ANTONIO MARTINEZ CANEDO

1. FUNDAMENTACIÓN TEÓRICA	5
1.1 HERRAMIENTAS CASE	5
Propiedades distintivas	5
Clasificación de herramientas CASE	5
1.2 DESARROLLO DE SOFTWARE BASADO EN COMPONENTES	6
Propiedades fundamentales	6
Beneficios del enfoque	6
Ejemplos de aplicación	7
1.3 PROCESO DE DISEÑO DE BASES DE DATOS	7
1.3.1 Diseño Conceptual	7
1.3.2 Diseño Lógico	7
1.3.3 Normalización	8
1.3.4 Diseño Físico	8
1.4 TRANSFORMACIONES EN BASES DE DATOS	8
1.4.1 De Diseño Conceptual a Diseño Lógico	9
1.4.2 De Diseño Lógico a Diseño Físico	9
1.4.3 De Diseño Conceptual a Código Backend	10
1.5 UML - DIAGRAMA DE CLASES	11
1.5.1 Características principales	11
1.5.2 Tipos de relaciones	11
1.5.3 Ventajas de los diagramas UML	11
1.6 ARQUITECTURA DEL SOFTWARE	11
1.6.1 Enfoque arquitectónico	12
1.6.2 Componentes principales	12
1.6.2.1 Frontend	12
1.6.2.2 Backend	12
1.6.2.3 Base de Datos	13
1.6.2.4 Integración con IA	13
1.6.3 Flujo de colaboración en tiempo real	13
1.7 APLICACIÓN DE LA INTELIGENCIA ARTIFICIAL EN INGENIERÍA DE SOFTWARE	14
1.7.1 Inteligencia Artificial (IA)	14
1.7.2 Inteligencia Artificial Generativa (IAG)	14
1.7.3 Ingeniería de Software	14
1.7.4 IA en Ingeniería de Software	14
1.7.5 IAG en Ingeniería de Software	14
1.8 SPRING BOOT, ORM Y ARQUITECTURA EN CAPAS	15
1.8.1 Spring Boot	15
1.8.2 ORM en Spring Boot	15
1.8.3 Arquitectura de 4 capas	15
1. Modelo (Entity)	15
2. Repositorio (Repository)	16

3. Servicio (Service)	16
4. Controlador (Controller)	16
1.8.4 DTOs (Data Transfer Objects)	16
1.9 POSTMAN	17
1.9.1 Características principales	17
1.10 PROCESO UNIFICADO DE DESARROLLO (PUDS)	17
1.10.1 Fases y Flujos de Trabajo del PUDS	17
1.10.2 Fases del PUDS	18
1. Inicio (Inception)	18
2. Elaboración (Elaboration)	18
3. Construcción (Construction)	18
4. Transición (Transition)	18
1.10.3 Flujos de trabajo del PUDS	18
1.10.3.1. Captura de Requisitos	18
1.10.3.2. Análisis	19
1.10.3.3. Diseño	19
1.10.3.4. Implementación	19
1.10.3.5. Pruebas	19
2. PUDS (PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE)	20
2.1 CAPTURA DE REQUISITOS - CICLO #1	20
2.1.1 Identificar Actores Y Casos de Usos	20
-2.1.1.1 Identificar Actores	20
2.1.1.2 Identificar Casos de Uso	20
2.1.1.3 Priorizar Casos de Uso	23
2.1.2 Especificar Casos de Uso	25
2.1.3 Estructurar Modelo de Casos de Uso	37
2.2. ANÁLISIS	38
2.2.1 Análisis de la Arquitectura	38
2.2.1.1 Identificar Paquetes	38
2.2.1.2 Relacionar Paquetes y Casos de Uso	39
2.2.2 Análisis de casos de uso	40
2.2.2.1 Diagramas de Colaboración/Comunicación	40
2.2.3 Análisis de Paquetes	46
2.3 DISEÑO	47
2.3.1 Diseño de Arquitectura	47
2.3.1.1 Diseño Físico	47
2.3.1.2 Diseño Lógico (Diagrama Organizado en Capas)	48
2.3.2 Diseño de Datos	49
2.3.2.1 Diseño Datos Lógicos	49
2.3.2.1.1 Diagrama de Clases	49
2.3.2.1.2 Mapeo	50

2.3.2.1.3 Normalización	51
2.3.2.2 Diseño Datos Físicos	51
2.3.2.2.1 Script	51
2.3.2.2.5 Diagrama Entidad Relación	57
2.3.3 Diagramas de secuencia	58
2.4 IMPLEMENTACIÓN	65
2.4.1 Elección de plataforma de Desarrollo de Software	65
2.4.1.1 Lenguaje de Programación	65
2.4.1.2 DBMS (Database Management System)	66
2.4.1.3 Sistemas Operativos e Infraestructura	67
2.4.2 Implementación de la Arquitectura del Sistema	68
2.5 AMPLIACIÓN FUNCIONAL - CICLO #2	72
2.5.1 Objetivo de la ampliación	72
2.5.2 Nuevos actores y requisitos	73
2.5.3 Casos de uso ampliados	74
2.5.4 Arquitectura objetivo en Google Cloud	78
2.5.5 Requisitos no funcionales y aceptación	79
2.5.6 Requisitos de paridad funcional en Android	80
2.5.7 Casos de uso complementarios del sistema	81
2.5.8 Casos de uso de paridad móvil	82
2.5.9 Arquitectura Android	83
3. DOCUMENTACIÓN PARA USO DEL SOFTWARE	84
4. BIBLIOGRAFÍA	89
5. ANEXOS	90

1. FUNDAMENTACIÓN TEÓRICA

1.1 HERRAMIENTAS CASE

Las herramientas de Ingeniería de Software Asistida por Computadora (CASE, por sus siglas en inglés) constituyen aplicaciones especializadas que respaldan las distintas etapas del desarrollo de software. Estas herramientas ofrecen un ambiente integrado que abarca desde la especificación inicial de requerimientos hasta el mantenimiento continuo del sistema, incluyendo las fases intermedias de modelado, diseño e implementación. El propósito central de estas tecnologías es incrementar la eficiencia del proceso de desarrollo, elevar la calidad del producto final y garantizar la coherencia del software, mientras se optimizan los recursos de tiempo y presupuesto a lo largo del proyecto.

Propiedades distintivas

- Representación gráfica mediante estándares reconocidos (como UML, BPMN y diagramas ERD)
- Capacidad de crear código fuente desde modelos abstractos
- Comprobación automática de consistencia en los diagramas
- Producción automatizada de documentación técnica
- Funcionalidades para trabajo en equipo distribuido geográficamente

Clasificación de herramientas CASE

1. Upper CASE (Fase superior): orientadas a las etapas tempranas de análisis y diseño

- *Rational Rose*: herramienta emblemática de los años 90-2000, enfocada en UML y análisis orientado a objetos
- *Enterprise Architect*: solución actual especializada en modelado UML y diseño arquitectónico
- *Visual Paradigm*: plataforma que integra UML, BPMN, casos de uso, con capacidades de ingeniería directa e inversa
- *React Flow*: biblioteca moderna de JavaScript/TypeScript para crear editores de diagramas interactivos, utilizada en aplicaciones web para visualización de flujos y diagramas UML

2. Lower CASE (Fase inferior): destinadas a implementación, validación y mantenimiento

- *JUnit*: framework de pruebas unitarias para Java, considerado lower CASE por su aplicación en la fase de codificación
- *SonarQube*: plataforma de análisis de calidad y métricas de código

3. Integrated CASE (I-CASE): fusionan ambos enfoques, cubriendo el ciclo completo

- *StarUML*: permite el modelado y la creación de estructuras de código en múltiples lenguajes
- *IBM Rational Software Architect*: evolución de Rational Rose con cobertura integral del ciclo de vida
- *GenMyModel*: herramienta CASE basada en la nube con colaboración sincrónica y generación de código
- *Plataformas web modernas*: aplicaciones como la presente que combinan editores visuales, colaboración en tiempo real y generación automática de código utilizando tecnologías web (Next.js, NestJS, React Flow)

1.2 DESARROLLO DE SOFTWARE BASADO EN COMPONENTES

El paradigma de Desarrollo de Software Basado en Componentes (CBSE, Component-Based Software Engineering) representa un enfoque metodológico donde los sistemas se construyen ensamblando unidades reutilizables previamente desarrolladas, en contraposición al desarrollo desde cero. Un componente se conceptualiza como un módulo de software independiente que posee una interfaz claramente definida y que encapsula su funcionalidad interna.

Propiedades fundamentales

- **Reutilización**: los componentes son aplicables en distintos contextos y proyectos, reduciendo esfuerzo y costos
- **Encapsulamiento**: cada componente mantiene oculta su lógica interna, exponiendo solamente las interfaces públicas
- **Interoperabilidad**: diseñados para integrarse sin fricciones mediante contratos de interfaz explícitos
- **Independencia tecnológica**: pueden desarrollarse en diferentes lenguajes/plataformas mientras respeten los estándares
- **Mantenibilidad mejorada**: el aislamiento de responsabilidades facilita actualizaciones sin comprometer el sistema global

Beneficios del enfoque

- Acelera significativamente los tiempos de desarrollo mediante la reutilización de módulos probados
- Reduce la incidencia de defectos al emplear componentes validados previamente
- Promueve escalabilidad y adaptabilidad gracias a la arquitectura modular
- Simplifica la gestión de sistemas complejos

Ejemplos de aplicación

- **JavaBeans / Enterprise JavaBeans (EJB):** en ecosistemas empresariales Java, los EJB proporcionan servicios de transaccionalidad, persistencia y autenticación
- **.NET Framework:** emplea ensamblados y librerías reutilizables para construir soluciones multiplataforma
- **CORBA:** estándar que facilitó la interoperación entre componentes en diferentes lenguajes
- **Arquitecturas de Servicios Web y Microservicios:** paradigmas contemporáneos donde cada servicio funciona como componente autónomo comunicándose vía APIs

1.3 PROCESO DE DISEÑO DE BASES DE DATOS

El diseño de bases de datos constituye una fase crítica en el desarrollo de sistemas informáticos, asegurando que la información se almacene de forma consistente, eficiente y segura, habilitando su consulta, modificación y análisis posteriores. Un diseño robusto garantiza la integridad informacional y optimiza el desempeño de las aplicaciones dependientes.

Este proceso típicamente se estructura en múltiples etapas progresivas, desde una descripción conceptual abstracta hasta su materialización física en un Sistema Gestor de Bases de Datos (SGBD).

1.3.1 Diseño Conceptual

Esta etapa inicial define los requerimientos informativos del sistema sin limitaciones tecnológicas. Se emplean modelos abstractos como el Modelo Entidad-Relación (ER) o diagramas de clases UML.

Ejemplo: representar entidades como Estudiante, Tarea y Carrera, especificando sus propiedades y vínculos.

Resultado: diagrama conceptual que ilustra entidades, propiedades, relaciones y multiplicidades.

1.3.2 Diseño Lógico

En esta fase se convierte el modelo conceptual en un modelo lógico, ajustado al paradigma de base de datos seleccionado (relacional, orientado a objetos, documental, etc.). Para bases relacionales, se traduce en estructuras de tablas, columnas y claves.

Ejemplo: la entidad Estudiante se transforma en la tabla `Estudiantes(id, nombre, correo)`, y su relación con Tarea se implementa mediante clave foránea `id_estudiante` en la tabla `Tareas`.

Resultado: esquema lógico especificando tablas, claves primarias y foráneas, restricciones de integridad y tipos de datos.

1.3.3 Normalización

Proceso que aplica reglas formales para eliminar redundancias e inconsistencias informacionales.

Ejemplo: si la tabla Estudiantes almacena también el nombre de Carrera, se genera duplicidad. La normalización separa esta información en dos tablas relacionadas (`Estudiantes` y `Carreras`).

Ventajas: previene anomalías de actualización, inserción y eliminación.

1.3.4 Diseño Físico

Define la implementación específica del modelo lógico en un SGBD concreto (MySQL, PostgreSQL, Oracle, etc.). Se establecen aspectos como:

- Tipos de datos precisos (VARCHAR, INTEGER, DATE)
- Índices para optimizar consultas
- Particiones, triggers, procedimientos almacenados
- Estrategias de seguridad y respaldo

Resultado: esquema físico implementado y operativo en el SGBD seleccionado.

1.4 TRANSFORMACIONES EN BASES DE DATOS

El diseño de bases de datos implica transformaciones sucesivas, donde cada nivel de abstracción se traduce en un modelo más próximo a la implementación. Estas conversiones aseguran que los requisitos conceptuales se preserven consistentemente hasta el código ejecutable.

1.4.1 De Diseño Conceptual a Diseño Lógico

Objetivo: convertir un modelo abstracto e independiente de tecnología (diagrama ER o UML) en un esquema adaptado al modelo de datos seleccionado (típicamente relacional).

Procedimiento:

- Cada entidad → tabla correspondiente
- Cada atributo → columna con tipo de dato apropiado
- Las relaciones → claves foráneas o tablas intermedias (para relaciones N:M)
- Definición de claves primarias para garantizar unicidad

Ejemplo:

- *Conceptual:* Entidad Estudiante con atributos id, nombre y relación 1..* con Tarea
- *Lógico:* Tablas `Estudiantes(id, nombre)` y `Tareas(id, titulo, id_estudiante)` con clave foránea

1.4.2 De Diseño Lógico a Diseño Físico

Objetivo: implementar el esquema lógico en un SGBD específico.

Procedimiento:

- Especificación de tipos de datos precisos (VARCHAR(100), BIGINT, DATE)
- Creación de índices para optimización
- Definición de restricciones (NOT NULL, UNIQUE, CHECK)
- Configuración de aspectos físicos (particionamiento, triggers, procedimientos, seguridad)

Ejemplo:

- *Lógico:* `Estudiantes(id, nombre)`
- *Físico (PostgreSQL):*

```
CREATE TABLE Estudiantes (  
  id_estudiante BIGSERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL  
);
```

1.4.3 De Diseño Conceptual a Código Backend

En enfoques modernos, resulta beneficioso extender la transformación directamente desde el diseño conceptual hasta la generación automática de código backend.

Concepto central: el diagrama de clases UML no solo describe entidades sino también componentes de software (modelo, repositorio, servicio, controlador).

Transformaciones principales:

- Clases UML → Entidades JPA (@Entity)
- Relaciones UML → Anotaciones JPA (@OneToMany, @ManyToOne, @ManyToMany)
- Atributos UML → Campos con validaciones (@NotNull, @Column)
- Proyecto UML → Código Spring Boot estructurado en capas

Ejemplo (Estudiante ↔ Tarea):

- *Conceptual:* Estudiante 1..* Tarea
- *Lógico:* Tablas con claves foráneas
- *Físico:* Tablas SQL en PostgreSQL
- *Backend generado:*
 - `Estudiante.java` (@Entity, colección de tareas)
 - `Tarea.java` (@Entity, referencia a estudiante)
 - `EstudianteRepository.java`, `EstudianteService.java`, `EstudianteController.java`

1.5 UML - DIAGRAMA DE CLASES

El Lenguaje Unificado de Modelado (UML, Unified Modeling Language) es un estándar establecido por la OMG (Object Management Group) para representar gráficamente elementos y relaciones de sistemas software. El diagrama de clases es uno de los más utilizados, describiendo la estructura estática del sistema mediante clases, atributos, operaciones y sus interrelaciones.

1.5.1 Características principales

- **Clases:** representan entidades u objetos del sistema con sus datos y comportamientos
- **Atributos:** propiedades o características de las clases (ej.: nombre, edad)
- **Métodos:** comportamientos o funcionalidades que ejecuta la clase (ej.: registrarTarea())
- **Visibilidad:** nivel de acceso (+ público, - privado, # protegido)
- **Relaciones:** interacciones entre clases

1.5.2 Tipos de relaciones

1. **Asociación:** vínculo entre clases (ej.: Estudiante realiza Tarea)
2. **Agregación:** relación débil "todo-parte" (ej.: Curso agrega Estudiante)
3. **Composición:** relación fuerte "todo-parte" donde la parte depende del todo (ej.: Pedido contiene DetallePedido)
4. **Herencia:** clase hija hereda de clase padre (ej.: Docente hereda de Persona)
5. **Dependencia:** uso temporal de una clase por otra (ej.: Controlador depende de Servicio)

1.5.3 Ventajas de los diagramas UML

- Comunicación efectiva entre analistas, desarrolladores y clientes
- Documentación estandarizada reconocida internacionalmente
- Fundamento para diseño de bases de datos relacionales
- Facilita generación automática de código
- Guía para mantener coherencia durante implementación y mantenimiento

1.6 ARQUITECTURA DEL SOFTWARE

La arquitectura define la estructura organizativa del sistema, identificando sus componentes principales y cómo interactúan. En este proyecto, la arquitectura persigue dos objetivos centrales:

1. Facilitar la creación y edición de diagramas UML
2. Habilitar colaboración en tiempo real entre múltiples usuarios

1.6.1 Enfoque arquitectónico

El sistema implementa un patrón cliente-servidor multicapa, donde el frontend web se comunica con el backend mediante APIs REST y WebSockets (Socket.IO) para funcionalidades colaborativas en tiempo real.

1.6.2 Componentes principales

1.6.2.1 Frontend

- **Framework:** Next.js 14 con App Router (basado en React 18)
- **Lenguaje:** TypeScript para tipado estático
- **Librería de diagramado:** React Flow para representación gráfica de diagramas UML
- **Gestión de estado:** Zustand para manejo de estado global (autenticación, workspaces)
- **Estilos:** Tailwind CSS para diseño responsivo
- **Comunicación:**
 - HTTP/REST con Fetch API para operaciones estándar (autenticación, CRUD, descarga)
 - Socket.IO Client para sincronización en tiempo real (movimiento, edición, creación)

Características destacadas del frontend:

- App Router de Next.js 14 para routing del lado del servidor
- Hooks personalizados (useSocket, useAuth, useWorkspace)
- Componentes UML especializados (UMLClassNode, UMLRelationshipEdge)
- Persistencia de estado con localStorage
- Auto-save y sincronización automática

1.6.2.2 Backend

- **Framework:** NestJS (Node.js con TypeScript)
- **ORM:** Prisma con PostgreSQL como motor de base de datos
- **Autenticación:** Passport.js con estrategias JWT y Local, bcrypt para hash de contraseñas
- **Comunicación en tiempo real:** Socket.IO para WebSockets
- **Estructura modular:**
 - Módulo Auth: autenticación y autorización con JWT
 - Módulo Workspace: gestión de espacios de trabajo y colaboradores
 - Módulo Diagram: operaciones CRUD de diagramas UML
 - Módulo Collaboration: sincronización en tiempo real vía WebSockets
 - Módulo AI-Chat: integración con Gemini AI de Google para generación automática
 - Módulo Code-Generation: transformación de diagramas a código Spring Boot

Características del backend:

- DTOs con class-validator para validación de entrada
- Guards personalizados para protección de rutas
- Manejo centralizado de excepciones
- Sistema de roles (OWNER, EDITOR, VIEWER)
- Logging estructurado de actividades

1.6.2.3 Base de Datos

- **Motor:** PostgreSQL
- **ORM:** Prisma para modelado y migraciones
- **Características:**
 - IDs con CUID para mejor distribución
 - Relaciones CASCADE para integridad referencial
 - Enums de TypeScript para tipos controlados
 - Timestamps automáticos
 - Índices únicos para optimización

Modelos principales: User, Workspace, WorkspaceCollaborator, Diagram, UMLClass, UMLAttribute, UMLMethod, UMLRelation, DiagramActivity, GeneratedCode

1.6.2.4 Integración con IA

- **Proveedor:** Gemini AI (Google)
- **Funcionalidad:** generación automática de diagramas UML desde descripciones en lenguaje natural
- **Características:**
 - Prompts optimizados para JSON estructurado
 - Sistema de fallback con plantillas predefinidas
 - Validación y corrección automática de modelos
 - Chat conversacional para refinamiento

1.6.3 Flujo de colaboración en tiempo real

1. Usuario propietario crea workspace y diagrama
2. Invita colaboradores mediante email con roles específicos
3. Usuarios se conectan al diagrama estableciendo canal Socket.IO
4. Backend crea sala (room) por diagrama para agrupar conexiones
5. Cada modificación emite eventos: **diagram_change**
6. Servidor propaga cambios a todos los clientes en la misma sala
7. Frontend actualiza canvas de React Flow instantáneamente
8. Sistema registra actividades en base de datos para auditoría

1.7 APLICACIÓN DE LA INTELIGENCIA ARTIFICIAL EN INGENIERÍA DE SOFTWARE

1.7.1 Inteligencia Artificial (IA)

La IA es una disciplina informática enfocada en desarrollar sistemas que emulen capacidades cognitivas humanas: razonamiento, aprendizaje, resolución de problemas y reconocimiento de patrones. Su propósito es habilitar decisiones autónomas o asistir al humano en tareas complejas.

Ejemplos tradicionales: sistemas expertos, motores de búsqueda, sistemas de recomendación, visión computacional, análisis predictivo.

1.7.2 Inteligencia Artificial Generativa (IAG)

Subdisciplina de IA especializada en crear contenido nuevo (texto, imágenes, audio, código) aprendiendo de grandes volúmenes de datos. Utiliza modelos como GANs o LLMs.

Ejemplos: generación de imágenes (DALL·E, Stable Diffusion), texto y código (ChatGPT, Copilot), música, diseño 3D.

1.7.3 Ingeniería de Software

Disciplina que aplica principios ingenieriles, metodologías y mejores prácticas al desarrollo, operación y mantenimiento de software de calidad. Cubre el ciclo completo: análisis, diseño, implementación, pruebas, despliegue y mantenimiento.

1.7.4 IA en Ingeniería de Software

La IA tradicional optimiza procesos del ciclo de vida:

- **Análisis de requisitos:** minería textual para detectar inconsistencias
- **Pruebas:** generación automática de casos de prueba, detección de anomalías
- **Mantenimiento predictivo:** detección y predicción de defectos
- **Gestión de proyectos:** estimación de esfuerzo basada en históricos
- **Calidad de código:** análisis estático identificando vulnerabilidades

1.7.5 IAG en Ingeniería de Software

La IAG automatiza tareas creativas y de desarrollo:

- **Generación de código:** desde lenguaje natural a código (GitHub Copilot)
- **Asistencia en diseño UML:** sugerencias de diagramas desde descripciones
- **Documentación automática:** manuales, comentarios, especificaciones API

- **Prototipado rápido:** interfaces y pruebas de concepto
- **Refactorización:** reestructuración para eficiencia y legibilidad
- **Pruebas automáticas:** generación de tests desde lógica existente

1.8 SPRING BOOT, ORM Y ARQUITECTURA EN CAPAS

1.8.1 Spring Boot

Framework Java basado en Spring Framework que simplifica el desarrollo de aplicaciones empresariales. Su ventaja principal es reducir configuración e integrar herramientas para crear aplicaciones production-ready con mínimo esfuerzo.

Características:

- Configuración automática (auto-configuration)
- Servidor embebido (Tomcat, Jetty)
- Integración con Spring Data, Spring Security, JPA
- Escalabilidad y soporte para microservicios

1.8.2 ORM en Spring Boot

Spring Boot utiliza Hibernate como ORM mediante Spring Data JPA, permitiendo mapear clases Java a tablas relacionales sin SQL manual.

Ejemplo:

```
@Entity
public class Estudiante {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_estudiante;
    private String nombre_estudiante;
}
```

1.8.3 Arquitectura de 4 capas

1. Modelo (Entity)

- Representa entidades del sistema mapeadas a BD
- Anotación: `@Entity`
- Ejemplo: Estudiante, Tarea

2. Repositorio (Repository)

- Gestiona persistencia de datos
- Extiende `JpaRepository`

```
public interface EstudianteRepository extends JpaRepository<Estudiante, Long> {}
```

3. Servicio (Service)

- Contiene lógica de negocio
- Anotación: `@Service`

`@Service`

```
public class EstudianteService {  
    private final EstudianteRepository repo;  
    public Estudiante crear(Estudiante e) { return repo.save(e); }  
}
```

4. Controlador (Controller)

- Expone endpoints REST
- Anotación: `@RestController`

`@RestController`

`@RequestMapping("/api/estudiantes")`

```
public class EstudianteController {  
    private final EstudianteService service;  
    @PostMapping  
    public Estudiante crear(@RequestBody Estudiante e) {  
        return service.crear(e);  
    }  
}
```

1.8.4 DTOs (Data Transfer Objects)

Objetos diseñados para transferir datos entre capas o entre backend-frontend.

Ventajas:

- Evitan exposición directa de entidades
- Definen exactamente qué datos se transmiten
- Facilitan validaciones y transformaciones

```
public class EstudianteDTO {  
    private String nombre;  
}
```

1.9 POSTMAN

Herramienta ampliamente adoptada para probar, documentar y automatizar APIs. Permite enviar peticiones HTTP (GET, POST, PUT, DELETE, etc.) y visualizar respuestas fácilmente.

Su popularidad radica en facilitar la validación ágil de servicios backend, acelerando el ciclo de desarrollo de aplicaciones basadas en APIs REST.

1.9.1 Características principales

- **Interfaz intuitiva:** creación de peticiones sin código adicional
- **Métodos HTTP:** GET, POST, PUT, PATCH, DELETE, etc.
- **Colecciones:** agrupación reutilizable de peticiones
- **Variables de entorno:** configuración dinámica (URLs, tokens, parámetros)
- **Pruebas automatizadas:** scripts JavaScript para validar respuestas automáticamente
- **Documentación:** generación de documentación interactiva compatible
- **Integraciones:** compatible con CI/CD y control de versiones

1.10 PROCESO UNIFICADO DE DESARROLLO (PUDS)

Metodología de ingeniería de software que propone un marco iterativo e incremental para desarrollo de sistemas. Creado por Ivar Jacobson, Grady Booch y James Rumbaugh, adoptado por Rational Software (IBM).

Su enfoque se centra en arquitectura del sistema y cumplimiento de requisitos funcionales/no funcionales, reduciendo riesgos y mejorando calidad.

1.10.1 Fases y Flujos de Trabajo del PUDS

El Proceso Unificado de Desarrollo de Software (PUDS) organiza el ciclo de vida del software en fases y en flujos de trabajo que se ejecutan de forma iterativa e incremental.

1.10.2 Fases del PUDS

El ciclo de vida se organiza en fases desarrolladas mediante iteraciones que producen entregables parciales.

1. Inicio (Inception)

- Definir visión, objetivos y alcance del sistema
- Identificar actores y casos de uso principales
- Estimar costos, tiempos y riesgos
- **Producto:** documento de visión, casos de uso preliminares, plan inicial

2. Elaboración (Elaboration)

- Refinar requisitos críticos y casos de uso detallados
- Diseñar arquitectura base
- Validar viabilidad técnica con prototipos
- **Producto:** modelo de análisis y diseño, prototipo arquitectónico, plan de construcción

3. Construcción (Construction)

- Implementar funcionalidades priorizadas en iteraciones
- Ejecutar pruebas unitarias e integrales
- Consolidar arquitectura y completar sistema
- **Producto:** versiones incrementales validables del sistema

4. Transición (Transition)

- Entregar sistema a producción
- Capacitar usuarios y preparar documentación
- Corregir errores detectados
- **Producto:** software en producción, documentación final, retroalimentación

1.10.3 Flujos de trabajo del PUDS

1.10.3.1. Captura de Requisitos

Identificar y documentar necesidades de clientes y usuarios mediante casos de uso (use-case driven).

- **Actividades:** entrevistas, talleres, elaboración de casos de uso, diagramas de actores
- **Objetivo:** asegurar satisfacción de necesidades reales
- **Entregable:** modelo de requisitos (casos de uso, historias, especificaciones)

1.10.3.2. Análisis

Comprender problema y solución conceptualmente sin detalles técnicos de implementación.

- **Actividades:** identificación de clases, relaciones, responsabilidades, subsistemas
- **Objetivo:** modelo analítico abstracto del sistema
- **Entregable:** modelo de análisis (diagramas de clases conceptuales, secuencia de alto nivel)

1.10.3.3. Diseño

Definir arquitectura de software y decisiones técnicas para implementación. Transformar modelo de análisis en modelo detallado.

- **Actividades:** diseño de clases detallado, interfaces, patrones, organización en capas
- **Objetivo:** garantizar sistema robusto, escalable y mantenible
- **Entregable:** modelo de diseño (diagramas UML detallados: clases, componentes, despliegue)

1.10.3.4. Implementación

Convertir diseño en código ejecutable. Clases y componentes se implementan en lenguaje específico.

- **Actividades:** programación, integración de módulos, uso de frameworks
- **Objetivo:** construir sistema incrementalmente con funcionalidades completas por iteración
- **Entregable:** código fuente, ejecutables, configuraciones

1.10.3.5. Pruebas

Asegurar calidad verificando cumplimiento de requisitos. Pruebas multinivel: unitarias, integración, sistema, aceptación.

- **Actividades:** diseño de casos de prueba, ejecución automatizada/manual, validación contra casos de uso
- **Objetivo:** detectar errores tempranamente, garantizar fiabilidad, aumentar confianza del cliente
- **Entregable:** reportes de pruebas, métricas de calidad, software validado

2. PUDS (PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE)

2.1 CAPTURA DE REQUISITOS - CICLO #1

2.1.1 Identificar Actores Y Casos de Usos

-2.1.1.1 Identificar Actores

ID	ACTOR	DESCRIPCIÓN
A1	Ingeniero Anfitrión	Ingeniero de software responsable de crear el workspace de diseño de base de datos e invitar colaboradores. Actúa como propietario del proyecto con total control. Corresponde al rol OWNER.
A2	Ingeniero Colaborador	Ingeniero de software invitado al diseño colaborativo con permisos de edición del modelo conceptual. Puede modificar el diagrama pero no gestionar miembros. Corresponde al rol EDITOR.

2.1.1.2 Identificar Casos de Uso

ID	NOMBRE	ACTORES	DESCRIPCIÓN
CU-01	Gestionar acceso al sistema	A1, A2	Permite a los ingenieros registrarse como nuevos usuarios, autenticarse con credenciales existentes y finalizar sesión de forma segura. Implementa autenticación mediante JWT y encriptación bcrypt para proteger credenciales.
CU-02	Administrar proyectos de bases de datos	A1, A2	Permite visualizar todos los proyectos de diseño de bases de datos disponibles para el ingeniero, tanto los creados como anfitrión como aquellos donde participa como

			colaborador, mostrando roles y cantidad de diagramas conceptuales.
CU-03	Construir modelo conceptual de base de datos	A1	Permite crear un nuevo diagrama de clases UML mediante un editor gráfico interactivo dentro de un workspace. El ingeniero puede dibujar y modelar visualmente las entidades donde cada clase UML representa el modelo conceptual de una tabla candidata de la base de datos con sus atributos (campos), métodos y relaciones (claves foráneas). Inicializa canvas de React Flow con estructura vacía lista para diseñar la persistencia arrastrando elementos desde una paleta lateral.
CU-04	Sincronizar diseño colaborativo en tiempo real	A1, A2	Permite que múltiples ingenieros distribuidos geográficamente diseñen simultáneamente el modelo conceptual de base de datos. Sincroniza en tiempo real mediante WebSockets todos los cambios (agregar entidades, atributos, claves, relaciones, cardinalidades) como si estuvieran frente a una pizarra física compartida.
CU-05	Incorporar ingenieros al diseño colaborativo	A1	Permite al ingeniero anfitrión invitar otros ingenieros al workspace de diseño asignándoles roles específicos (EDITOR o VIEWER). Establece permisos de acceso y habilita la colaboración distribuida en el modelado de la base de datos sin necesidad de reunirlos físicamente.
CU-06	Remover modelos conceptuales del proyecto	A1	Permite al ingeniero anfitrión eliminar diagramas conceptuales del workspace mediante soft delete (isActive=false), preservando datos para auditoría pero removiéndolos de vistas normales. Útil para cerrar sesiones de diseño o archivar modelos obsoletos.

CU-07	Generar backend completo con ORM	A1, A2	Transforma el modelo conceptual (diagrama de clases) en un proyecto Spring Boot completo con 4 capas (Model, Repository, Service, Controller). Genera entidades JPA con anotaciones @Entity, @Table, @Column que permiten crear automáticamente la base de datos mediante Hibernate ORM. Incluye configuración application.properties con conexión a BD PostgreSQL/MySQL. Entrega proyecto funcional en ZIP descargable.
CU-08	Construir modelo mediante asistente inteligente	A1, A2	Acelera el diseño de la base de datos permitiendo que la IA (Gemini) genere automáticamente el modelo conceptual a partir de descripciones en lenguaje natural o templates predefinidos (E-commerce, Sistema Educativo, Hospital). Crea entidades, atributos, tipos de datos, claves primarias/foráneas y relaciones de forma inteligente.
CU-09	Establecer workspace de diseño	A1	Permite al ingeniero anfitrión crear un nuevo workspace (proyecto de base de datos) con nombre y descripción. Actúa como contenedor principal donde se crearán los modelos conceptuales y se gestionarán los colaboradores. Es el equivalente a "levantar la pizarra" para iniciar una sesión de diseño colaborativo.
CU-10	Generar colección de pruebas automatizadas	A1, A2	Genera automáticamente una colección completa de Postman con todos los endpoints REST del backend generado. Incluye casos de prueba para operaciones CRUD (Create, Read, Update, Delete) de cada entidad, ejemplos de requests con formato JSON, variables de entorno y tests básicos de validación. Facilita probar la funcionalidad sin configuración manual.

CU-11	Generar frontend completo con Flutter	A1, A2	Crea un proyecto Flutter listo para producción: arquitectura limpia, go_router , Riverpod, cliente API (dio /OpenAPI), auth, theming M3, responsive, i18n, almacenamiento, tests y CI. Entrega README, íconos/splash, APK y ZIP descargable.
-------	---------------------------------------	--------	---

2.1.1.3 Priorizar Casos de Uso

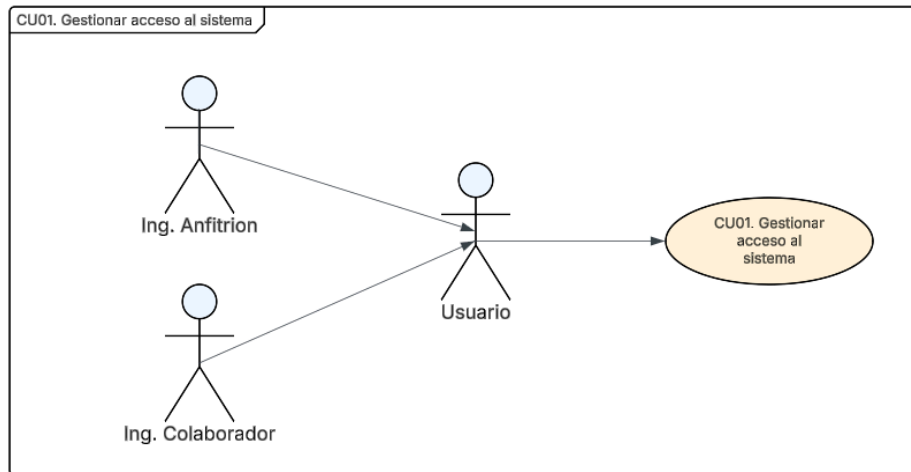
CICLO #1

ID	NOMBRE	ACTORES	PRIORIDAD
CU-01	Gestionar acceso al sistema	A1, A2	ALTA
CU-09	Establecer workspace de diseño	A1	ALTA
CU-03	Construir modelo conceptual de base de datos	A1	ALTA
CU-04	Sincronizar diseño colaborativo en tiempo real	A1, A2	ALTA

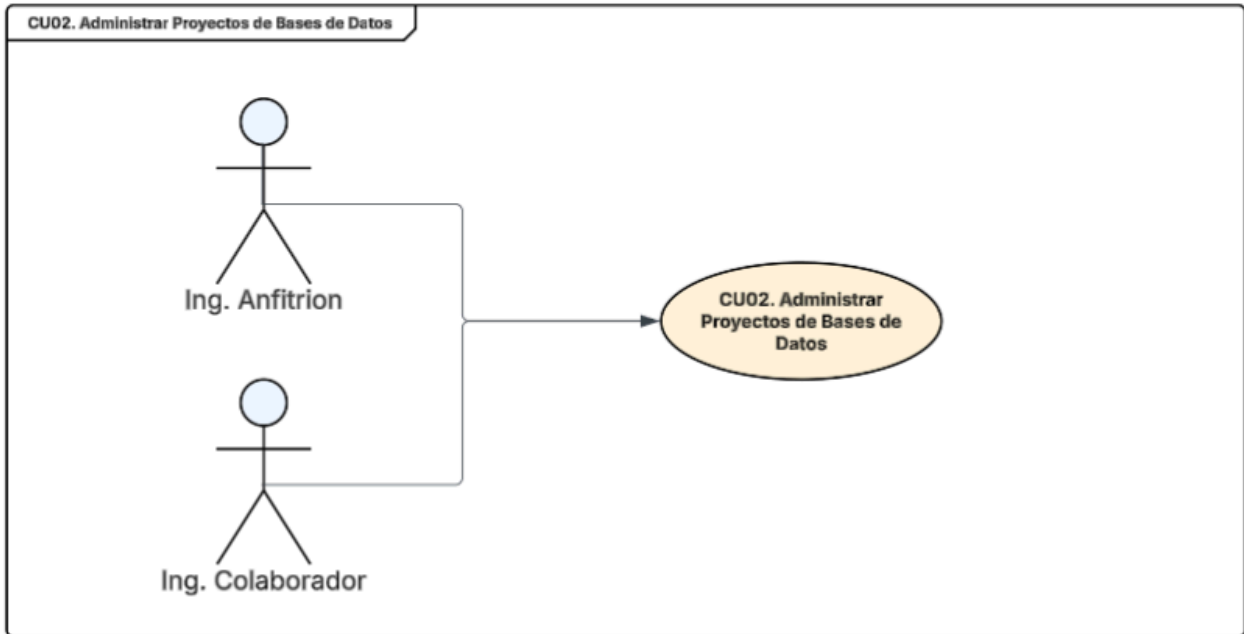
CU-02	Administrar proyectos de bases de datos	A1, A2	ALTA
CU-07	Generar backend completo con ORM	A1, A2	ALTA
CU-08	Construir modelo mediante asistente inteligente	A1, A2	MEDIA
CU-05	Incorporar ingenieros al diseño colaborativo	A1	MEDIA
CU-10	Generar colección de pruebas automatizadas	A1, A2	MEDIA
CU-06	Remover modelos conceptuales del proyecto	A1	MEDIA
CU-11	Generar Frontend completo con Flutter	A1, A2	ALTA

2.1.2 Especificar Casos de Uso

Ciclo #1

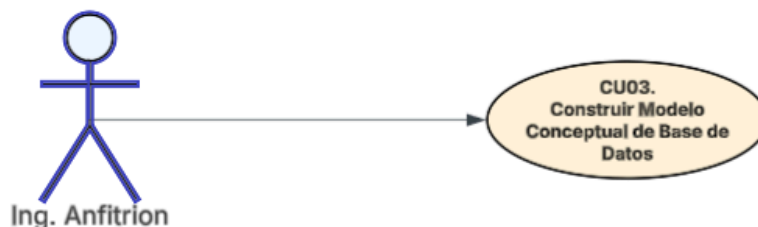


CU01. Gestionar Acceso al Sistema	
Nombre de CU	CU01. Gestionar Acceso al Sistema
Propósito	Que los ingenieros puedan registrarse, iniciar sesión y cerrar sesión en la plataforma.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador
Actor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	Para registro: usuario sin cuenta. Para login: usuario registrado. Para logout: usuario autenticado.
Flujo de trabajo	REGISTRO: • Usuario accede a /register • Completa formulario (email, username, password) • Sistema valida con class-validator • Sistema encripta password con bcrypt • Sistema crea usuario en PostgreSQL • Sistema genera JWT y almacena en localStorage • Sistema redirige al Dashboard LOGIN: • Usuario ingresa credenciales en /login • Sistema valida con Passport • Sistema verifica password • Sistema genera JWT y actualiza Zustand • Sistema redirige al Dashboard LOGOUT: • Usuario hace clic en cerrar sesión • Sistema elimina JWT de localStorage • Sistema limpia estado Zustand • Sistema cierra WebSockets • Sistema redirige a /login
Postcondición	Usuario registrado/autenticado o sesión cerrada correctamente.
Excepción	Credenciales inválidas, email duplicado, error de conexión, JWT expirado.



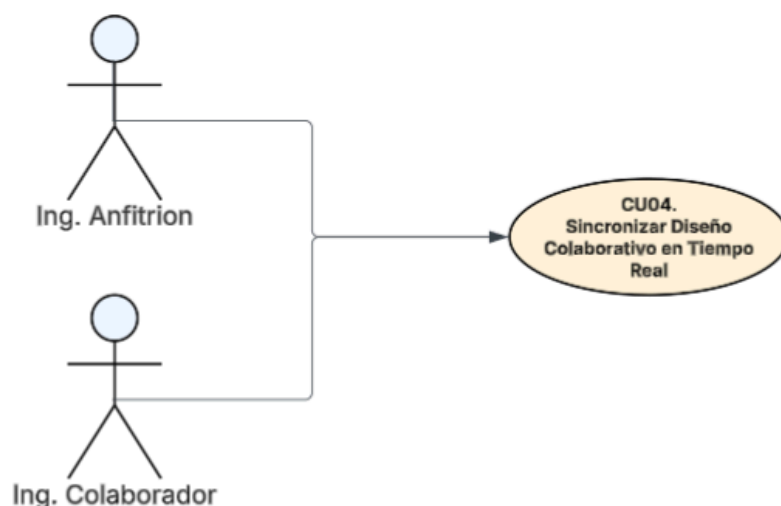
CU02. Administrar Proyectos de Bases de Datos	
Nombre de CU	CU02. Administrar Proyectos de Bases de Datos
Propósito	Que los ingenieros visualicen todos sus workspaces de diseño de bases de datos.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador
Autor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	CU01: Usuario autenticado. Usuario en Dashboard.
Flujo de trabajo	Visualización de Workspaces: • Usuario accede al Dashboard • Sistema extrae userId del JWT • Sistema consulta PostgreSQL con Prisma (workspaces propios y colaborativos) • Sistema organiza en "Mis Workspaces" y "Workspaces Compartidos" • Usuario visualiza lista con: nombre, rol, cantidad de diagramas • Usuario selecciona workspace para acceder
Postcondición	Usuario visualiza todos sus workspaces disponibles.
Excepción	Error de consulta a base de datos, JWT inválido, workspace sin permisos.

CU03. Construir Modelo Conceptual de Base de Datos



CU03. Construir Modelo Conceptual de Base de Datos	
Nombre de CU	CU03. Construir Modelo Conceptual de Base de Datos
Propósito	Que el anfitrión pueda crear diagramas de clases UML para diseñar bases de datos.
Actores	Ingeniero Anfitrión
Actor principal	Ingeniero Anfitrión
Precondición	CU01: Usuario autenticado. CU09: Workspace creado.
Flujo de trabajo	Creación de Diagrama: • Usuario hace clic en "Nuevo Diagrama" dentro del workspace • Sistema muestra modal con campos: nombre (requerido), descripción (opcional) • Usuario completa datos y confirma • Sistema crea registro en tabla Diagram con model JSON vacío, version=1, isActive=true • Sistema registra actividad en DiagramActivity (CREATED) • Sistema redirige al editor UML con canvas React Flow vacío • Usuario comienza a diseñar arrastrando clases desde paleta lateral
Postcondición	Diagrama creado y editor UML disponible para diseño.
Excepción	Nombre vacío, error al crear diagrama, workspace no encontrado.

CU04. Sincronizar Diseño Colaborativo en Tiempo Real



CU04. Sincronizar Diseño Colaborativo en Tiempo Real

Nombre de CU	CU04. Sincronizar Diseño Colaborativo en Tiempo Real
Propósito	Que múltiples ingenieros distribuidos geográficamente diseñen simultáneamente el modelo conceptual.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador
Actor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	CU03: Diagrama creado. Usuario con permisos OWNER o EDITOR. Uno o más usuarios conectados al diagrama.
Flujo de trabajo	Sincronización en Tiempo Real: • Usuario modifica diagrama en React Flow (agregar clases, atributos, métodos, relaciones, mover elementos) • Sistema actualiza estado local React inmediatamente • Sistema emite evento Socket.IO "diagram_change" con: diagramId, userId, changeData, timestamp • Backend recibe evento via WebSocket Gateway • Backend actualiza Diagram: incrementa version++, actualiza model JSON • Backend propaga cambio a todos los clientes en la sala • Backend registra actividad en DiagramActivity • Clientes conectados actualizan canvas automáticamente sin recargar
Postcondición	Diagrama sincronizado en tiempo real entre todos los usuarios conectados.
Excepción	Pérdida de conexión WebSocket, conflicto de versiones, usuario sin permisos de edición.

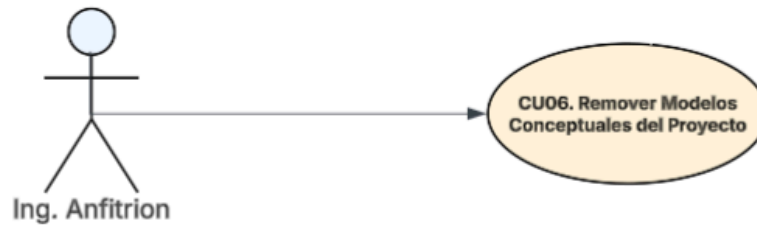
CU05. Incorporar Ingenieros al Diseño Colaborativo



CU05. Incorporar Ingenieros al Diseño Colaborativo

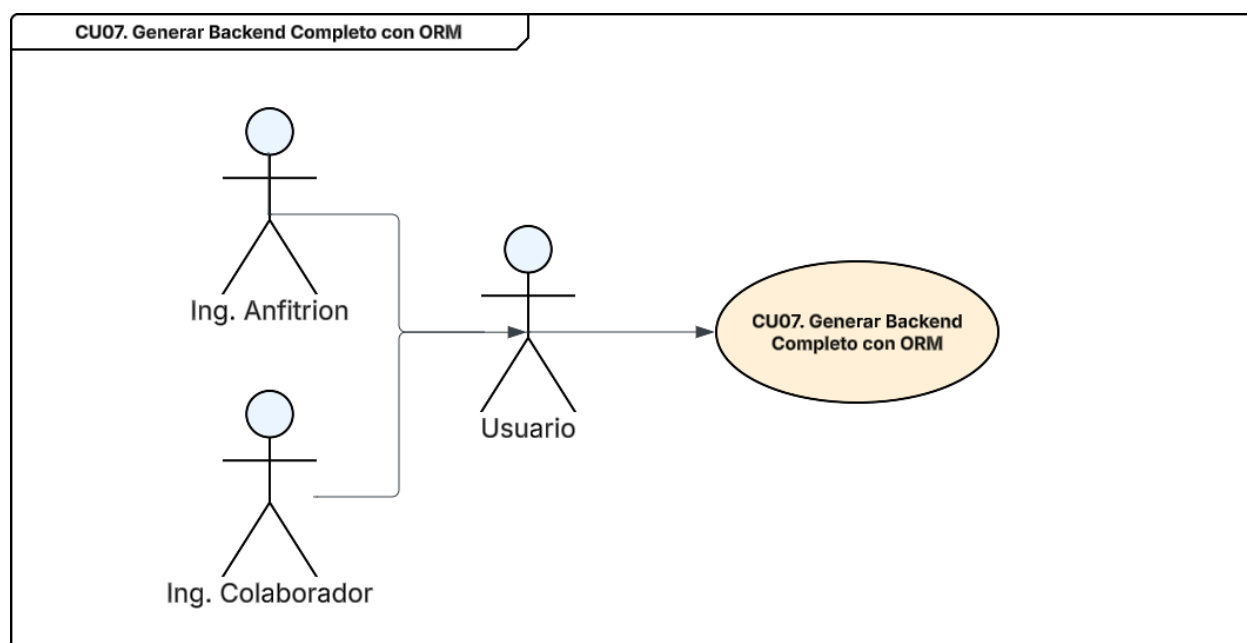
Nombre de CU	CU05. Incorporar Ingenieros al Diseño Colaborativo
Propósito	Que el anfitrión invite otros ingenieros al workspace para colaborar.
Actores	Ingeniero Anfitrión
Actor principal	Ingeniero Anfitrión
Precondición	CU01: Usuario autenticado con rol OWNER. CU09: Workspace creado.
Flujo de trabajo	Invitación de Colaborador: • Usuario hace clic en "Invitar Colaborador" • Sistema muestra modal con campos: email, rol (EDITOR/VIEWER) • Usuario ingresa email y selecciona rol • Sistema valida formato de email • Sistema busca usuario en tabla User • Si existe: crea registro en WorkspaceCollaborator con workspaceId, userId, role, invitedBy • Si no existe: muestra "Usuario no encontrado. Debe registrarse primero" • Sistema actualiza lista de colaboradores • Colaborador ve workspace en su dashboard inmediatamente
Postcondición	Colaborador agregado con acceso según rol asignado.
Excepción	Email inválido, usuario no registrado, colaborador ya agregado, error al crear registro.

CU06. Remover Modelos Conceptuales del Proyecto



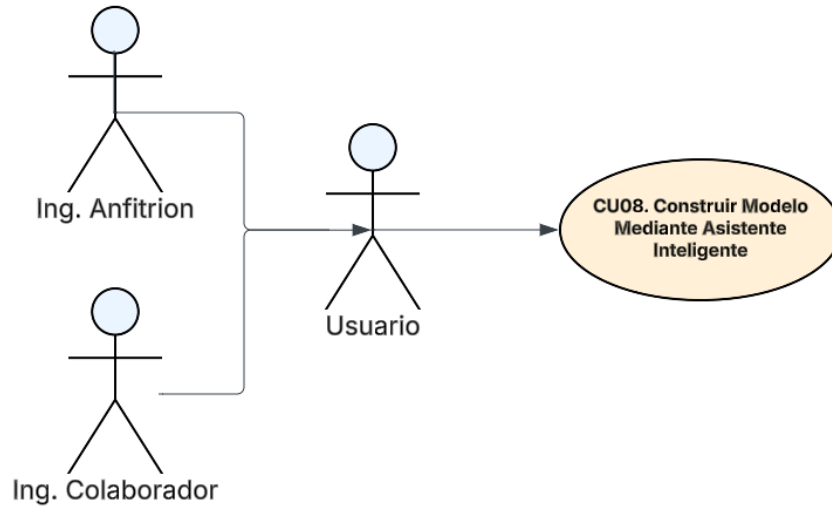
CU06. Remover Modelos Conceptuales del Proyecto

Nombre de CU	CU06. Remover Modelos Conceptuales del Proyecto
Propósito	Que el anfitrión elimine diagramas preservando datos para auditoría.
Actores	Ingeniero Anfitrión
Actor principal	Ingeniero Anfitrión
Precondición	CU01: Usuario con rol OWNER. Diagrama existe y isActive=true.
Flujo de trabajo	Eliminación Lógica: • Usuario selecciona diagrama y hace clic en "Eliminar" • Sistema muestra modal de confirmación • Usuario confirma eliminación • Sistema ejecuta soft delete: actualiza isActive=false en Diagram • Sistema registra actividad en DiagramActivity (DELETED) • Sistema notifica via WebSocket a usuarios conectados • Sistema remueve diagrama de la vista
Postcondición	Diagrama marcado como inactivo, preservado en base de datos para auditoría.
Excepción	Usuario sin permisos OWNER, diagrama ya eliminado, error al actualizar registro.



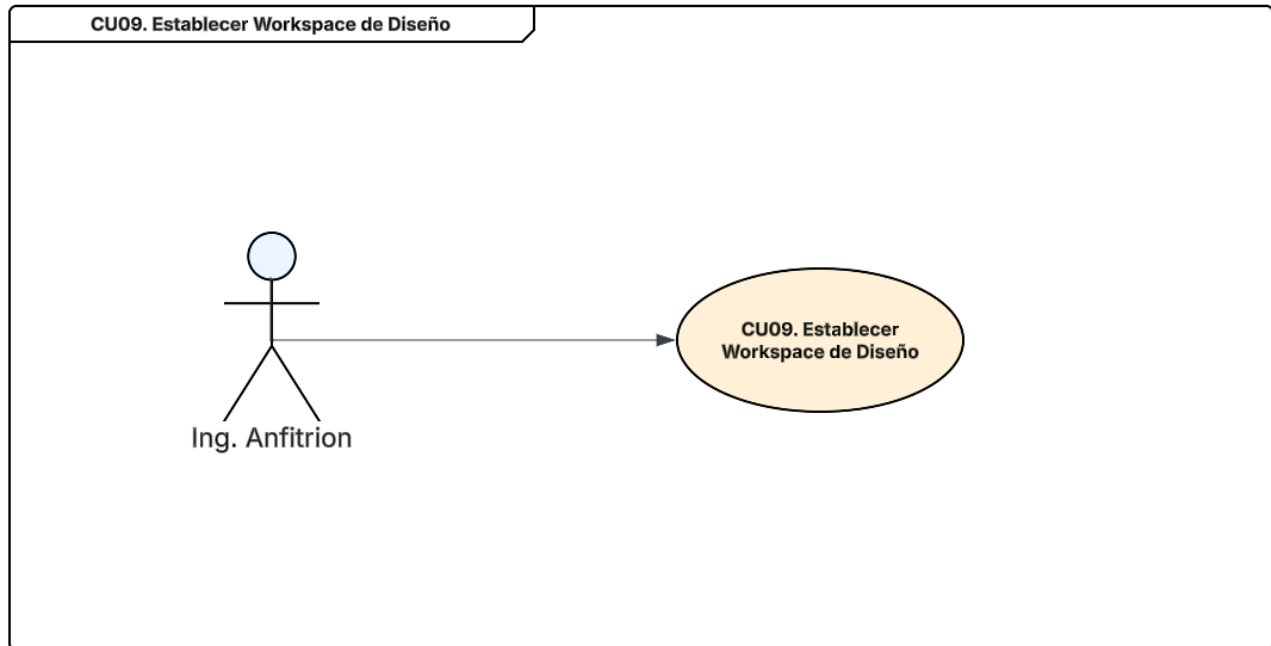
CU07. Generar Backend Completo con ORM	
Nombre de CU	CU07. Generar Backend Completo con ORM
Propósito	Generar un archivo ZIP con código Spring Boot completo que coincida con el diagrama de clases modelado.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador
Actor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	CU03/CU04: Diagrama con clases UML creado. Usuario en página de modelamiento.
Flujo de trabajo	Generación de Código: • Usuario hace clic en "Generar Código" • Sistema muestra panel con opciones: Spring Boot 3.2.0, Maven, package base, Java version • Usuario confirma opciones • Backend obtiene model JSON desde Diagram.model • Backend genera entidades JPA, repositorios JpaRepository, servicios con CRUD, controladores REST • Sistema genera archivos de configuración: pom.xml, application.properties, Application.java • Sistema organiza en estructura Maven y comprime en ZIP • Sistema persiste en GeneratedCode y retorna URL de descarga • Navegador descarga ZIP automáticamente
Postcondición	Proyecto Spring Boot completo con ORM Hibernate que crea base de datos automáticamente.
Excepción	Diagrama vacío, error al generar código, fallo en descarga del archivo ZIP.

CU08. Construir Modelo Mediante Asistente Inteligente



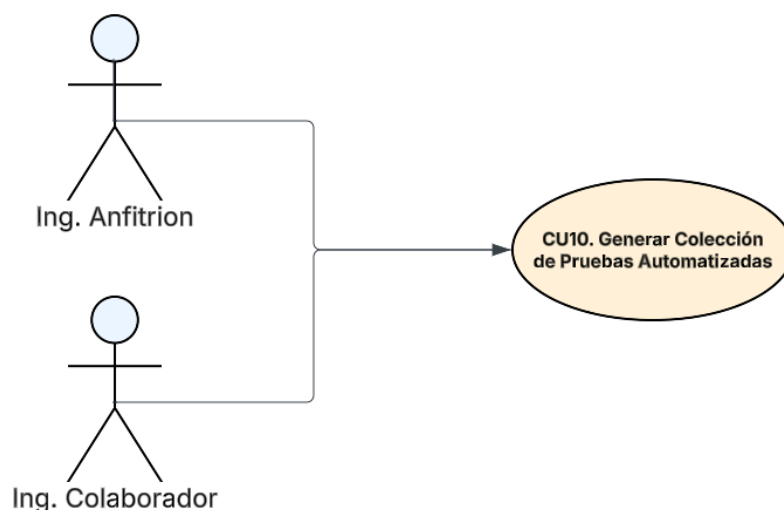
CU08. Construir Modelo Mediante Asistente Inteligente

Nombre de CU	CU08. Construir Modelo Mediante Asistente Inteligente
Propósito	Acelerar el diseño usando inteligencia artificial que genera y refina el modelo conceptual automáticamente.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador, Sistema de IA (Gemini)
Actor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	CU01: Usuario autenticado. Usuario en editor UML. API key Gemini AI configurada.
Flujo de trabajo	Generación con IA: • Usuario hace clic en "Chat con IA" • Sistema muestra panel lateral de chat • Usuario escribe descripción o selecciona template • Usuario presiona "Generar Diagrama" • Frontend envía request al backend ai-chat • Backend construye prompt optimizado e invoca Gemini AI • Backend procesa respuesta: limpia markdown, parsea JSON, valida estructura • Si válida: sistema aplica modelo al canvas React Flow con layout automático • Si IA no disponible: usa template fallback • Sistema actualiza diagrama con version++ • REFINAMIENTO: Usuario escribe modificaciones, IA actualiza diagrama en tiempo real
Postcondición	Diagrama generado o refinado automáticamente por inteligencia artificial.
Excepción	API key inválida, servicio IA no disponible, respuesta IA con formato incorrecto, timeout en generación.



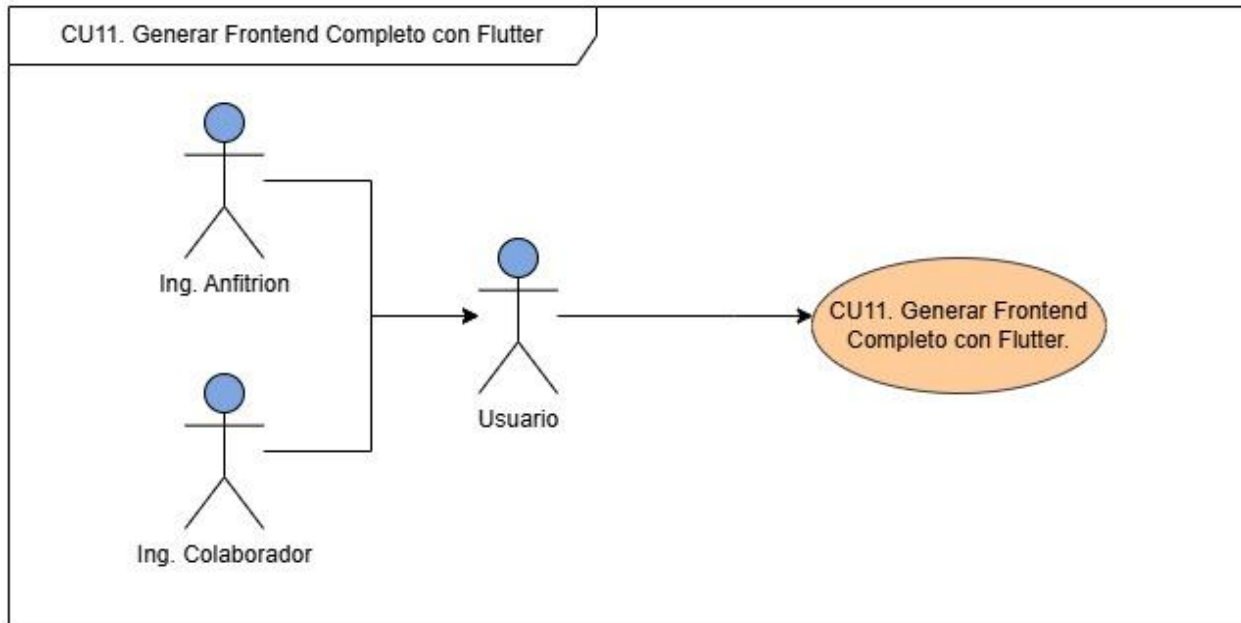
CU09. Establecer Workspace de Diseño	
Nombre de CU	CU09. Establecer Workspace de Diseño
Propósito	Que el anfitrión cree el workspace donde diseñará la base de datos.
Actores	Ingeniero Anfitrión
Actor principal	Ingeniero Anfitrión
Precondición	CU01: Usuario autenticado. Usuario en Dashboard.
Flujo de trabajo	Creación de Workspace: • Usuario hace clic en "Crear Workspace" • Sistema muestra modal con campos: nombre, descripción • Usuario completa datos y confirma • Sistema valida nombre no vacío • Sistema crea registro en Workspace: name, description, createdBy=userId, isActive=true • Sistema redirige a vista del workspace • Workspace aparece en "Mis Workspaces" del Dashboard
Postcondición	Workspace creado y disponible para crear diagramas e invitar colaboradores.
Excepción	Nombre vacío, error al crear registro en base de datos, fallo en redirección.

CU10. Generar Colección de Pruebas Automatizadas



CU10. Generar Colección de Pruebas Automatizadas

Nombre de CU	CU10. Generar Colección de Pruebas Automatizadas
Propósito	Generar colección Postman para testear endpoints del backend generado.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador
Actor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	CU07: Backend Spring Boot generado. Usuario visualiza botón "Crear Tests en Postman".
Flujo de trabajo	Generación de Tests: • Usuario hace clic en "Crear Tests en Postman" • Sistema obtiene metadata del proyecto desde GeneratedCode • Sistema analiza entidades JPA generadas • Para cada entidad genera casos de prueba CRUD: GET listar, GET por id, POST crear, PUT actualizar, DELETE eliminar • Sistema genera ejemplos de requests con JSON body • Sistema crea variables de entorno: {{baseUrl}}, {{entityId}} • Sistema genera colección Postman en formato JSON • Sistema retorna archivo .json para descarga • Usuario importa colección en Postman y ejecuta tests
Postcondición	Colección Postman generada con tests CRUD para cada entidad del modelo.
Excepción	Backend no generado previamente, error al analizar entidades, fallo en generación de archivo JSON.

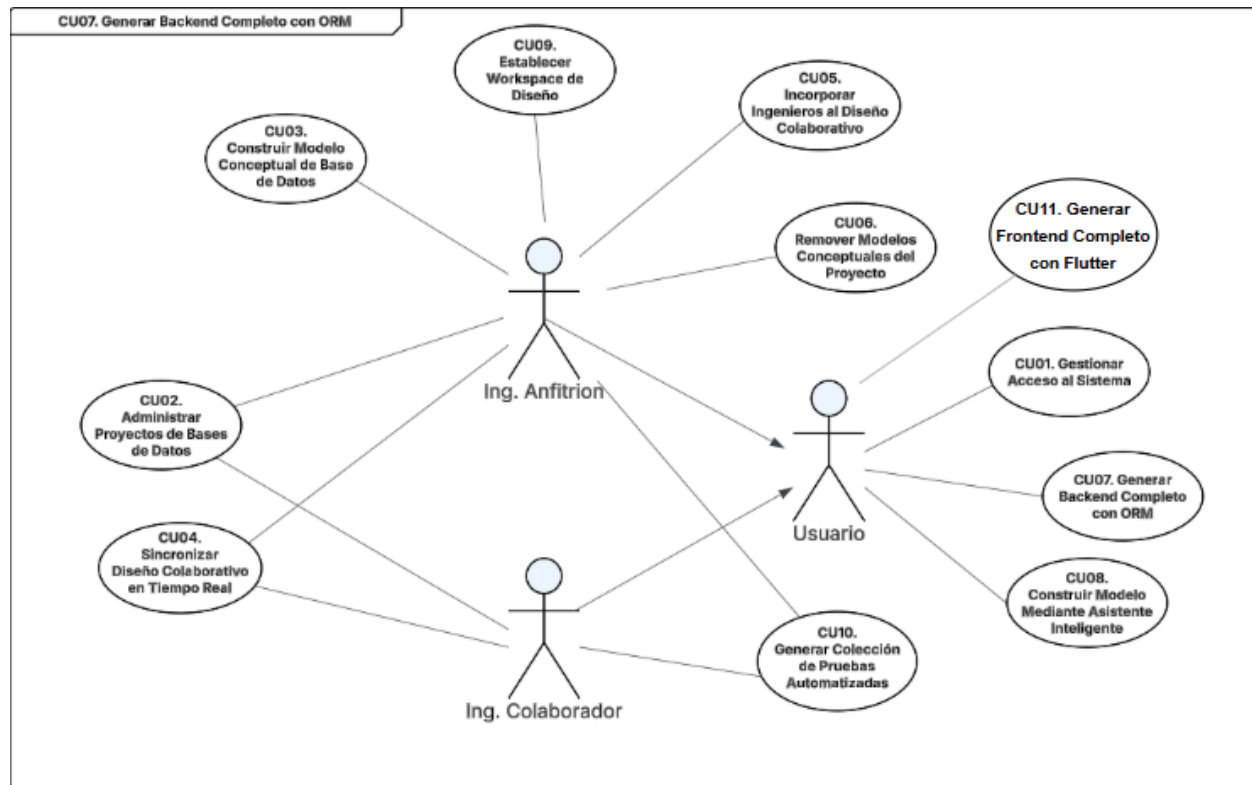


CU11. Generar Frontend Completo con Flutter	
Nombre de CU	CU11. Generar Frontend Completo con Flutter
Propósito	Generar un proyecto Flutter completo y listo para producción, con arquitectura limpia, herramientas modernas y funcionalidades clave, empaquetado en un ZIP descargable.
Actores	Ingeniero Anfitrión, Ingeniero Colaborador
Actor principal	Ingeniero Anfitrión, Ingeniero Colaborador
Precondición	CU03/CU04: Diagrama con clases UML creado. Usuario en página de modelamiento.

Flujo de trabajo	• Usuario hace clic en “Generar Frontend” • Sistema muestra panel con opciones: Flutter 3.x, nombre del proyecto, package base, temas claros/oscuros, idiomas soportados, opciones de autenticación • Usuario configura opciones y confirma • Backend obtiene model JSON desde Diagram model • Backend genera: - Estructura de proyecto Flutter con arquitectura limpia - Navegación con go_router - Gestión de estado con Riverpod - Cliente API con Dio/OpenAPI a partir de entidades del modelo - Módulo de autenticación - Theming con Material Design 3 (M3) - Diseño responsive - Soporte para i18n (internacionalización) - Módulo de almacenamiento local - Tests unitarios y de widget - Configuración básica de CI (GitHub Actions) • Sistema genera archivos adicionales: README detallado, íconos y splash screen personalizables, scripts de build • Sistema compila una APK de ejemplo y organiza todo en estructura Flutter estándar • Sistema comprime el proyecto en ZIP y persiste en GeneratedCode • Sistema retorna URL de descarga • Navegador descarga ZIP automáticamente
Postcondición	Proyecto Flutter completo, listo para abrir, personalizar y compilar para producción.
Excepción	Diagrama vacío, error al generar código, fallo en compilación de APK, error en descarga del ZIP.

2.1.3 Estructurar Modelo de Casos de Uso

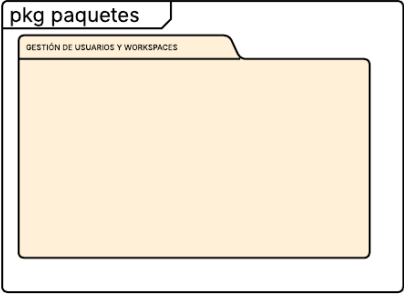
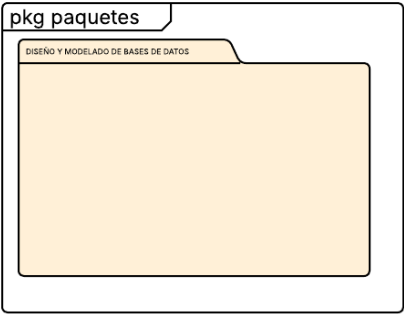
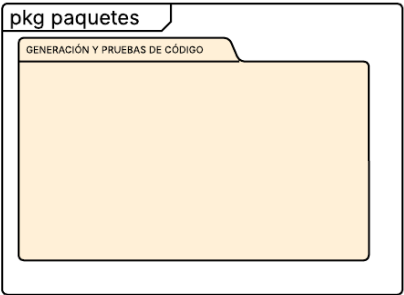
Ciclo #1



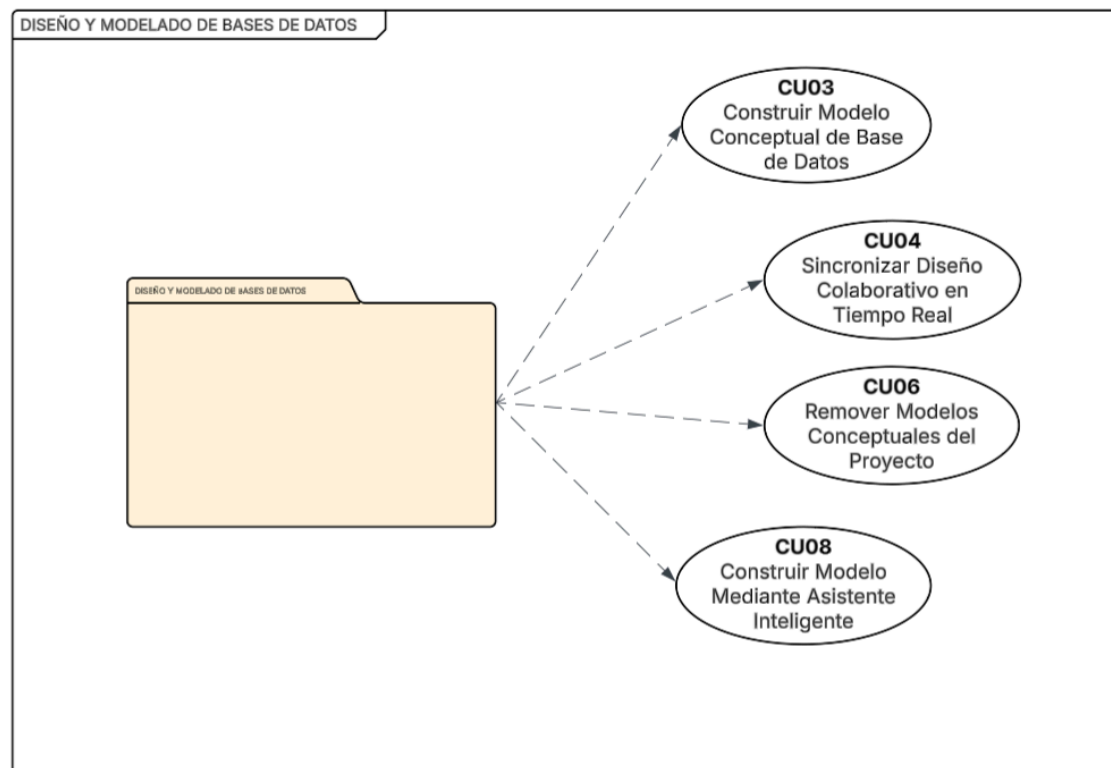
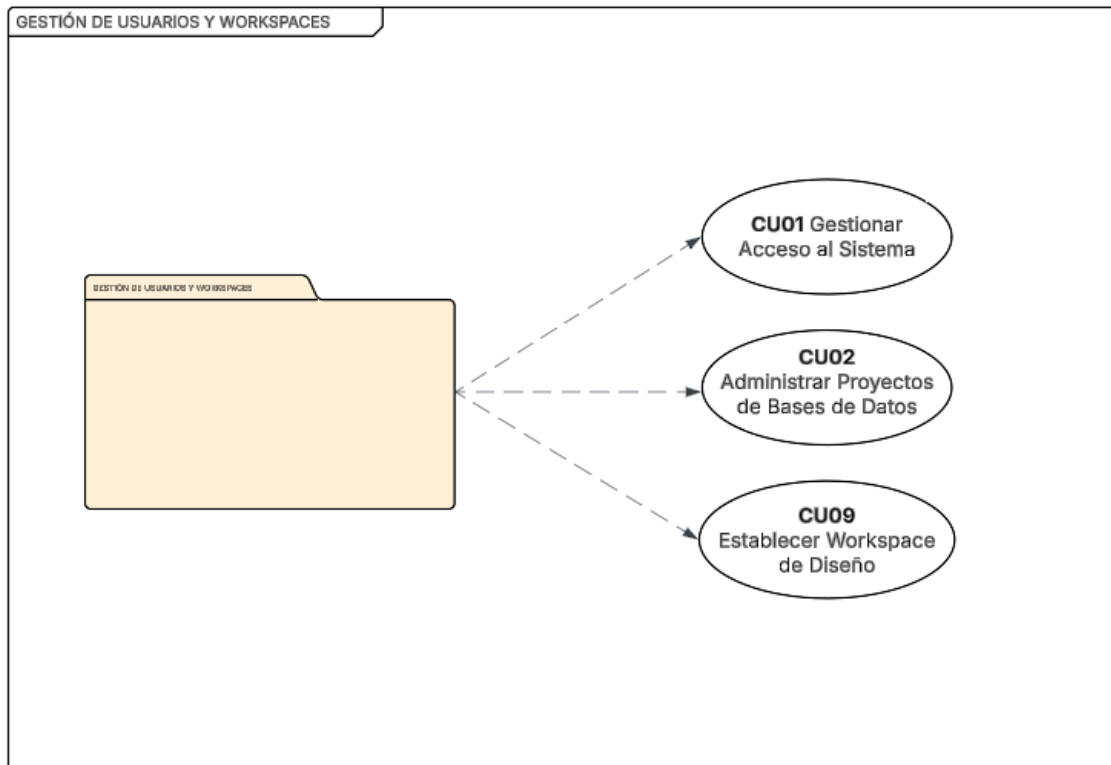
2.2. ANÁLISIS

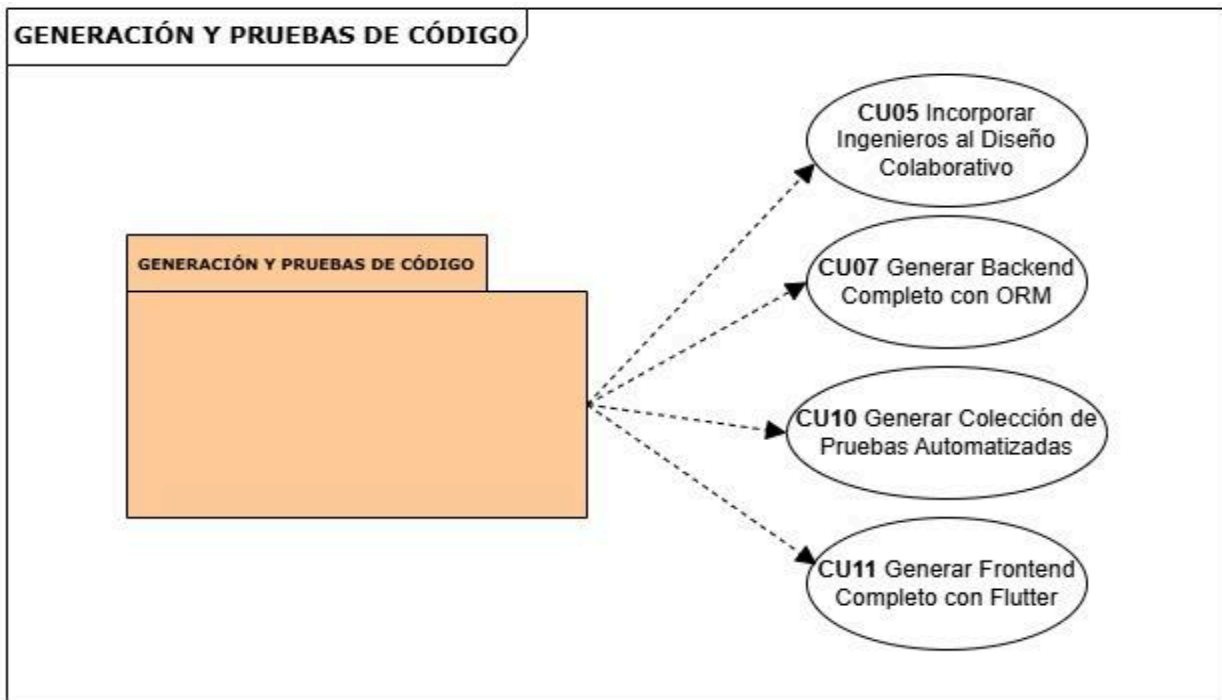
2.2.1 Análisis de la Arquitectura

2.2.1.1 Identificar Paquetes

	Todo lo relativo al ciclo de vida del usuario: registro, login/logout, gestión de perfil y baja de cuenta (soft delete vía isActive). Define políticas de credenciales, sesiones y validaciones de seguridad transversales. Incluye la gestión de workspaces personales y colaborativos (listar, crear, visualizar). Maneja la persistencia de tokens JWT en localStorage y estado global con Zustand. Establece las bases para que los ingenieros accedan y organicen sus proyectos de diseño de bases de datos.
	CRUD y mantenimiento del artefacto principal (Diagrama): listar, crear, ver detalle, editar metadatos, actualizar modelo canónico (JSON) y desactivar/eliminar (soft delete). Establece reglas de versión, validaciones y ownership. Incluye el editor gráfico React Flow para diseñar diagramas de clases UML donde cada clase representa una tabla candidata de la base de datos. Maneja la sincronización en tiempo real mediante WebSockets (Socket.IO) entre múltiples ingenieros distribuidos geográficamente. Integra asistente IA (Gemini) para generar y refinar modelos conceptuales mediante lenguaje natural.
	Flujos de invitaciones (por email o enlace), tokens, expiración/revocación, y la gestión de miembros del diagrama (listar, roles, remover). Cubre seguridad a nivel de permisos y trazabilidad de acceso. Incluye la generación automática de código Spring Boot con 4 capas (Model, Repository, Service, Controller) y configuración ORM (Hibernate) que crea automáticamente la base de datos desde las entidades JPA. Genera colección Postman con casos de prueba CRUD para validar endpoints REST del backend generado. Transforma el modelo conceptual en código ejecutable funcional descargable en ZIP.

2.2.1.2 Relacionar Paquetes y Casos de Uso

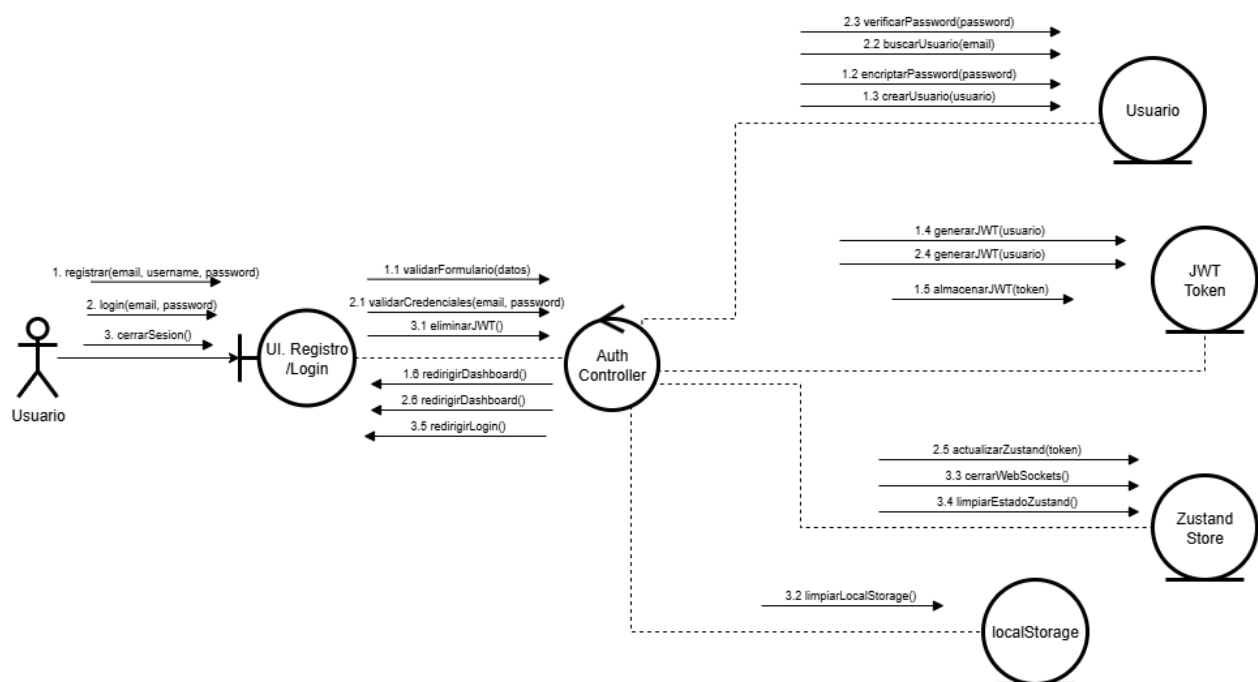




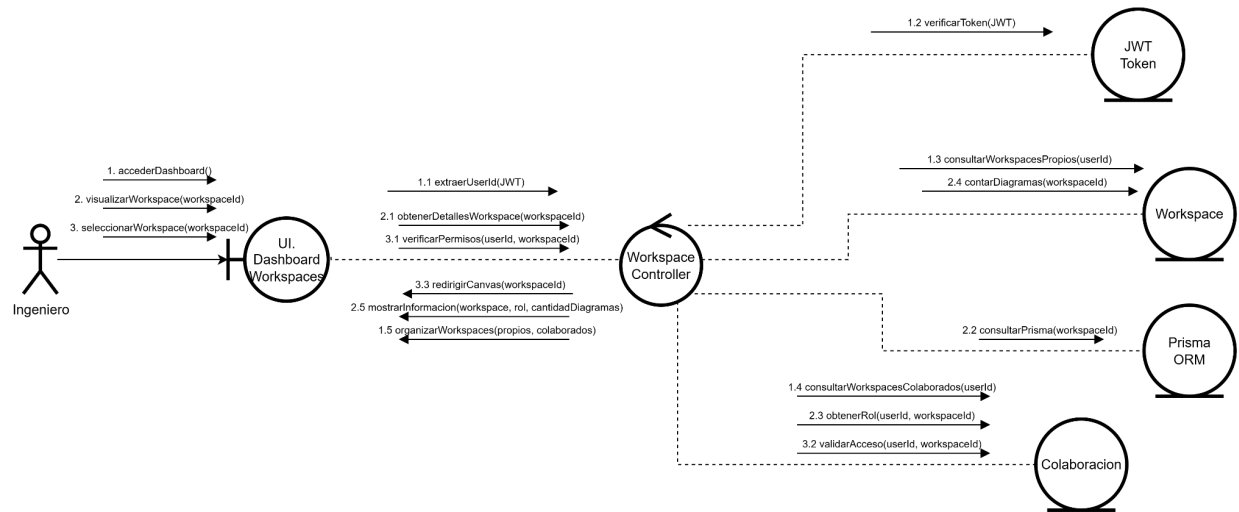
2.2.2 Análisis de casos de uso

2.2.2.1 Diagramas de Colaboración/Comunicación

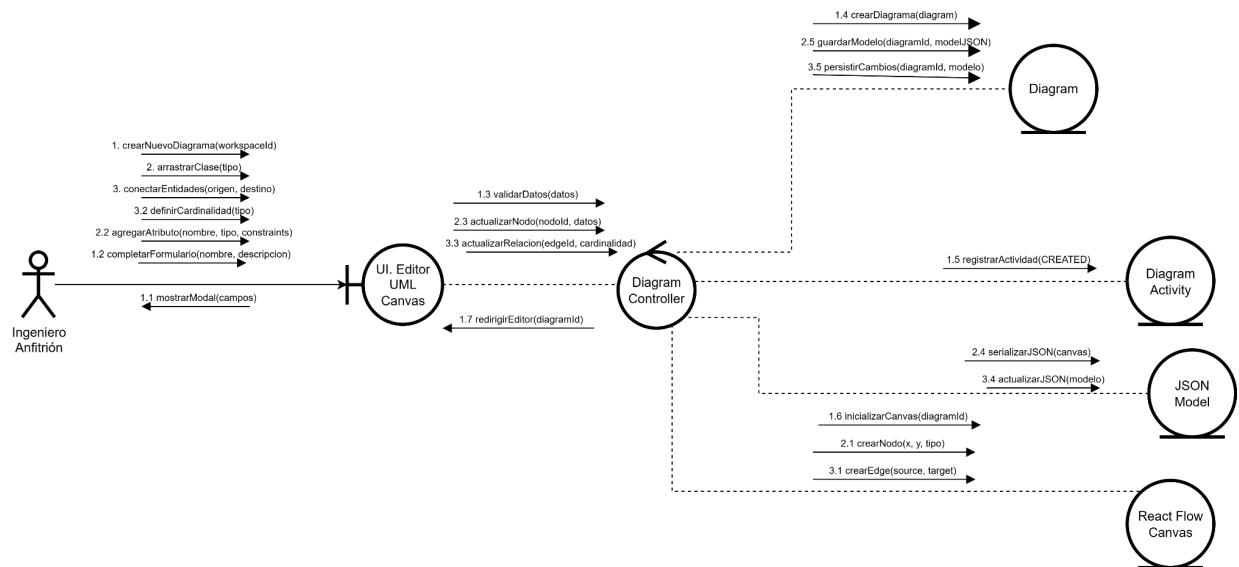
CU1:



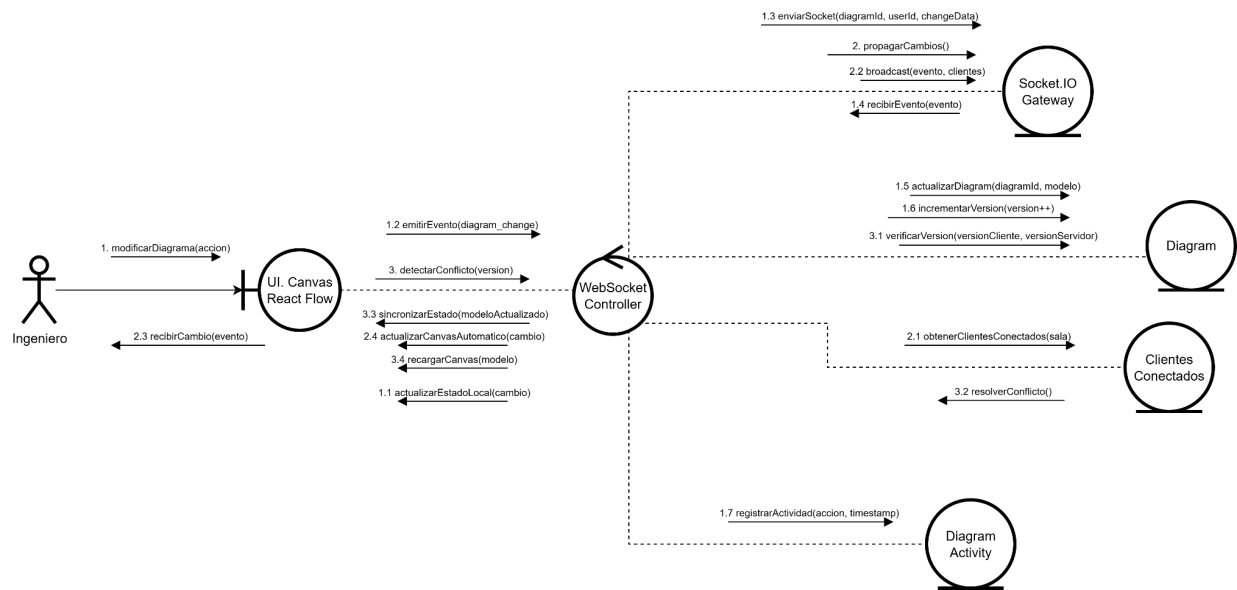
CU2:



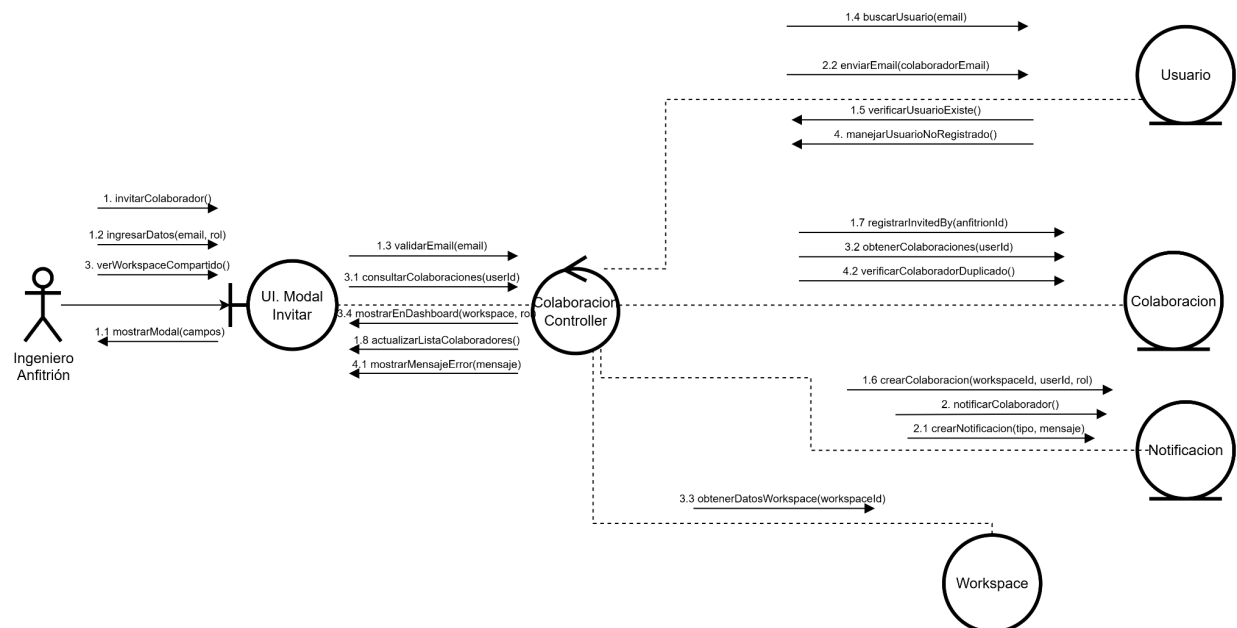
CU3:



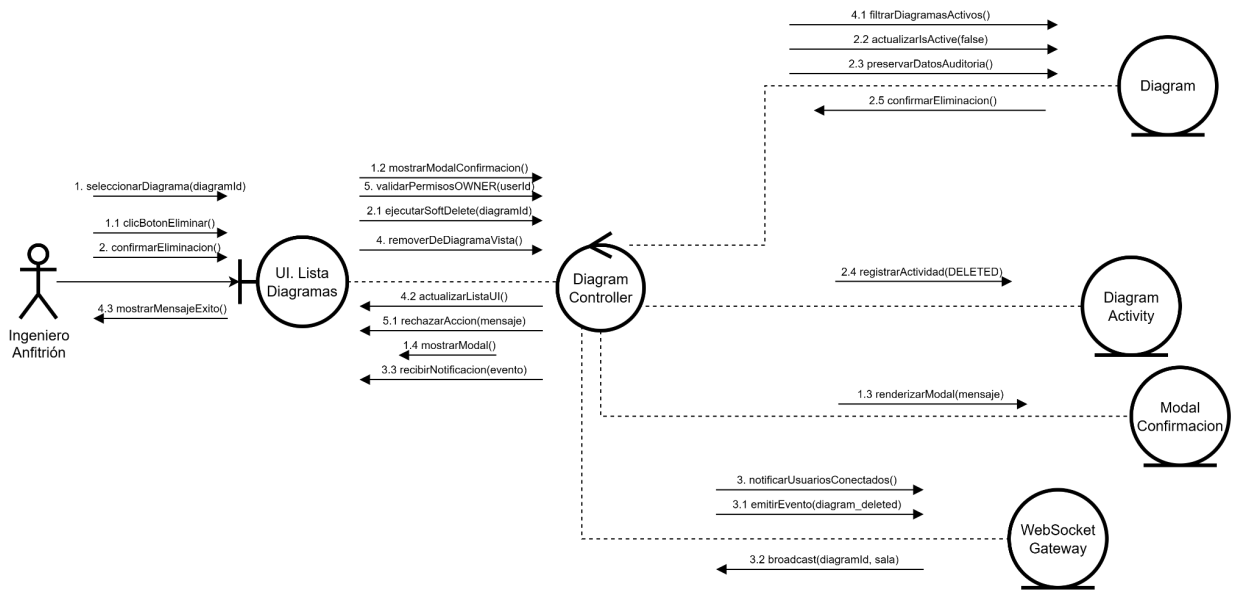
CU4.



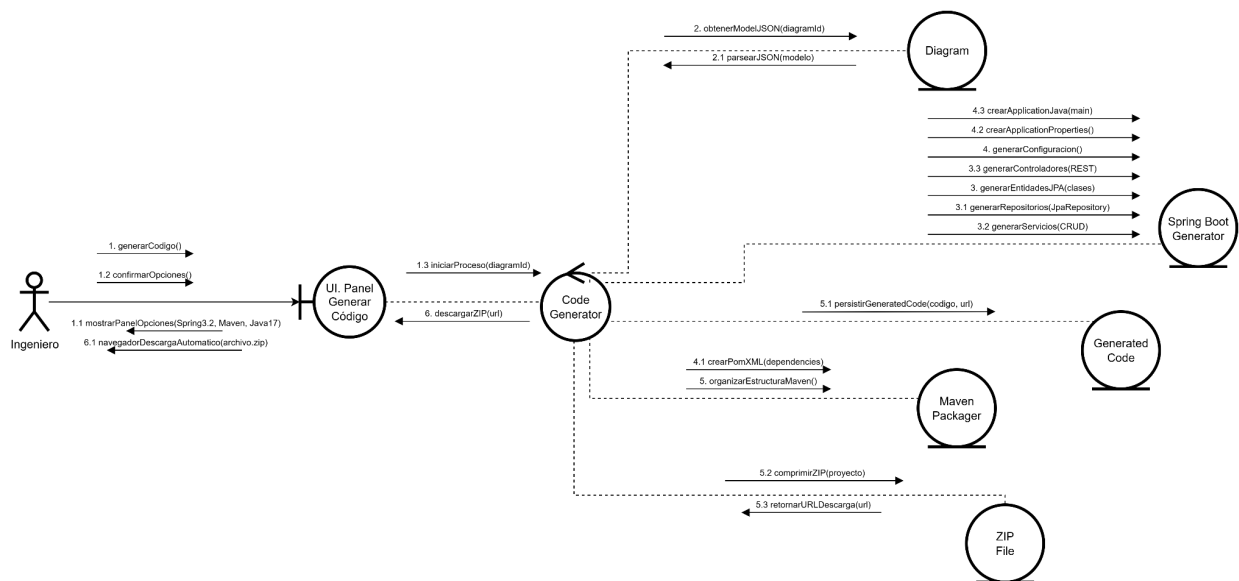
CU5.



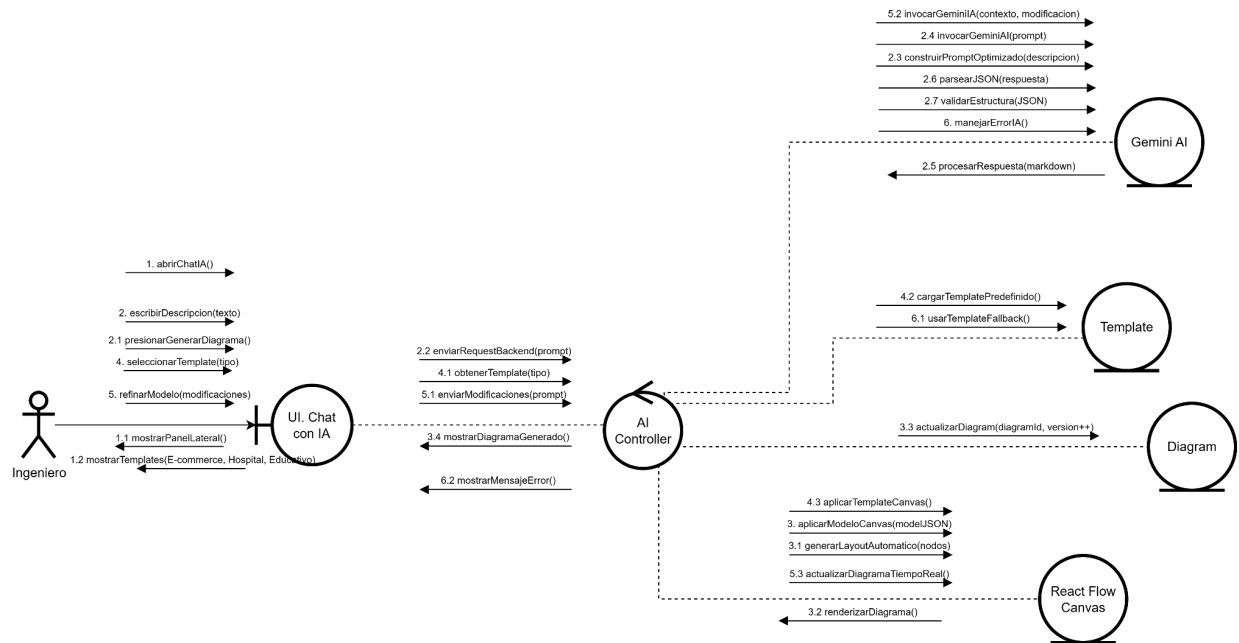
CU6.



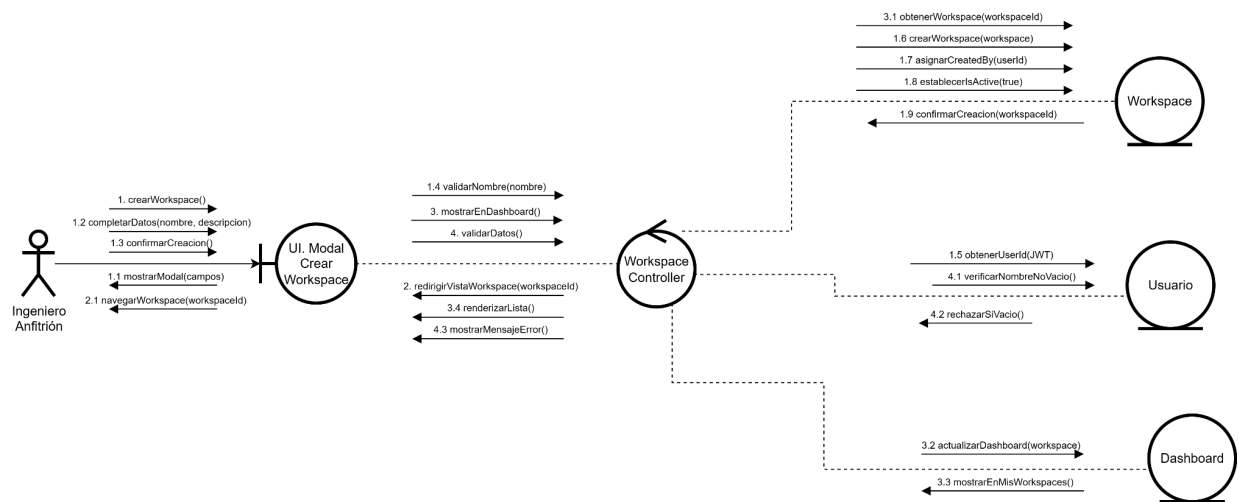
CU7.



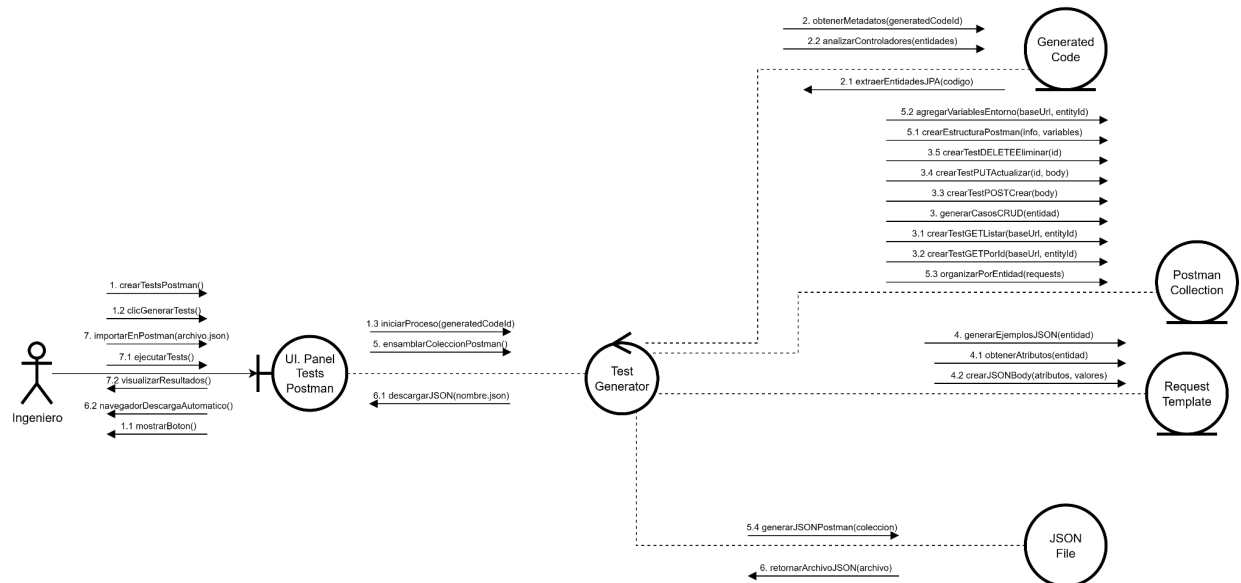
CU8.



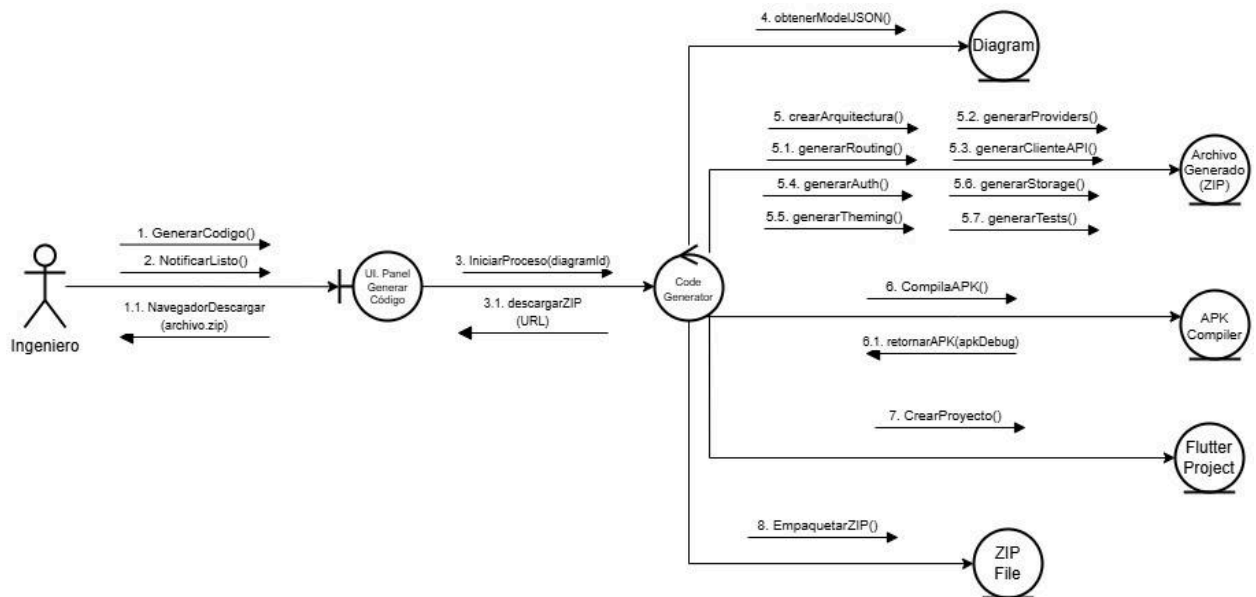
CU9.



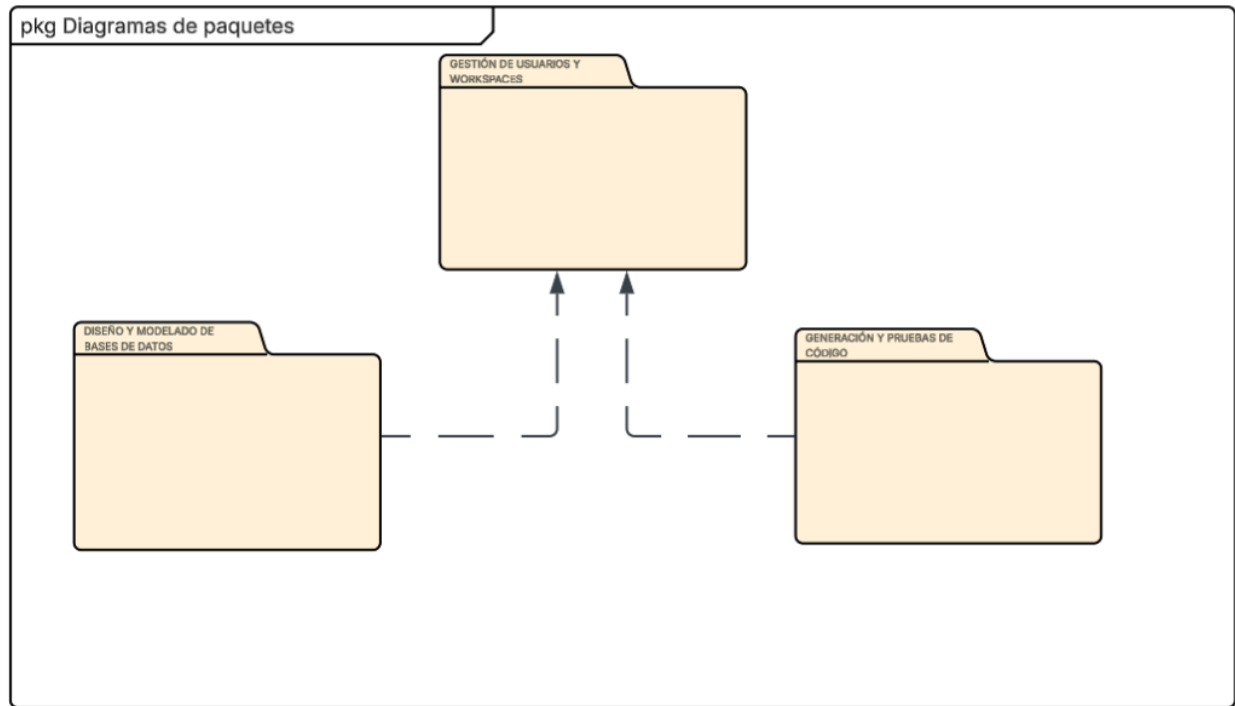
CU10.



CU11



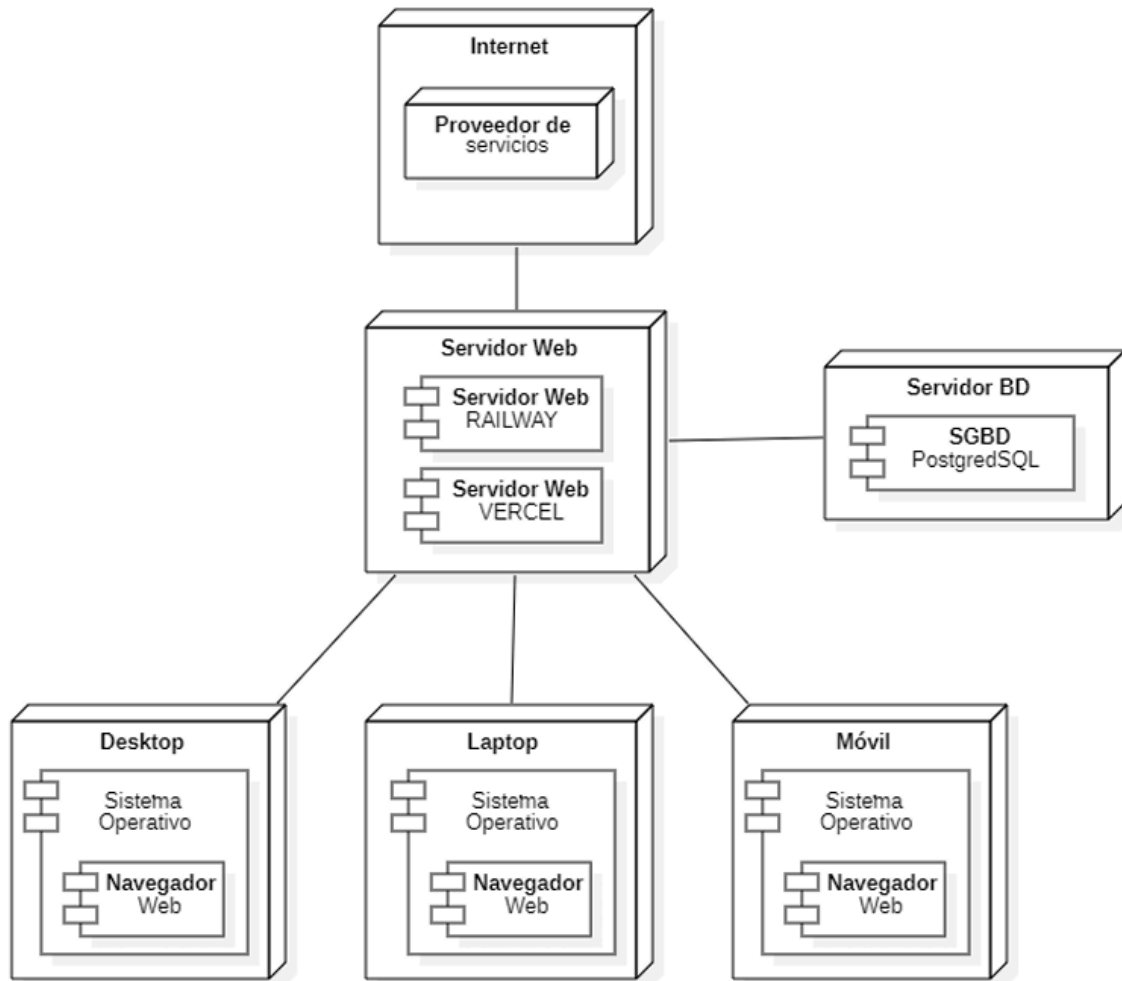
2.2.3 Análisis de Paquetes



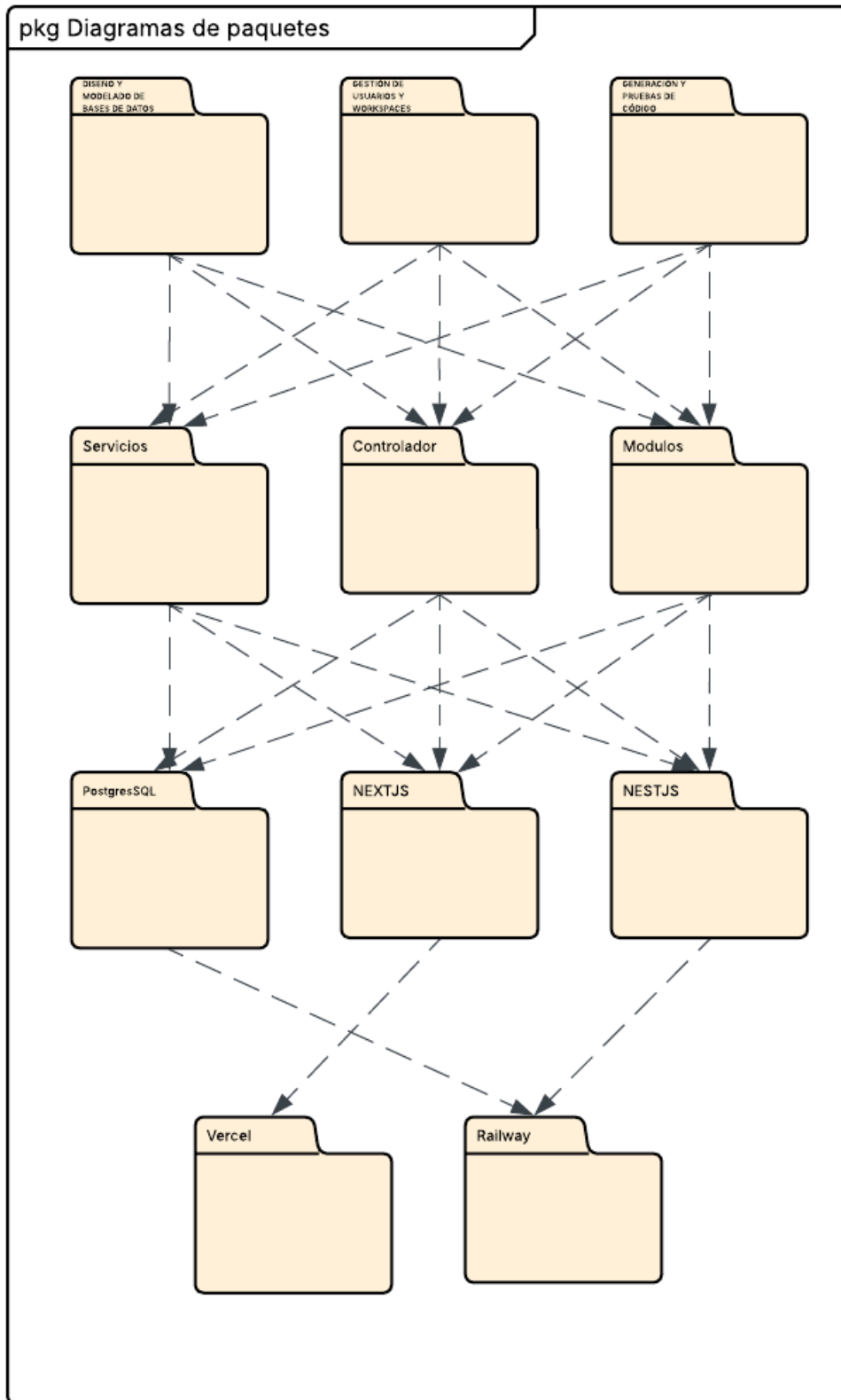
2.3 DISEÑO

2.3.1 Diseño de Arquitectura

2.3.1.1 Diseño Físico



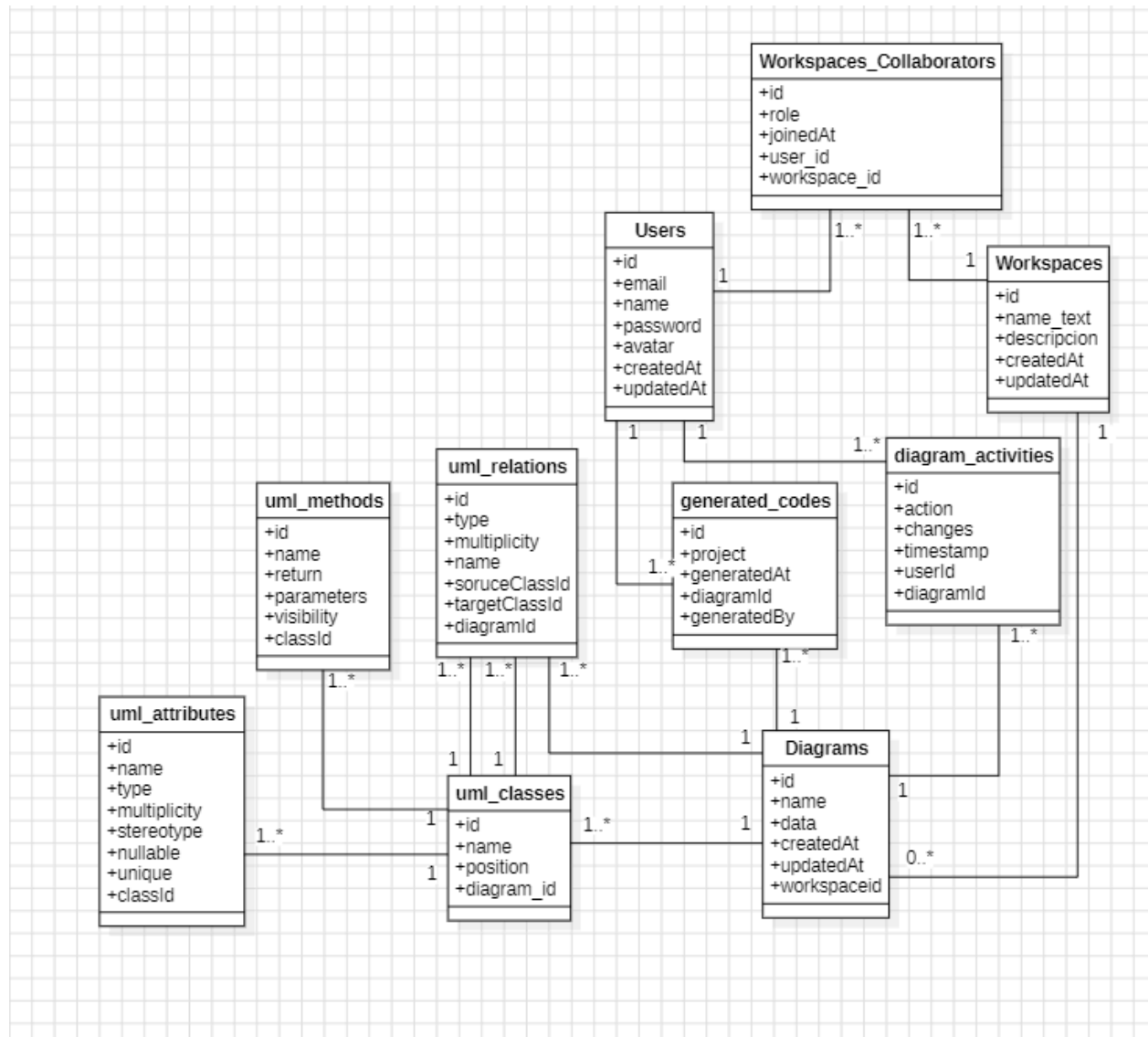
2.3.1.2 Diseño Lógico (Diagrama Organizado en Capas)



2.3.2 Diseño de Datos

2.3.2.1 Diseño Datos Lógicos

2.3.2.1.1 Diagrama de Clases



2.3.2.1.2 Mapeo

Users

PK						
id	email	name	password	avatar	createdAt	updatedAt

Workspaces

PK	FK				
id	ownerId	name	description	createdAt	updatedAt

Workspace_Collaborators

PK	FK	FK		
id	userId	workspaceId	role	joinedAt

Diagrams

PK	FK					
id	workspaceId	name	data	version	createdAt	updatedAt

Uml_Classes

PK	FK		
id	diagramId	name	position

Uml_Attributes

PK	FK						
id	classId	name	type	multiplicity	stereotype	nullable	unique

Uml_Methods

PK	FK				
id	classId	name	returnType	parameters	visibility

Uml_Relations

PK	FK	FK	FK			
id	diagramId	sourceClassId	targetClassId	type	multiplicity	name

Generated_Codes

PK	FK	FK			
id	diagramId	generatedBy	projectType	zipPath	generatedAt

2.3.2.1.3 Normalización

Las Clases ya se encuentran completamente normalizadas.

2.3.2.2 Diseño Datos Físicos

2.3.2.2.1 Script

-- This script was generated by the ERD tool in pgAdmin 4.
-- Please log an issue at <https://github.com/pgadmin-org/pgadmin4/issues/new/choose> if you find any bugs, including reproduction steps.

BEGIN;

```
CREATE TABLE IF NOT EXISTS public._prisma_migrations
(
    id character varying(36) COLLATE pg_catalog."default" NOT NULL,
    checksum character varying(64) COLLATE pg_catalog."default" NOT NULL,
    finished_at timestamp with time zone,
    migration_name character varying(255) COLLATE pg_catalog."default" NOT NULL,
    logs text COLLATE pg_catalog."default",
    rolled_back_at timestamp with time zone,
    started_at timestamp with time zone NOT NULL DEFAULT now(),
    applied_steps_count integer NOT NULL DEFAULT 0,
    CONSTRAINT _prisma_migrations_pkey PRIMARY KEY (id)
);
```

```
CREATE TABLE IF NOT EXISTS public.diagram_activities
(
    id text COLLATE pg_catalog."default" NOT NULL,
    action "ActivityType" NOT NULL,
    changes jsonb NOT NULL DEFAULT '{}::jsonb',
    "timestamp" timestamp(3) without time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "userId" text COLLATE pg_catalog."default" NOT NULL,
    "diagramId" text COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT diagram_activities_pkey PRIMARY KEY (id)
);
```

```
CREATE TABLE IF NOT EXISTS public.diagrams
(
    id text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    data jsonb NOT NULL DEFAULT '{}::jsonb',
    version integer NOT NULL DEFAULT 1,
    "createdAt" timestamp(3) without time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" timestamp(3) without time zone NOT NULL,
```

```

        "workspaceId" text COLLATE pg_catalog."default" NOT NULL,
        CONSTRAINT diagrams_pkey PRIMARY KEY (id)
    );

CREATE TABLE IF NOT EXISTS public.generated_codes
(
    id text COLLATE pg_catalog."default" NOT NULL,
    "projectType" "ProjectType" NOT NULL DEFAULT 'SPRING_BOOT'::"ProjectType",
    "zipPath" text COLLATE pg_catalog."default" NOT NULL,
    "generatedAt" timestamp(3) without time zone NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    "diagramId" text COLLATE pg_catalog."default" NOT NULL,
    "generatedBy" text COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT generated_codes_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.uml_attributes
(
    id text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    type text COLLATE pg_catalog."default" NOT NULL,
    multiplicity text COLLATE pg_catalog."default",
    stereotype text COLLATE pg_catalog."default",
    nullable boolean NOT NULL DEFAULT true,
    "unique" boolean NOT NULL DEFAULT false,
    "classId" text COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT uml_attributes_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.uml_classes
(
    id text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    "position" jsonb NOT NULL DEFAULT '{"x": 0, "y": 0}'::jsonb,
    "diagramId" text COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT uml_classes_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.uml_methods
(
    id text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    "returnType" text COLLATE pg_catalog."default" NOT NULL DEFAULT 'void'::text,
    parameters jsonb NOT NULL DEFAULT '[]'::jsonb,

```

```

visibility text COLLATE pg_catalog."default" NOT NULL DEFAULT 'public'::text,
"classId" text COLLATE pg_catalog."default" NOT NULL,
CONSTRAINT uml_methods_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.uml_relations
(
    id text COLLATE pg_catalog."default" NOT NULL,
    type "RelationType" NOT NULL,
    multiplicity text COLLATE pg_catalog."default",
    name text COLLATE pg_catalog."default",
    "sourceClassId" text COLLATE pg_catalog."default" NOT NULL,
    "targetClassId" text COLLATE pg_catalog."default" NOT NULL,
    "diagramId" text COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT uml_relations_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.users
(
    id text COLLATE pg_catalog."default" NOT NULL,
    email text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    password text COLLATE pg_catalog."default" NOT NULL,
    avatar text COLLATE pg_catalog."default",
    "createdAt" timestamp(3) without time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" timestamp(3) without time zone NOT NULL,
    CONSTRAINT users_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.workspace_collaborators
(
    id text COLLATE pg_catalog."default" NOT NULL,
    role "Role" NOT NULL DEFAULT 'VIEWER'::"Role",
    "joinedAt" timestamp(3) without time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "userId" text COLLATE pg_catalog."default" NOT NULL,
    "workspaceId" text COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT workspace_collaborators_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.workspaces
(
    id text COLLATE pg_catalog."default" NOT NULL,
    name text COLLATE pg_catalog."default" NOT NULL,
    description text COLLATE pg_catalog."default",

```

```
"createdAt" timestamp(3) without time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,  
"updatedAt" timestamp(3) without time zone NOT NULL,  
"ownerId" text COLLATE pg_catalog."default" NOT NULL,  
CONSTRAINT workspaces_pkey PRIMARY KEY (id)  
);
```

```
ALTER TABLE IF EXISTS public.diagram_activities  
  ADD CONSTRAINT "diagram_activities_diagramId_fkey" FOREIGN KEY ("diagramId")  
  REFERENCES public.diagrams (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.diagram_activities  
  ADD CONSTRAINT "diagram_activities_userId_fkey" FOREIGN KEY ("userId")  
  REFERENCES public.users (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.diagrams  
  ADD CONSTRAINT "diagrams_workspaceId_fkey" FOREIGN KEY ("workspaceId")  
  REFERENCES public.workspaces (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.generated_codes  
  ADD CONSTRAINT "generated_codes_diagramId_fkey" FOREIGN KEY ("diagramId")  
  REFERENCES public.diagrams (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.generated_codes  
  ADD CONSTRAINT "generated_codes_generatedBy_fkey" FOREIGN KEY ("generatedBy")  
  REFERENCES public.users (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.uml_attributes  
  ADD CONSTRAINT "uml_attributes_classId_fkey" FOREIGN KEY ("classId")  
  REFERENCES public.uml_classes (id) MATCH SIMPLE
```

```
ON UPDATE CASCADE  
ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.uml_classes  
  ADD CONSTRAINT "uml_classes_diagramId_fkey" FOREIGN KEY ("diagramId")  
  REFERENCES public.diagrams (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.uml_methods  
  ADD CONSTRAINT "uml_methods_classId_fkey" FOREIGN KEY ("classId")  
  REFERENCES public.uml_classes (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.uml_relations  
  ADD CONSTRAINT "uml_relations_diagramId_fkey" FOREIGN KEY ("diagramId")  
  REFERENCES public.diagrams (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.uml_relations  
  ADD CONSTRAINT "uml_relations_sourceClassId_fkey" FOREIGN KEY ("sourceClassId")  
  REFERENCES public.uml_classes (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.uml_relations  
  ADD CONSTRAINT "uml_relations_targetClassId_fkey" FOREIGN KEY ("targetClassId")  
  REFERENCES public.uml_classes (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

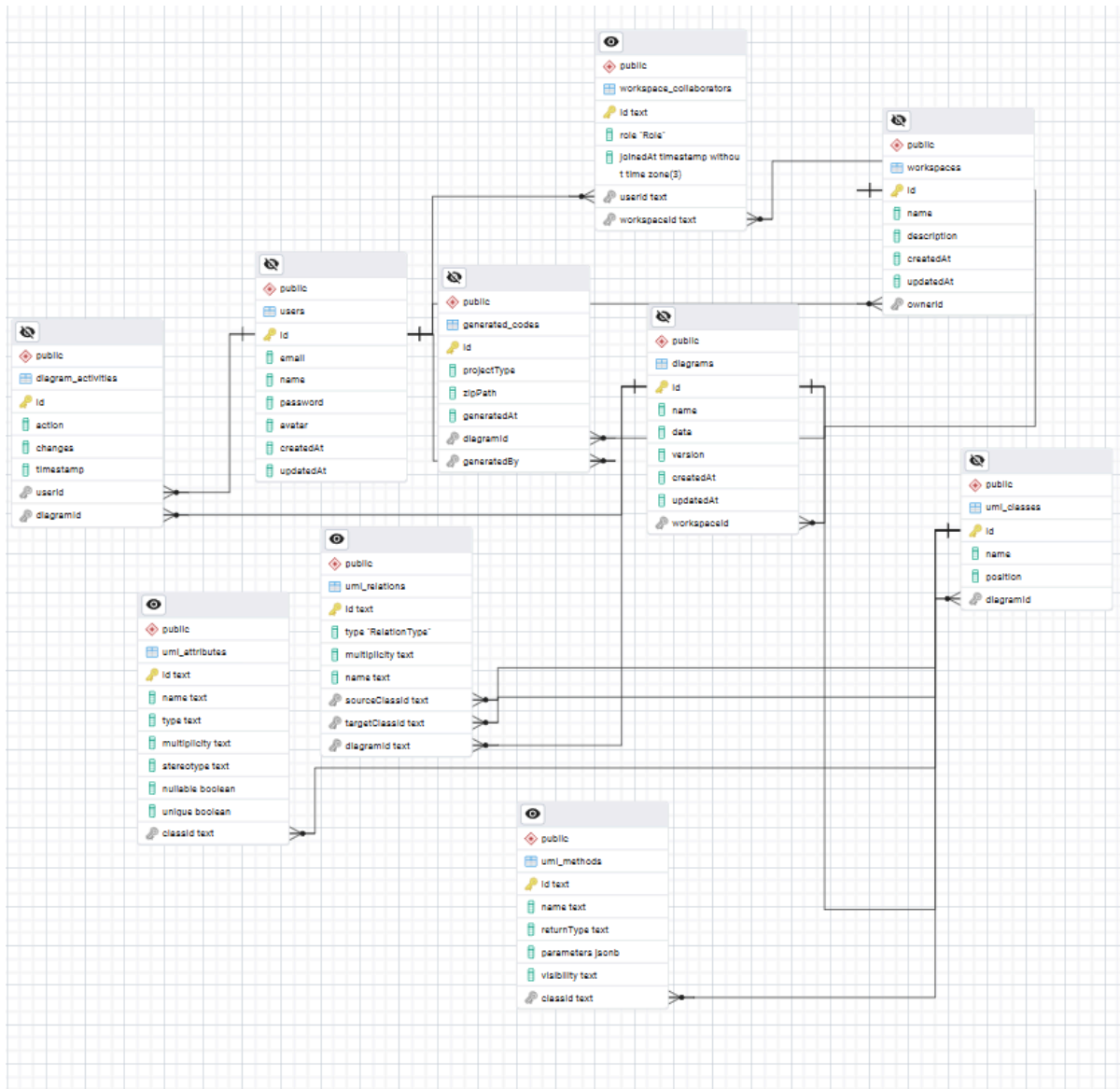
```
ALTER TABLE IF EXISTS public.workspace_collaborators  
  ADD CONSTRAINT "workspace_collaborators_userId_fkey" FOREIGN KEY ("userId")  
  REFERENCES public.users (id) MATCH SIMPLE  
  ON UPDATE CASCADE  
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.workspace_collaborators
  ADD CONSTRAINT "workspace_collaborators_workspaceId_fkey" FOREIGN KEY
("workspaceId")
  REFERENCES public.workspaces (id) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;
```

```
ALTER TABLE IF EXISTS public.workspaces
  ADD CONSTRAINT "workspaces_ownerId_fkey" FOREIGN KEY ("ownerId")
  REFERENCES public.users (id) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;
```

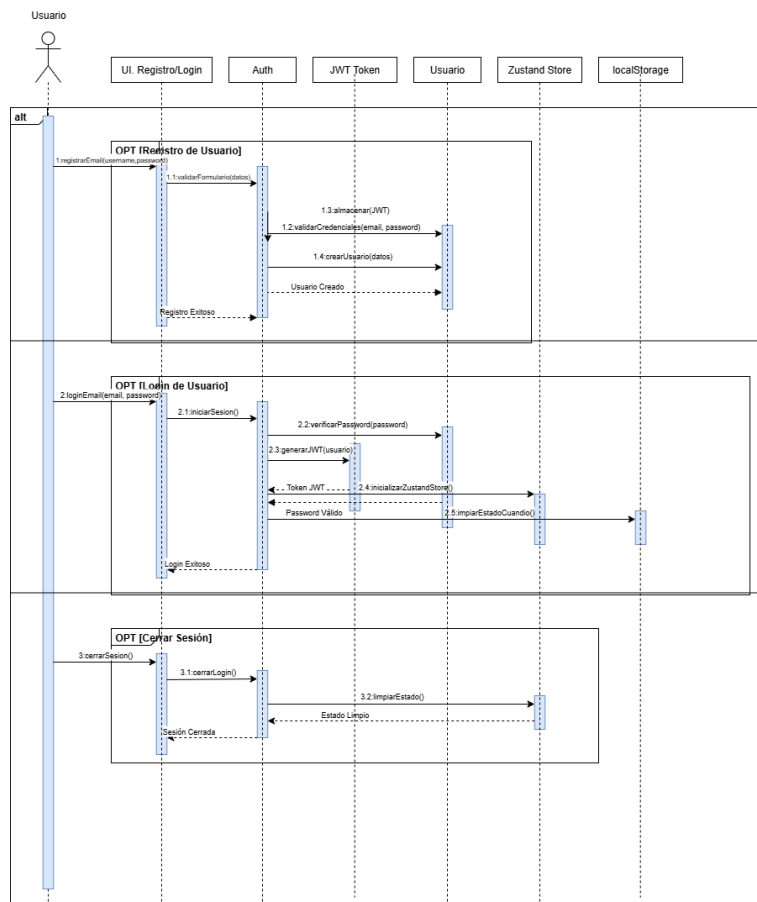
```
END;
```

2.3.2.2.5 Diagrama Entidad Relación



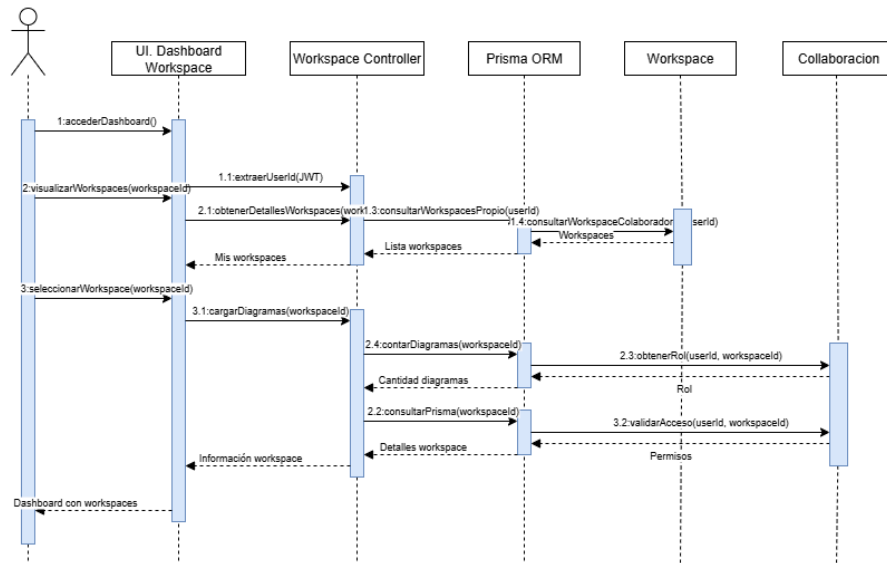
2.3.3 Diagramas de secuencia

CU1



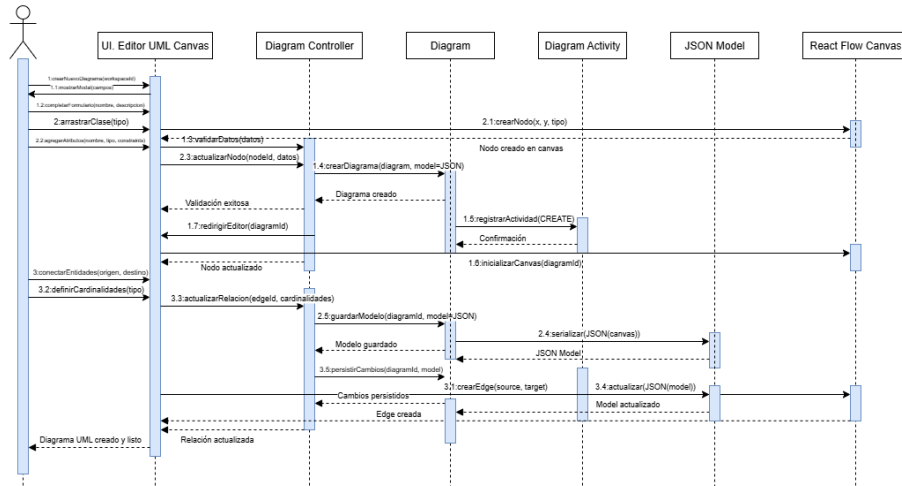
CU2

Ingeniero



CU3

Ing. Anfitrión



```

sequenceDiagram
    actor Ingeniero
    participant UI as UI.Canvas React Flow
    participant WS as WebSocket Controller
    participant SIO as Socket IO Gateway
    participant D as Diagram
    participant CC as Clientes Conectados
    participant DA as Diagram Activity

    Ingeniero->>UI: 1 modificaDiagrama(action)
    activate UI
    UI->>WS: 1.1 detectaConflicto(version)
    activate WS
    WS->>SIO: 1.2 actualizaDiagrama(action, changeData)
    activate SIO
    SIO->>D: 2. propagaCambios(cambios)
    activate D
    D->>D: 2.1 obtenerClientes(room)
    D->>CC: 3.1 obtenerClientes(ChangeCreatedData)
    activate CC
    CC->>D: 3.3 incrementarVersion()
    deactivate CC
    D->>DA: 1.7 registraActividad(action, timestamp)
    activate DA
    DA-->>D: 
    deactivate DA
    D->>CC: 3.2 notificaConflicto()
    activate CC
    CC-->>D: 
    deactivate CC
    SIO->>UI: 2.1 actualizaCanvas(diagram_version)
    SIO->>UI: 2.2 broadcastEvent(diagram_changed)
    deactivate SIO
    UI->>Ingeniero: 2.3 notificaConfirmacion()
    deactivate UI
  
```

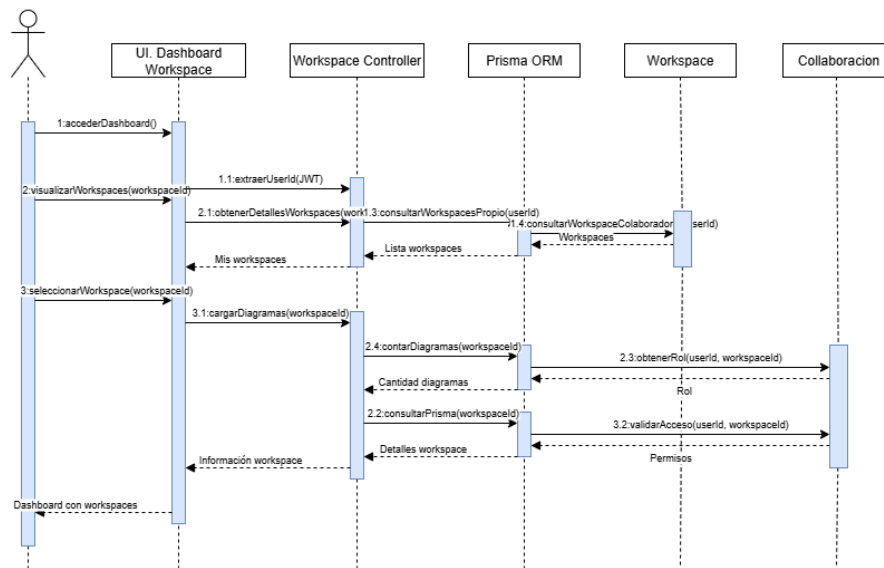
```

sequenceDiagram
    actor IngAnfitrión as Ingeniero Anfitrión
    participant UI as UI_Modal Invitar
    participant Controller as Colaboracion Controller
    participant Colaboracion
    participant Usuario
    participant Notificacion
    participant Workspace

    IngAnfitrión->>UI: 1.invitarColaborador()
    activate UI
    UI->>Controller: 1.2.validarEmail(email), rol
    deactivate UI
    activate Controller
    Controller->>UI: 3.verificarExisteColaborador(email)
    deactivate Controller
    activate UI
    UI->>Controller: 1.1.crearInvitation(workspaceId, email, rol)
    deactivate UI
    activate Controller
    Controller->>Colaboracion: 1.4.buscarUsuario(email)
    deactivate Controller
    activate Colaboracion
    Colaboracion->>Controller: 3.1.validarPermisoInvitation(userId, workspaceId)
    deactivate Colaboracion
    activate Controller
    Controller->>Colaboracion: 1.2.validarFormulario(data)
    deactivate Controller
    activate Colaboracion
    Colaboracion->>Usuario: 2.2.enviarEmailInvitation(email)
    deactivate Colaboracion
    activate Usuario
    Usuario->>Colaboracion: 3.3.obtenerDatosWorkspace(workspaceId)
    deactivate Usuario
    activate Colaboracion
    Colaboracion->>Colaboracion: Confirmitacion
    deactivate Colaboracion
    Colaboracion->>Colaboracion: 1.7.registrarInvite(userId, workspaceId)
    deactivate Colaboracion
    Colaboracion->>Notificacion: 3.2.obtenerColaboradores(workspaceId)
    deactivate Colaboracion
    activate Notificacion
    Notificacion->>Workspace: 1.6.crearColaboracion(workspaceId, userId, rol)
    deactivate Notificacion
    activate Workspace
    Workspace->>Colaboracion: 4.7.verificarColaboradorDuplicado()
    deactivate Workspace
    activate Colaboracion
    Colaboracion->>Colaboracion: Confirmitacion
    deactivate Colaboracion
    Colaboracion->>Controller: Colaborador agregado
    deactivate Colaboracion
    activate Controller
    Controller->>UI: 1.5.notificarUsuarioEncontrado()
    deactivate Controller
    activate UI
    UI->>IngAnfitrión: 4.mostrarUsuarioColable()
    deactivate UI
    deactivate IngAnfitrión
  
```

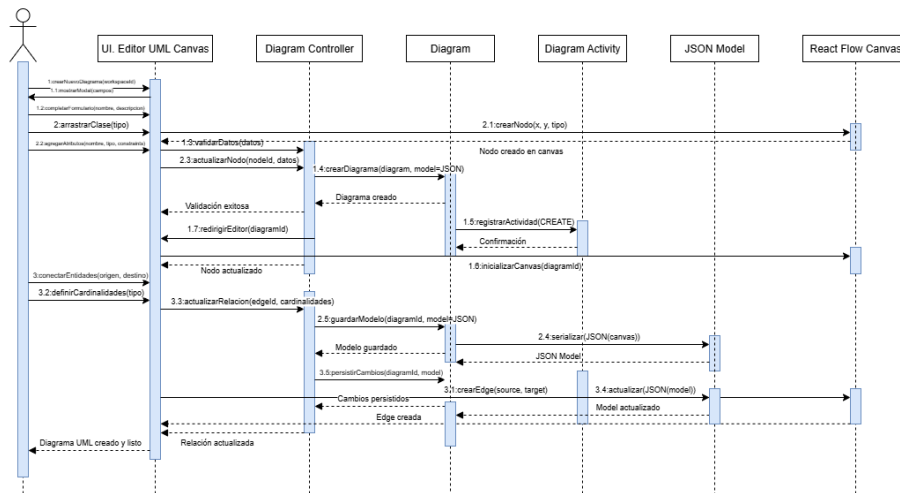
CU6

Ingeniero

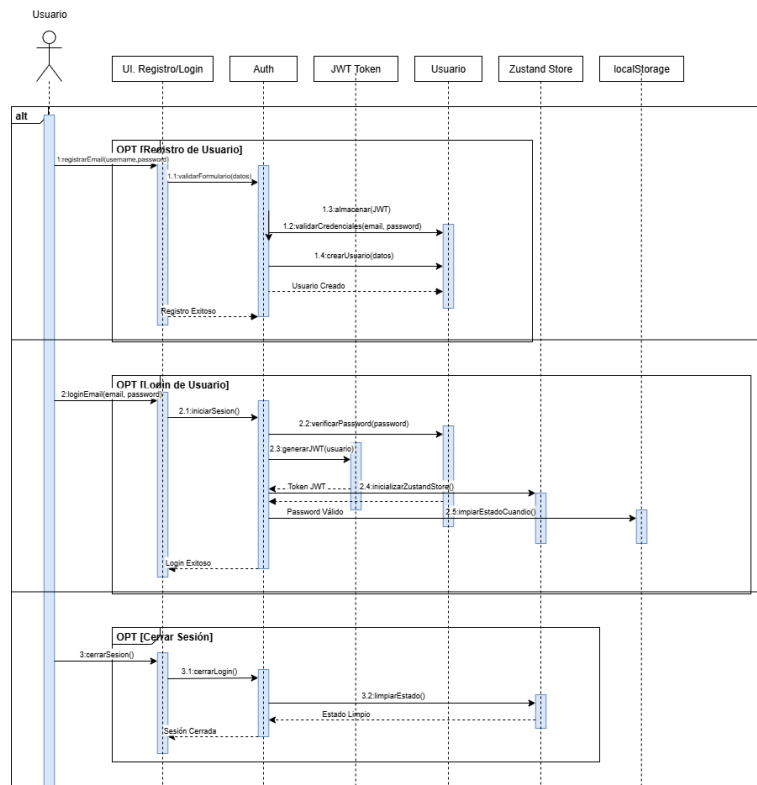


CU7

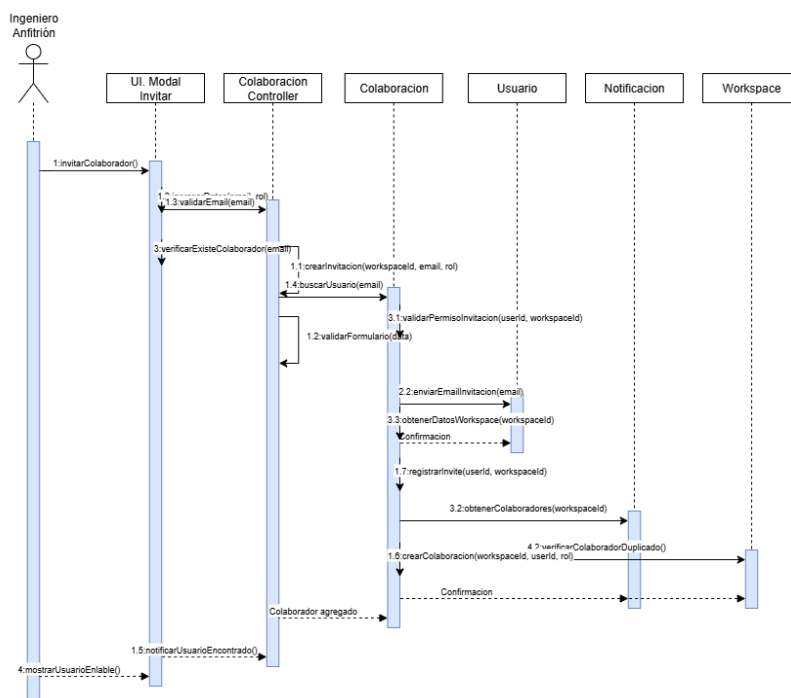
Ing. Anfitrión



CU8

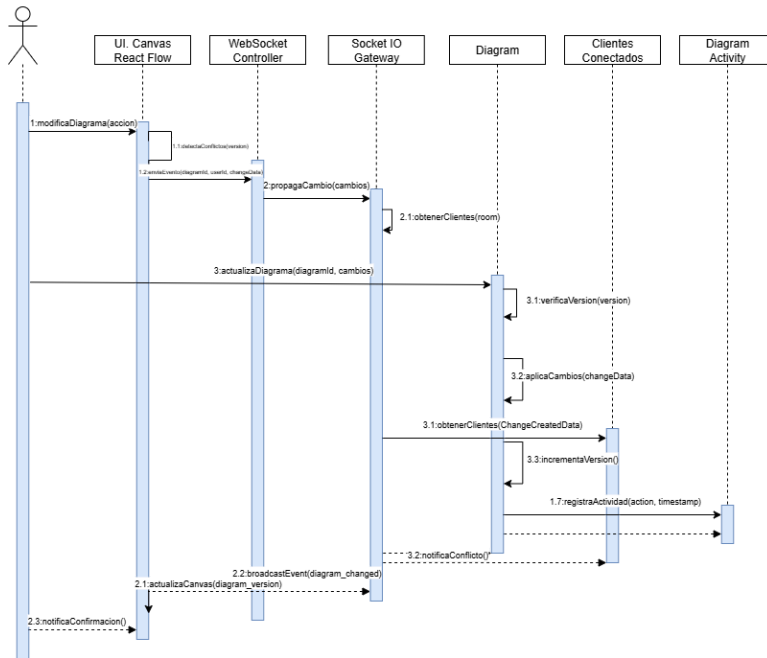


CU9

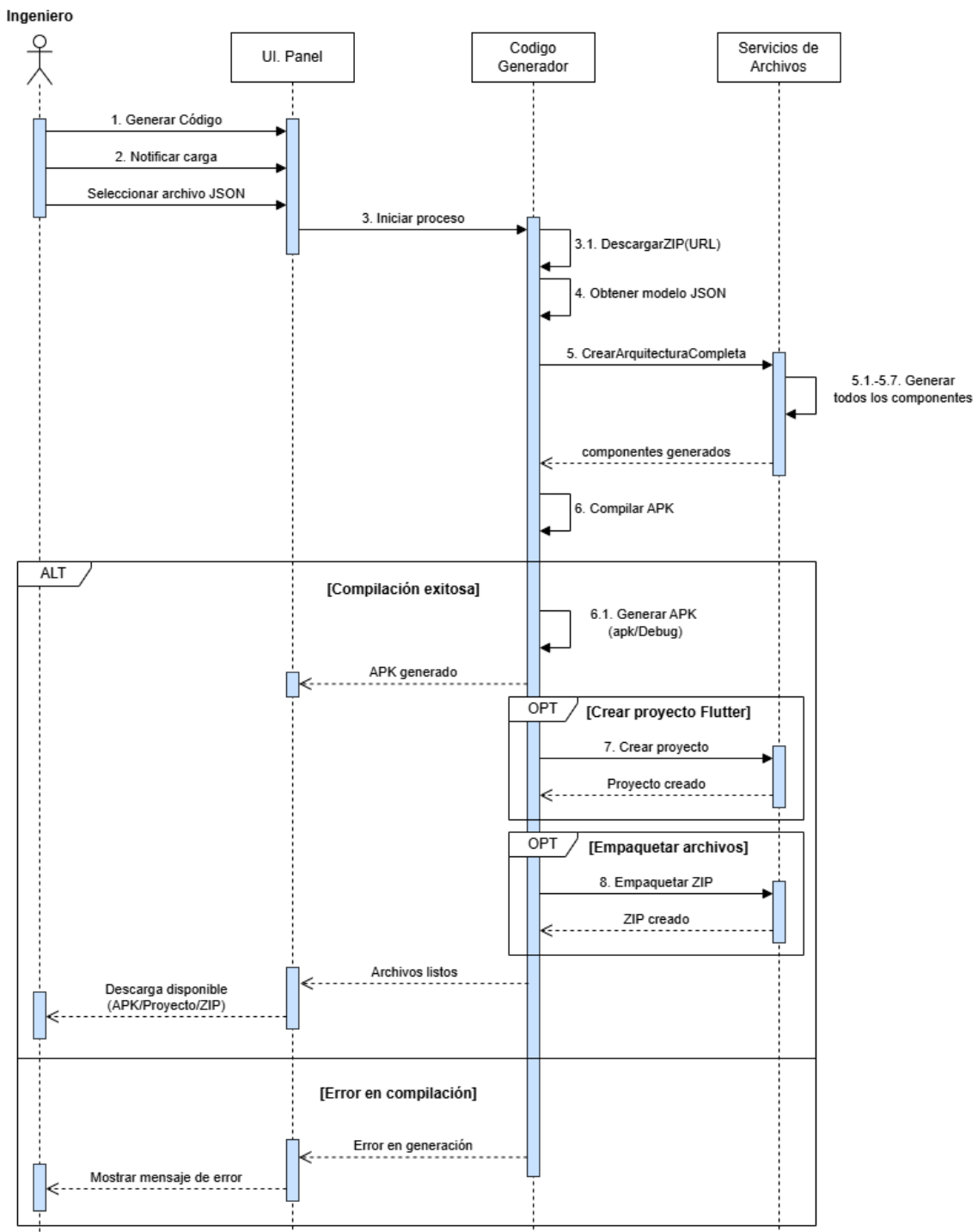


CU10

Ingeniero



CU11



2.4 IMPLEMENTACIÓN

2.4.1 Elección de plataforma de Desarrollo de Software

La plataforma seleccionada es TypeScript sobre Node.js para backend (NestJS) y frontend (Next.js 14). Esta combinación ofrece tipado estático, ecosistema maduro, desarrollo full-stack unificado y capacidades de tiempo real mediante WebSockets, alineadas con los requisitos del proyecto: editor UML interactivo, colaboración en tiempo real, generación de código Spring Boot y asistencia con IA.

2.4.1.1 Lenguaje de Programación

TypeScript fue elegido por:

Confiabilidad: Tipado estático reduce errores en producción. Los DTOs garantizan comunicación correcta entre frontend, backend y base de datos. Cuando el frontend solicita actualizar un diagrama, el backend valida automáticamente que la estructura JSON cumple con el tipo UpdateDiagramDto, evitando errores que podrían corromper el modelo.

Mantenibilidad: Refactorización segura. Si se modifica la estructura de una clase UML, TypeScript identifica automáticamente todos los usos en el código. Las interfaces TypeScript actúan como documentación viva del sistema.

Productividad: Autocompletado inteligente (IntelliSense), linters como ESLint con reglas específicas para TypeScript, y compilación veloz con esbuild aceleran el desarrollo. Las pruebas con Jest se benefician del tipado para crear mocks que reflejan el comportamiento real.

Full-stack unificado: Un solo lenguaje para frontend y backend reduce cambios de contexto mental. Las interfaces que definen la estructura de diagramas (DiagramModel, UMLClass, UMLRelation) pueden compartirse entre capas en un paquete común, garantizando interpretación idéntica de datos.

Ecosistema maduro: Integración perfecta con NestJS (decoradores, inyección de dependencias), Next.js 14 (tipado de rutas y componentes), Prisma (tipos generados desde BD), Socket.IO (eventos tipados), React Flow (nodos y edges tipados) y class-validator (validación con decoradores).

2.4.1.2 DBMS (Database Management System)

Se seleccionó PostgreSQL por:

Robustez empresarial: Más de 30 años de desarrollo enfocado en corrección y confiabilidad. Estabilidad probada bajo alta concurrencia, crítico cuando múltiples ingenieros modifican simultáneamente el mismo diagrama desde ubicaciones distribuidas.

Transacciones ACID: El motor ACID (Atomicity, Consistency, Isolation, Durability) garantiza que operaciones como actualizar el modelo canónico, registrar actividad e incrementar versión se ejecuten atómicamente. Si una parte falla, todas las operaciones se revierten evitando estados inconsistentes. Los niveles de aislamiento configurables permiten manejar concurrencia sin comprometer integridad.

Soporte JSONB: Característica clave para este proyecto. El tipo JSONB (JSON binario) permite almacenar el modelo canónico completo del diagrama UML en un solo campo con estructura compleja: classes (con id, name, position, attributes, methods), relations (con sourceId, targetId, type, multiplicity). PostgreSQL permite consultas eficientes sobre este JSON usando operadores nativos sin parsear en la aplicación, como buscar diagramas que contengan una clase específica.

Tipos de datos avanzados: ENUM para roles (OWNER, EDITOR, VIEWER) y tipos de actividad (CREATED, UPDATED, DELETED). TIMESTAMP WITH TIME ZONE para rastrear modificaciones con precisión en entornos distribuidos. TEXT sin límites arbitrarios para descripciones largas.

Índices especializados: GIN (Generalized Inverted Index) para búsquedas rápidas en campos JSONB. B-Tree para claves primarias y foráneas. Índices parciales para consultas específicas como listar solo diagramas activos.

Integración con Prisma ORM: Prisma genera automáticamente tipos TypeScript desde el esquema de base de datos definido en schema.prisma, garantizando type-safety completo. Permite migraciones declarativas, queries tipadas con autocompletado, y transacciones programáticas con prisma.\$transaction().

Compatibilidad cloud: Soportado nativamente por AWS RDS, Google Cloud SQL, Azure Database, y servicios PaaS como Railway, Heroku, Render, Supabase, Neon.

2.4.1.3 Sistemas Operativos e Infraestructura

El proyecto utiliza una arquitectura de despliegue distribuida en plataformas PaaS especializadas, donde cada componente se aloja en el servicio más adecuado para sus características. Esta estrategia aprovecha servicios administrados que ejecutan sobre sistemas operativos Linux optimizados, abstrayendo la gestión de infraestructura mientras mantiene compatibilidad y portabilidad.

Frontend → Vercel

El frontend Next.js 14 se despliega en Vercel, plataforma creada por los mismos desarrolladores de Next.js. Vercel ejecuta sobre infraestructura AWS con sistemas Linux optimizados para edge computing y serverless functions. Proporciona:

- Edge Network global con distribución automática desde ubicaciones cercanas al usuario
- Despliegues automáticos en cada push y preview deployments por pull request
- Optimizaciones automáticas (compresión, lazy loading, code splitting)
- Escalado automático según tráfico sin configuración manual

Backend y Base de Datos → Railway

El backend NestJS y PostgreSQL se despliegan en Railway, plataforma PaaS que ejecuta sobre Google Cloud Platform con contenedores Linux (Debian/Ubuntu). Railway proporciona:

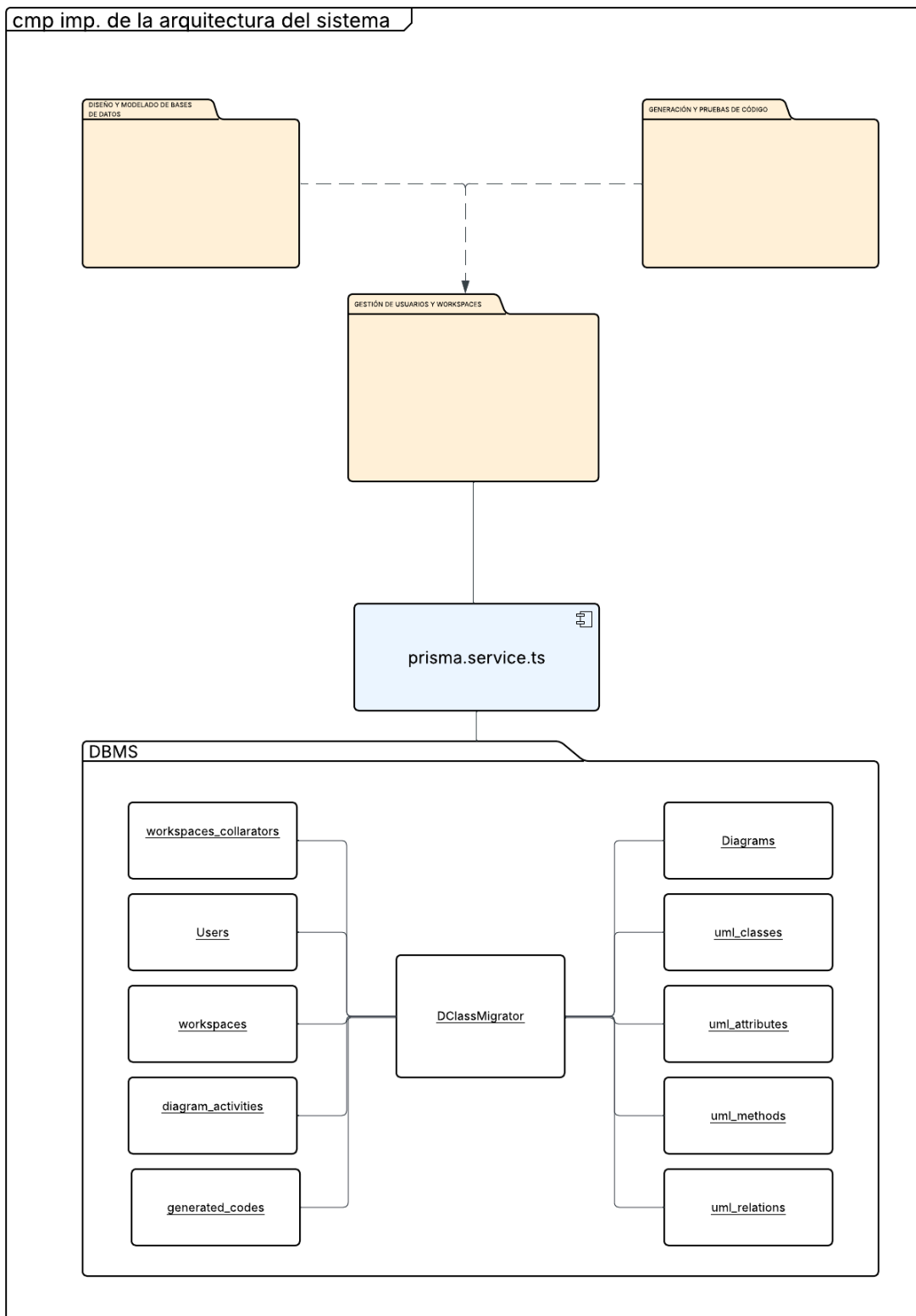
- Contenedores Docker automáticos detectando el proyecto NestJS
- PostgreSQL 16 administrado con backups automáticos diarios
- Networking privado entre backend y base de datos sin exposición pública
- Variables de entorno seguras con encriptación (DATABASE_URL, JWT_SECRET, GEMINI_API_KEY)
- Logs centralizados y métricas de rendimiento en tiempo real

Aunque Railway y Vercel abstraen la gestión directa del sistema operativo, ambos ejecutan sobre Linux, aprovechando características del kernel como namespaces (aislamiento de procesos), cgroups (límites de recursos) y overlay filesystems (capas eficientes para Docker).

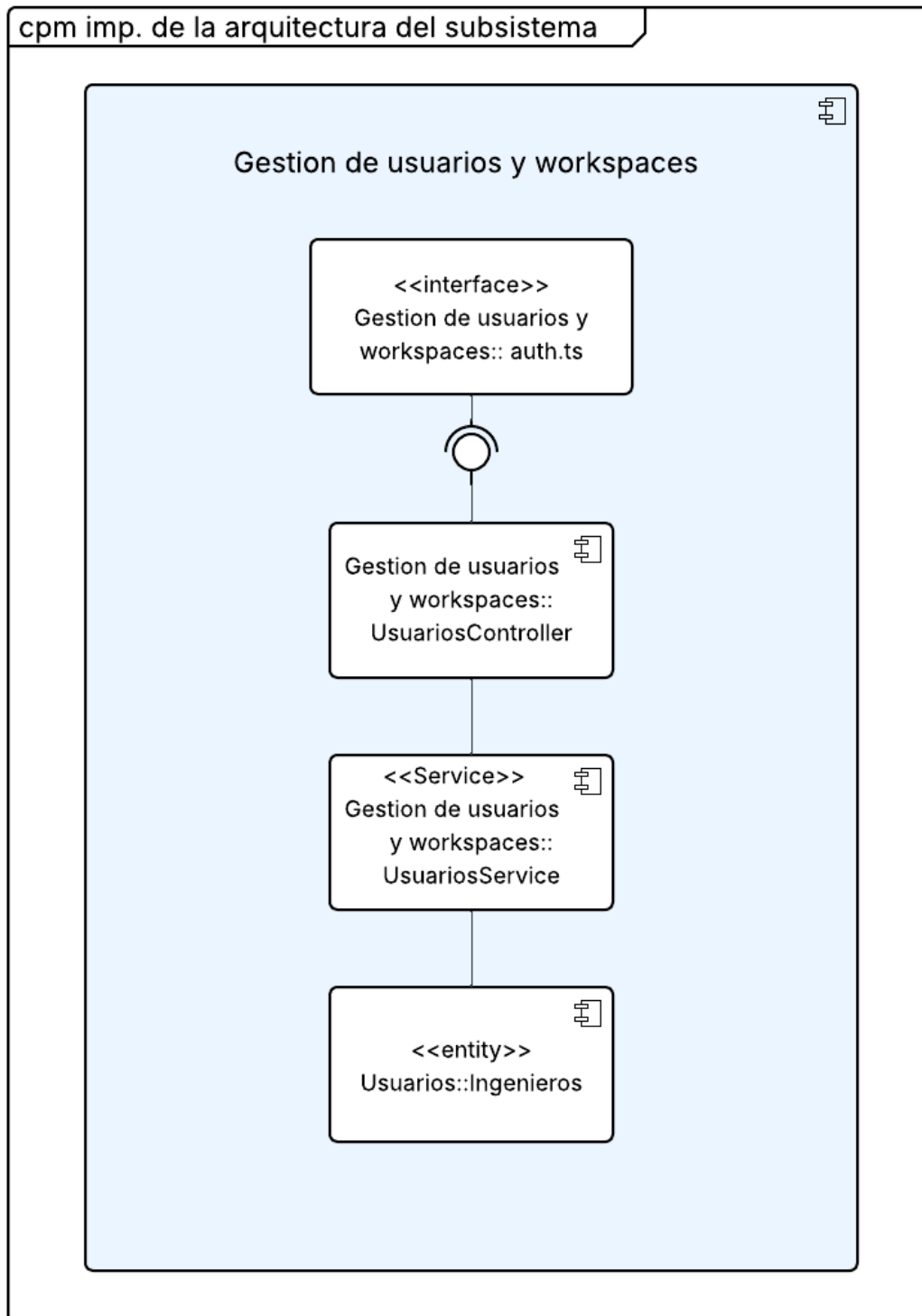
Ventajas de esta arquitectura:

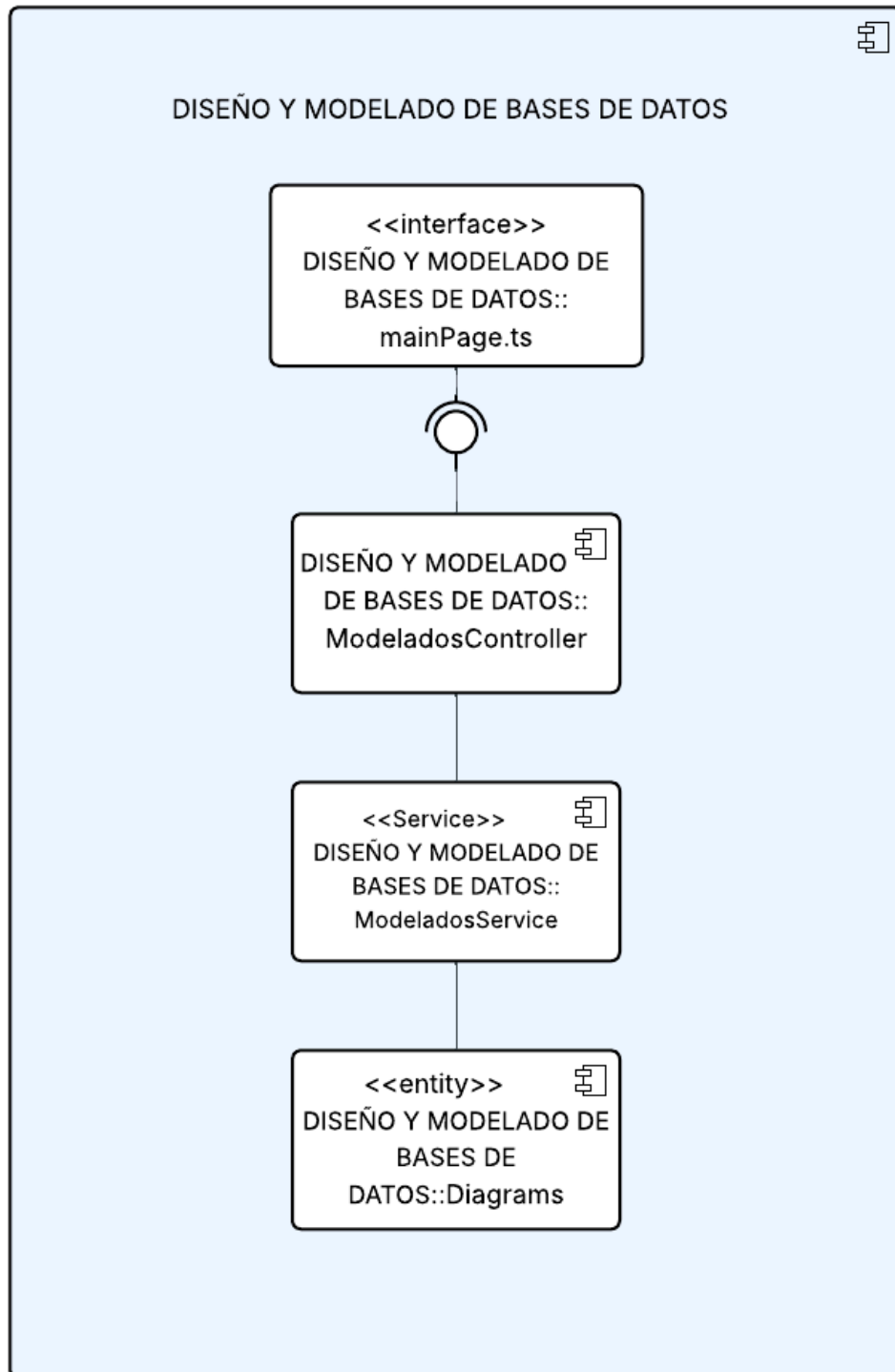
- Especialización: cada plataforma optimizada para su propósito (Vercel para frontend/edge, Railway para backend con estado)
- Despliegue independiente: cambios en UI no requieren redeploy del backend
- Escalado granular: cada componente escala según su demanda específica
- Resiliencia: arquitectura distribuida aumenta disponibilidad global

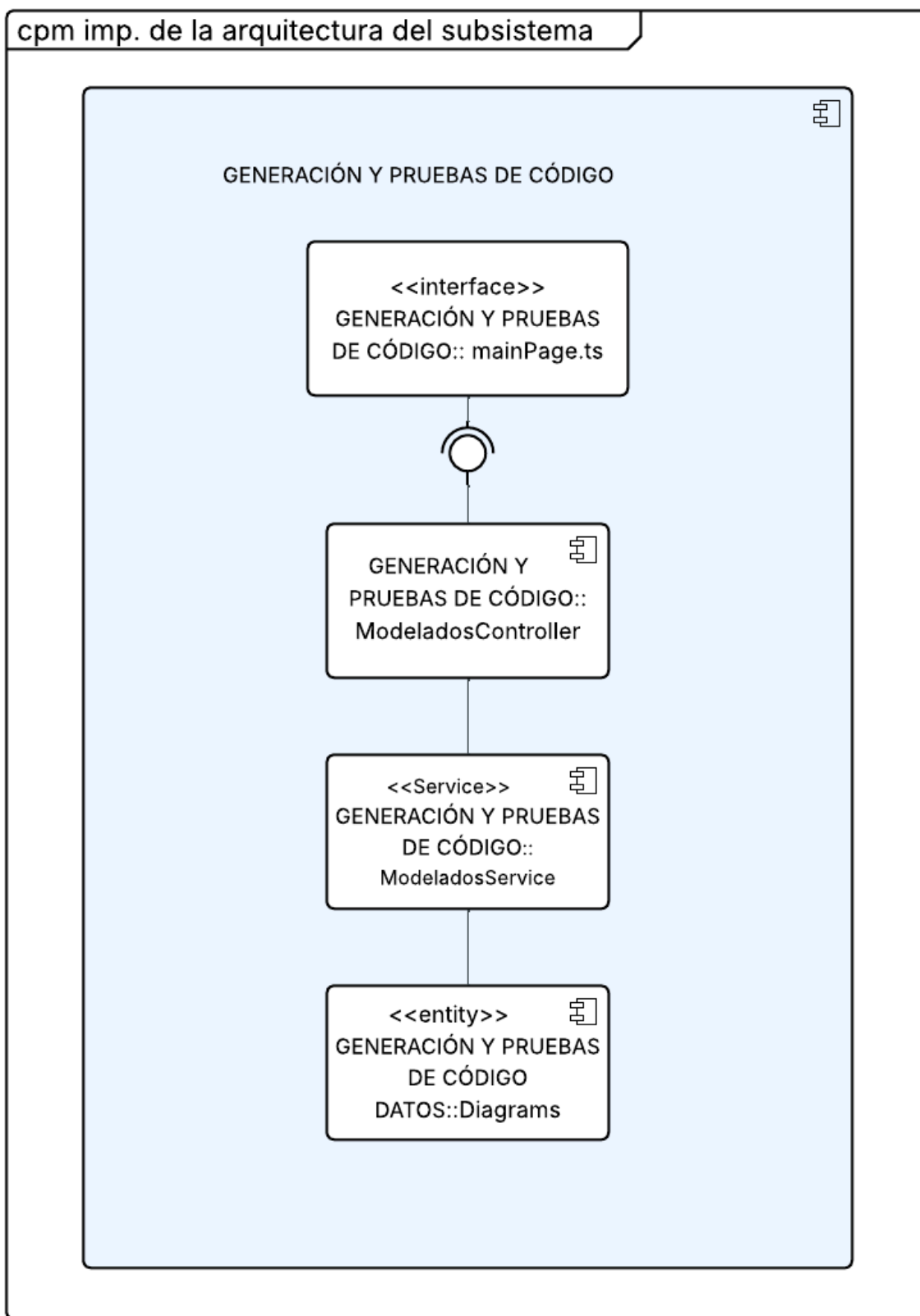
2.4.2 Implementación de la Arquitectura del Sistema



2.4.3 Implementación de la Arquitectura del Subsistema







2.5 AMPLIACIÓN FUNCIONAL - CICLO #2

Este ciclo amplía la plataforma CASE colaborativa para convertirla en un entorno integral de modelado, generación, consulta y evolución de software. Se conserva la arquitectura existente y se incorporan capacidades móviles, trabajo sin conexión, interacción multimodal, repositorio interno e interoperabilidad con herramientas UML externas.

2.5.1 Objetivo de la ampliación

Permitir que un equipo diseñe diagramas UML, genere backend Spring Boot y aplicación Flutter, revise el código producido, colabore en tiempo real o sin conexión y traslade sus modelos a otros programas, manteniendo trazabilidad entre cada diagrama y cada versión generada.

Alcance funcional

- Repositorio interno por workspace para archivos, carpetas, revisiones, autores y descargas.
- Aplicación Flutter con autenticación, proyectos, diagramas, código, colaboración e IA.
- Modo offline-first con base local, cola de operaciones y sincronización posterior.
- Asistente de IA por texto y audio, con capacidad local básica y servicio avanzado en línea.
- Importación y exportación de modelos mediante XMI, además de un paquete propio de respaldo.
- Despliegue en Google Cloud con servicios administrados y almacenamiento persistente.

Principios de diseño

- La generación de código será determinística y validada; la IA interpretará, sugerirá y asistirá.
- La pérdida de conectividad no impedirá consultar ni editar información ya sincronizada.
- Toda acción conservará autor, fecha, origen, versión del diagrama y resultado.

2.5.2 Nuevos actores y requisitos

Actores complementarios

ID	Actor	Responsabilidad
A3	Asistente de IA	Interpreta texto o audio, propone modelos y ejecuta funciones autorizadas.
A4	Herramienta UML externa	Intercambia modelos mediante archivos XMI compatibles.
A5	Servicios Google Cloud	Proveen ejecución, persistencia, sincronización, IA remota y observabilidad.

Requisitos funcionales

ID	Requisito	Descripción verificable
RF-12	Repositorio interno	Guardar y consultar el árbol de archivos generado por cada workspace.
RF-13	Versionado	Crear revisiones inmutables y comparar o restaurar resultados anteriores.
RF-14	Móvil offline	Consultar y editar datos locales sin conexión, registrando operaciones pendientes.
RF-15	IA multimodal	Aceptar texto o audio y entregar respuestas escritas o habladas.
RF-16	IA híbrida	Usar un modelo local para acciones básicas y Vertex AI para solicitudes complejas.
RF-17	Interoperabilidad	Importar y exportar diagramas de clases en XMI con validación previa.
RF-18	Sincronización	Conciliar cambios locales y remotos conservando trazabilidad de conflictos.
RF-19	Código desde UML	Generar Spring Boot y Flutter a partir de una versión concreta del modelo.
RF-20	Despliegue cloud	Persistir datos y artefactos fuera del sistema de archivos temporal.

2.5.3 Casos de uso ampliados

CU-12. Gestionar repositorio interno de código

Campo	Especificación
Actores	Ingeniero Anfitrión e Ingeniero Colaborador.
Propósito	Centralizar el código generado y permitir que todos los participantes autorizados consulten su historial.
Precondición	Workspace creado y diagrama válido con permisos suficientes.
Flujo	El usuario solicita una generación; el sistema valida el diagrama, genera los archivos, calcula su versión, almacena el contenido y registra autor, fecha y modelo de origen. Los participantes pueden navegar, descargar, comparar o restaurar revisiones.
Postcondición	Nueva revisión inmutable disponible en el repositorio del workspace.
Excepciones	Diagrama inválido, permisos insuficientes, generación fallida o almacenamiento no disponible.

Reglas del repositorio

- Cada revisión queda vinculada a un workspace, diagrama, usuario y tipo de proyecto.
- Los archivos se almacenan fuera del contenedor de ejecución; la base de datos conserva metadatos.
- Restaurar una revisión crea una revisión nueva y no elimina el historial anterior.
- OWNER administra miembros; EDITOR genera y modifica; VIEWER consulta y descarga.

Estructura lógica

/model contiene el modelo UML; /backend contiene Spring Boot; /mobile contiene Flutter; /docs conserva documentación y manifiestos de generación.

CU-13. Interactuar con IA móvil multimodal

El usuario interactúa desde un único chat mediante texto o voz. La aplicación transcribe el audio localmente, muestra el mensaje para su revisión y selecciona el motor de IA según conectividad, complejidad y recursos del dispositivo.

Flujo sin conexión

- El usuario escribe o mantiene presionado el botón de micrófono para dictar.
- Un modelo de reconocimiento de voz local convierte el audio en texto.
- FunctionGemma especializado transforma la orden en una función estructurada.
- La aplicación valida parámetros, permisos y versión antes de ejecutar la función.
- El resultado se guarda localmente y puede leerse mediante síntesis de voz offline.
- Las operaciones pendientes se sincronizan cuando vuelve la conexión.

Capacidades básicas offline

Categoría	Ejemplos
Edición UML	Crear clases, atributos, métodos y relaciones; cambiar nombres o tipos.
Consulta	Listar elementos, explicar relaciones y mostrar código descargado.
Validación	Detectar nombres duplicados, tipos inválidos y relaciones incompletas.
Generación	Ejecutar plantillas locales sobre diagramas disponibles en el dispositivo.

Capacidades en línea

La IA avanzada analiza descripciones extensas, propone arquitecturas completas, refina modelos grandes y explica código. Las operaciones destructivas siempre requieren confirmación explícita.

CU-14. Importar y exportar modelos UML

La plataforma intercambia modelos con herramientas externas, priorizando los elementos semánticos del diagrama de clases: paquetes, clases, atributos, operaciones, visibilidad, tipos, relaciones y multiplicidades.

Formatos

Formato	Uso	Alcance
XMI 2.1	Intercambio principal	Compatibilidad inicial con Enterprise Architect y herramientas UML equivalentes.
XMI 2.5.1	Intercambio estándar	Representación normativa para ampliar compatibilidad futura.
UMLStudio ZIP	Respaldo completo	Workspace, diagramas, revisiones, manifiesto y artefactos generados.
JSON	Integración técnica	Modelo interno validado para APIs, pruebas y migraciones.

Flujo de importación

- Seleccionar archivo y formato; verificar tamaño, codificación y estructura XML/JSON.
- Convertir el modelo externo al esquema canónico interno sin modificar todavía el workspace.
- Presentar una vista previa con elementos aceptados, advertencias y datos no compatibles.
- Confirmar la importación y crear una revisión nueva para permitir recuperación.

Reglas de compatibilidad

XMI estandariza el modelo, pero algunos programas añaden extensiones propietarias para colores, posiciones o características particulares. La primera versión garantiza los elementos UML soportados por la plataforma; el diseño visual se preservará cuando el archivo incluya coordenadas reconocibles y, en caso contrario, se aplicará auto-layout.

CU-15. Sincronizar trabajo offline y colaborativo

La aplicación móvil mantiene una copia local de los proyectos seleccionados y registra cada modificación como una operación ordenada. Al recuperar conectividad, autentica al usuario, obtiene la versión remota y sincroniza cambios pendientes.

Estados de sincronización

Estado	Comportamiento
Sincronizado	La copia local coincide con la versión confirmada por el servidor.
Pendiente	Existen operaciones locales guardadas que aún no llegaron al servidor.
En conflicto	Dos participantes modificaron el mismo elemento desde la misma versión base.
Requiere revisión	El sistema no puede fusionar automáticamente sin riesgo de pérdida.

Estrategia de conflictos

- Cambios sobre elementos distintos se fusionan automáticamente.
- Cambios sobre campos diferentes del mismo elemento se combinan cuando son compatibles.
- Cambios incompatibles conservan ambas variantes y solicitan decisión del usuario.
- Ninguna resolución elimina la revisión anterior ni oculta la identidad del autor.

Colaboración segura

Las conexiones WebSocket se autentican con un token de corta duración. Cada evento valida pertenencia al workspace y rol antes de publicarse. La presencia temporal se almacena en Redis y las revisiones confirmadas se guardan en PostgreSQL.

2.5.4 Arquitectura objetivo en Google Cloud

La plataforma utiliza servicios administrados y separa ejecución, persistencia y procesamiento inteligente. La aplicación móvil conserva sus componentes offline en el dispositivo y sincroniza mediante la misma API utilizada por el frontend web.

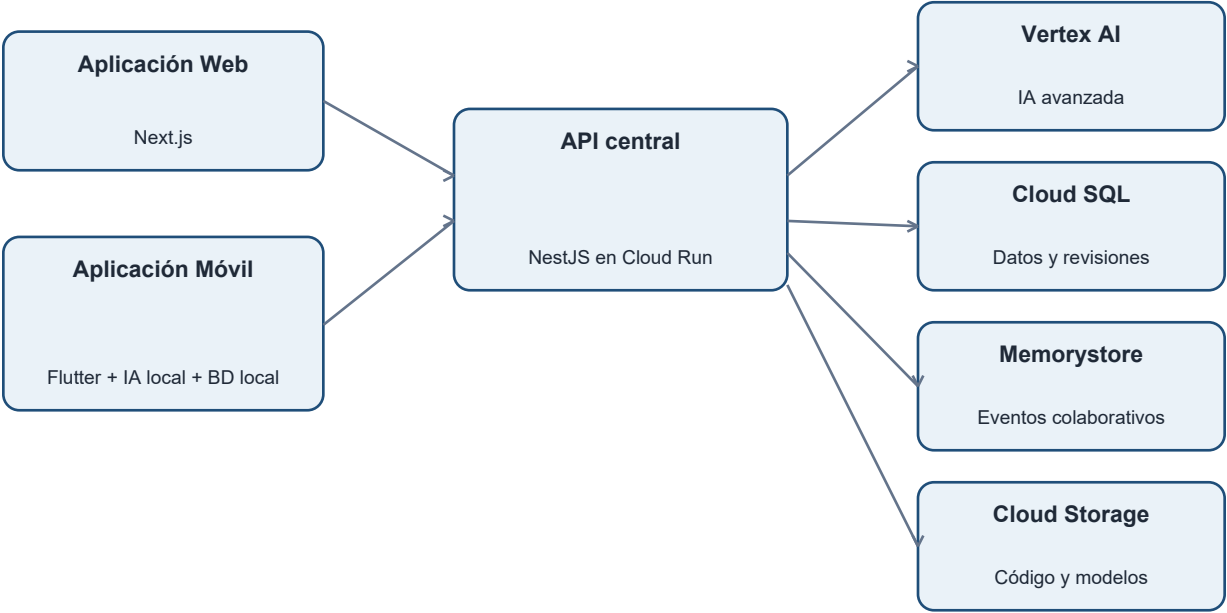


Figura 18. Arquitectura ampliada de la plataforma

Servicio	Responsabilidad
Cloud Run	Frontend, API REST, Socket.IO y generación temporal de proyectos.
Cloud SQL	Usuarios, workspaces, modelos, historial y metadatos del repositorio.
Memorystore	Presencia, salas y distribución de eventos entre instancias.
Cloud Storage	Archivos, ZIP, modelos móviles y artefactos versionados.
Vertex AI	Claude o modelo equivalente para asistencia compleja en línea.
Secret Manager	Credenciales de base de datos, tokens y claves de servicios.

2.5.5 Requisitos no funcionales y aceptación

Seguridad y privacidad

- Las claves de IA no se almacenan en el dispositivo; las solicitudes avanzadas pasan por la API.
- Los archivos se cifran en tránsito y se entregan mediante acceso temporal autorizado.
- Importaciones XML/ZIP se analizan de forma aislada, sin ejecutar código recibido.
- Los permisos se verifican en REST, WebSocket, sincronización y descarga.

Criterios de aceptación del ciclo

- Dos usuarios autorizados observan cambios del mismo diagrama y reciben reconexión automática.
- Un proyecto generado aparece como revisión navegable y descargable para sus participantes.
- La aplicación móvil permite editar, conversar por texto o audio y generar mediante plantillas sin internet.
- Al volver la conexión, las operaciones pendientes se sincronizan sin perder el historial.
- Un diagrama exportado en XML puede importarse nuevamente conservando su semántica UML.
- El backend desplegado no depende del almacenamiento local efímero de Cloud Run.

Referencias técnicas complementarias

Object Management Group. XML Metadata Interchange (XMI), versión 2.5.1. <https://www.omg.org/spec/XMI/2.5.1>

Sparx Systems. Model Exchange in XMI Format.

https://sparxsystems.com/enterprise_architect_user_guide/17.1/model_exchange/importexport.html

Google Cloud. Using WebSockets; Cloud Run and Cloud SQL documentation.

<https://cloud.google.com/run/docs/triggering/websockets>

Google AI for Developers. FunctionGemma and on-device deployment. <https://ai.google.dev/gemma/docs/functiongemma>

2.5.6 Requisitos de paridad funcional en Android

La aplicación Android debe ofrecer los mismos resultados funcionales que la aplicación web, adaptando la interacción a pantallas táctiles y manteniendo el funcionamiento offline definido. La paridad no obliga a copiar la distribución visual del navegador, pero sí a conservar permisos, reglas, datos, trazabilidad y resultados.

ID	Requisito	Descripción verificable
RF-21	Roles multicanal	Aplicar OWNER, EDITOR y VIEWER en REST, WebSocket, descargas y sincronización.
RF-22	Historial de código	Descargar, comparar y restaurar revisiones autorizadas sin eliminar el historial.
RF-23	Persistencia	Conservar datos y artefactos fuera de instancias efímeras de ejecución.
RF-24	Auditoría	Registrar accesos, cambios, generaciones, importaciones y resoluciones relevantes.
RF-25	Paridad Android	Registrar sesión, gestionar perfil, idioma, tema y todas las funciones del sistema web.
RF-26	Editor táctil	Crear y editar clases, atributos, métodos y relaciones mediante lienzo y formularios.
RF-27	Ciclo móvil	Crear, configurar, archivar, restaurar y eliminar proyectos y diagramas según permisos.
RF-28	Invitación portable	Invitar por correo, enlace o código temporal, revocable y asociado a un rol.
RF-29	Colaboración móvil	Mostrar presencia y distribuir operaciones confirmadas en tiempo real desde Android.
RF-30	Código en móvil	Generar, navegar, comentar, descargar, comparar y restaurar código desde Android.
RF-31	Intercambio móvil	Importar y exportar XMI, JSON y ZIP desde el almacenamiento del dispositivo.
RF-32	Conflictos móviles	Mostrar variantes, autor y versión, y permitir una resolución explícita sin pérdida.

2.5.7 Casos de uso complementarios del sistema

Los siguientes casos completan la trazabilidad de las funciones ya definidas para generación, revisión, seguridad y persistencia.

ID	Caso de uso	Resultado esperado
CU-16	Generar o refinar backend con IA	Validar el modelo, obtener una propuesta explicada, confirmar y publicar una revisión trazable.
CU-17	Revisar código colaborativamente	Navegar archivos y revisiones, añadir comentarios de archivo o línea y recibir actualizaciones.
CU-18	Resolver conflictos de sincronización	Comparar variantes local y remota, elegir o combinar el resultado y conservar evidencia.
CU-19	Administrar roles y permisos	Asignar OWNER, EDITOR o VIEWER y aplicar la autorización de forma uniforme.
CU-20	Descargar, comparar o restaurar	Descargar ZIP, inspeccionar diferencias y restaurar como una revisión nueva.

Reglas comunes

- Toda propuesta de IA que modifique un modelo o código requiere una confirmación identificable.
- Las revisiones publicadas son inmutables; restaurar o refinar crea una revisión adicional.
- Los comentarios, descargas y eventos respetan el rol vigente del participante.
- Un conflicto conserva ambas variantes hasta que un usuario autorizado confirme la resolución.
- Las claves de proveedores externos permanecen únicamente en servicios protegidos del backend.

2.5.8 Casos de uso de paridad móvil

ID	Caso	Flujo y resultado
CU-21	Gestionar cuenta, proyectos y diagramas	El usuario se registra o inicia sesión, administra su perfil y ejecuta el ciclo de vida permitido de proyectos y diagramas.
CU-22	Editar un diagrama UML táctil	El usuario desplaza y conecta elementos en el lienzo; los formularios editan clases, atributos, métodos, relaciones y multiplicidades.
CU-23	Invitar mediante enlace o código	El propietario crea una invitación con rol y vencimiento; el receptor autenticado la acepta por URL o código y el sistema impide reutilización inválida.
CU-24	Generar y revisar artefactos	El usuario genera Spring Boot o Flutter, consulta el repositorio, comenta, compara, descarga y restaura según su rol.
CU-25	Intercambiar y sincronizar trabajo	El usuario importa o exporta modelos, colabora en línea y resuelve operaciones pendientes o conflictos al reconectar.

Invitaciones por enlace y código

- El servidor genera un token aleatorio y un código corto vinculados al workspace, rol, creador y fecha de vencimiento.
- La invitación puede revocarse antes de su vencimiento y solo se acepta con una sesión válida.
- La aceptación es idempotente: repetirla no duplica miembros ni eleva privilegios.
- El enlace abre la aplicación cuando esté instalada y ofrece el código como alternativa manual.

2.5.9 Arquitectura Android



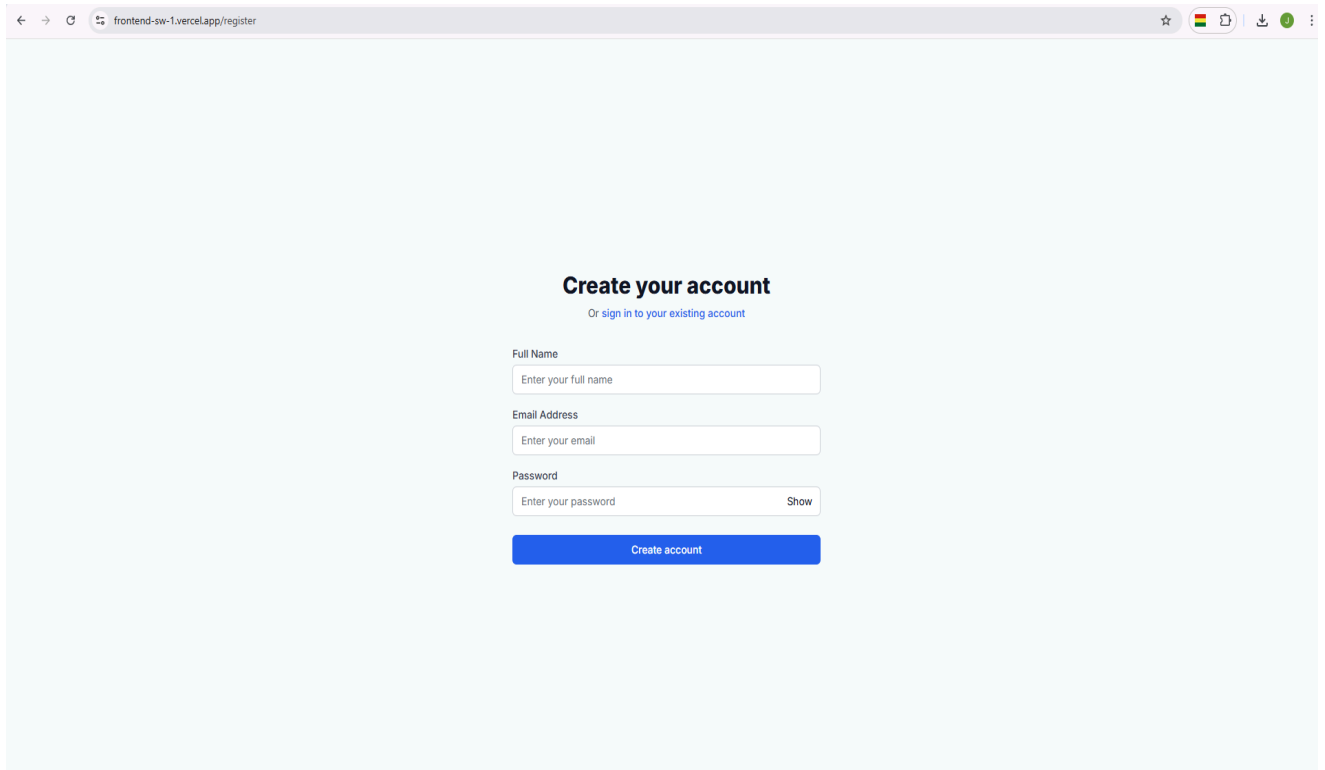
Figura 19. Componentes para la paridad funcional móvil

Criterios de aceptación móvil

- Un usuario completa desde Android cada caso de uso CU-01 a CU-25 que corresponda a su rol.
- El editor táctil permite crear, mover, conectar y modificar todos los elementos UML soportados por la web.
- Una invitación por enlace o código incorpora al usuario con el rol definido y rechaza tokens vencidos o revocados.
- Las operaciones offline sobreviven al cierre de la aplicación y se sincronizan al recuperar conectividad.
- La IA local procesa texto o audio para acciones básicas sin enviar datos a internet; la ruta avanzada se identifica como servicio en línea.
- Generación, repositorio, comentarios, historial e intercambio producen los mismos datos que la versión web.
- Las pruebas cubren permisos, reconexión, idempotencia, conflictos y recorridos Android de extremo a extremo.

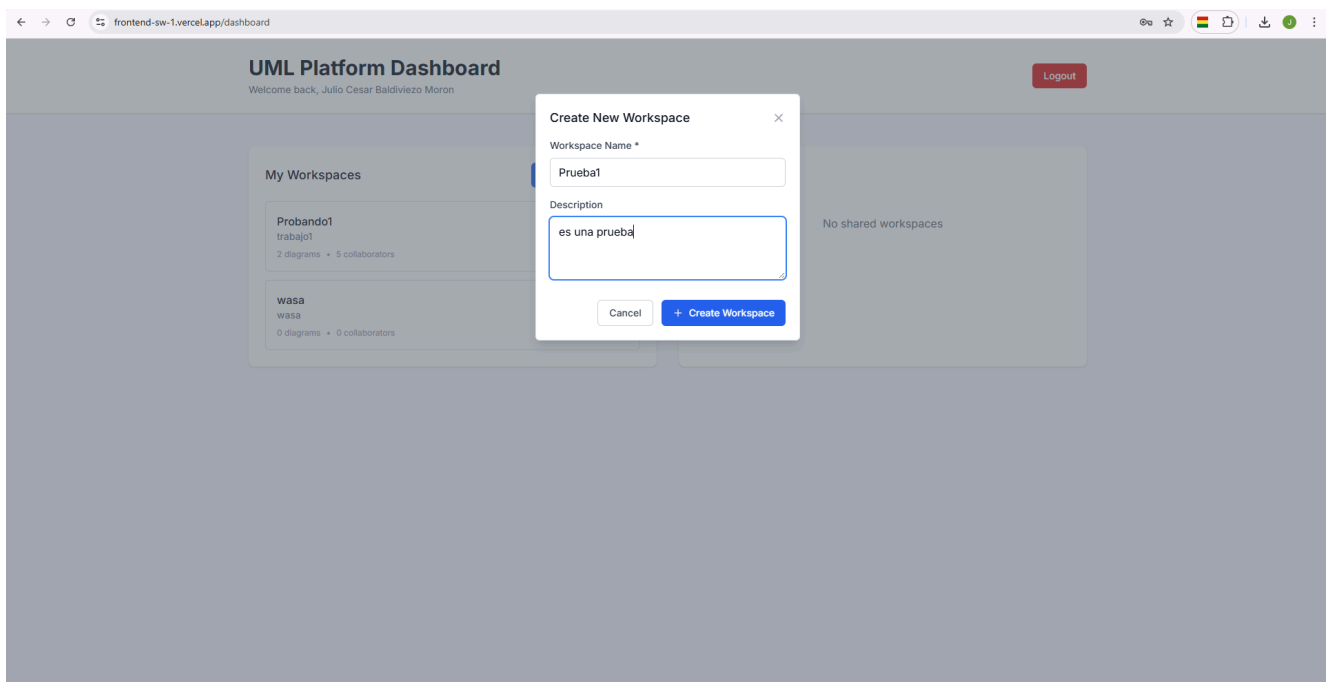
3. DOCUMENTACIÓN PARA USO DEL SOFTWARE

Crearse Cuenta:



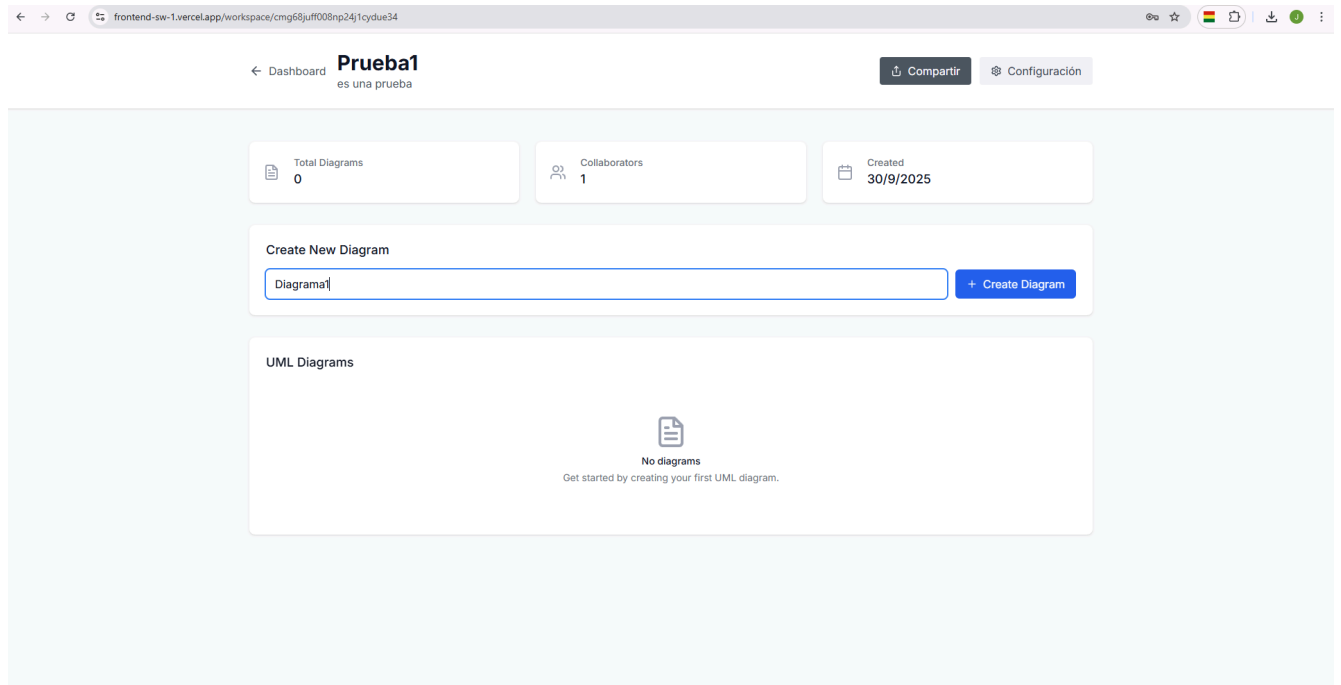
The screenshot shows a web browser window with the address bar displaying 'frontend-sw-1.vercelApp/register'. The page has a light blue background. In the center, there is a white box titled 'Create your account' with a link 'Or sign in to your existing account' below it. The form contains three input fields: 'Full Name' with placeholder text 'Enter your full name', 'Email Address' with placeholder text 'Enter your email', and 'Password' with placeholder text 'Enter your password' and a 'Show' toggle. A blue 'Create account' button is at the bottom.

Crear Espacio de trabajo

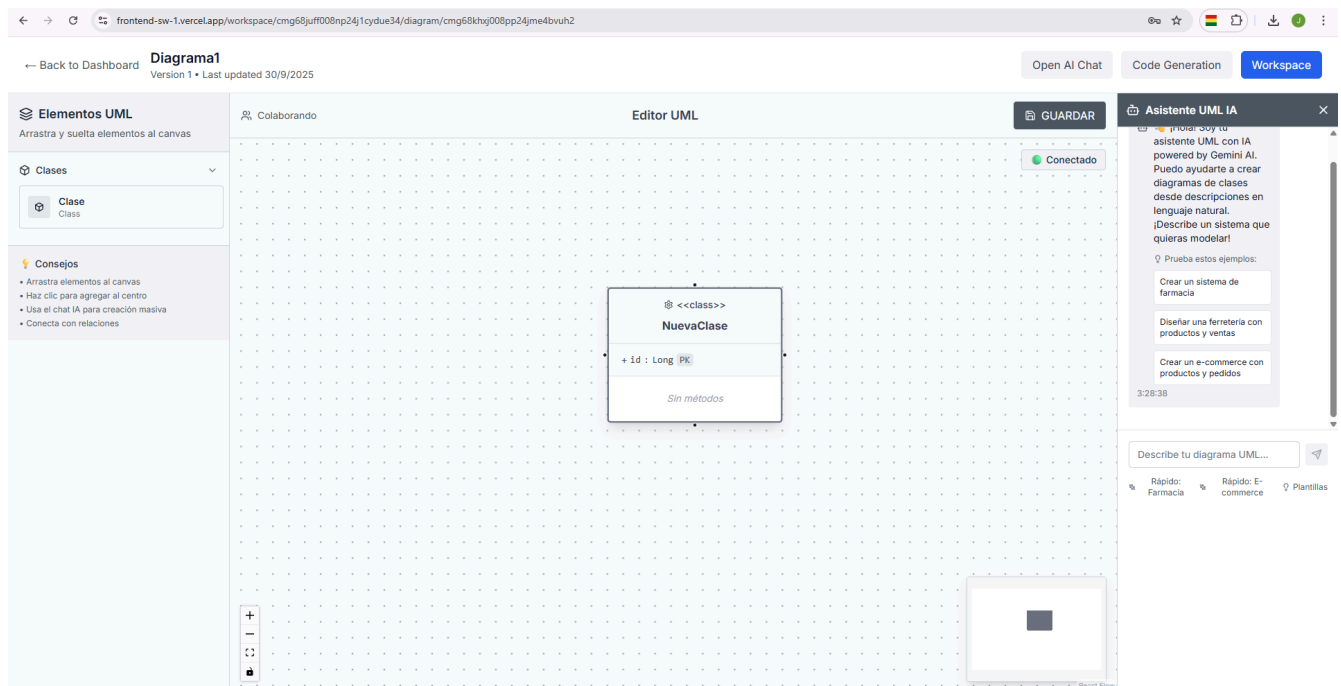


The screenshot shows a web browser window with the address bar displaying 'frontend-sw-1.vercelApp/dashboard'. The page is titled 'UML Platform Dashboard' and includes a 'Logout' button. A modal window titled 'Create New Workspace' is open, featuring a 'Workspace Name' field with the value 'Prueba1' and a 'Description' field with the value 'es una prueba'. The modal has 'Cancel' and '+ Create Workspace' buttons. In the background, the 'My Workspaces' section lists 'Probando1' (with 2 diagrams and 5 collaborators) and 'wasa' (with 0 diagrams and 0 collaborators). A 'No shared workspaces' message is also visible.

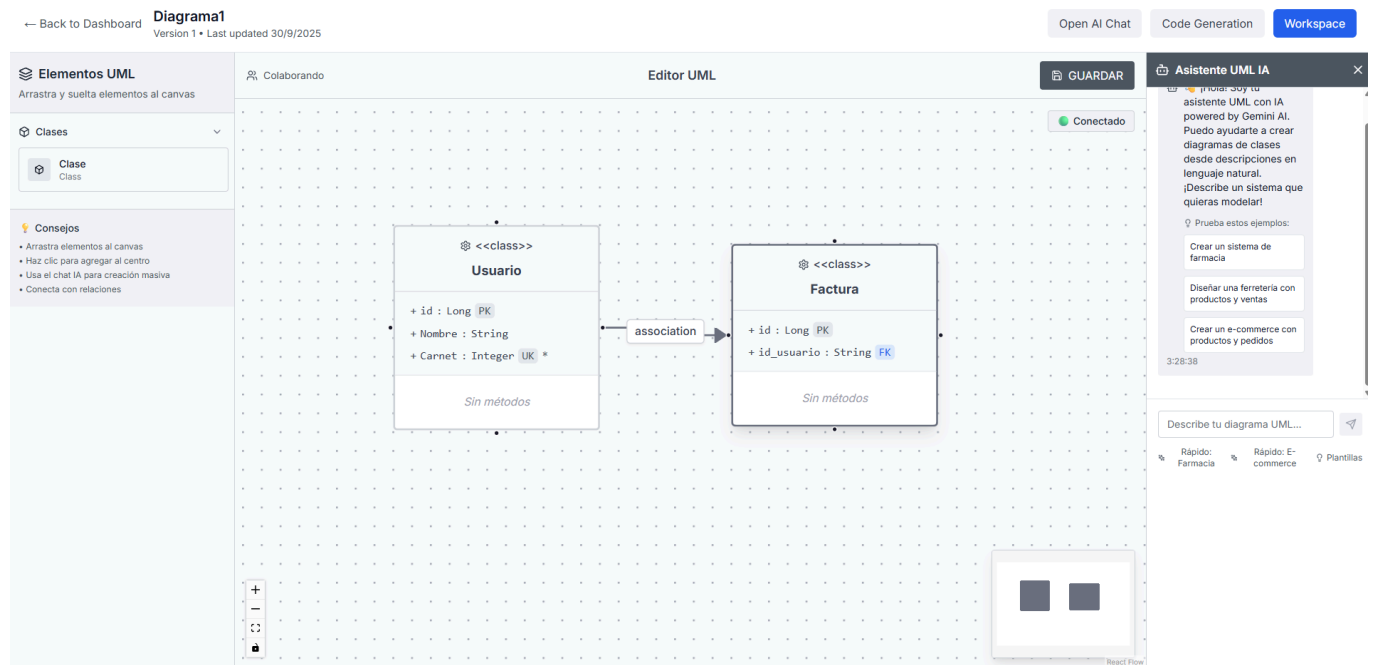
Crear Diagrama



Crear una Clase



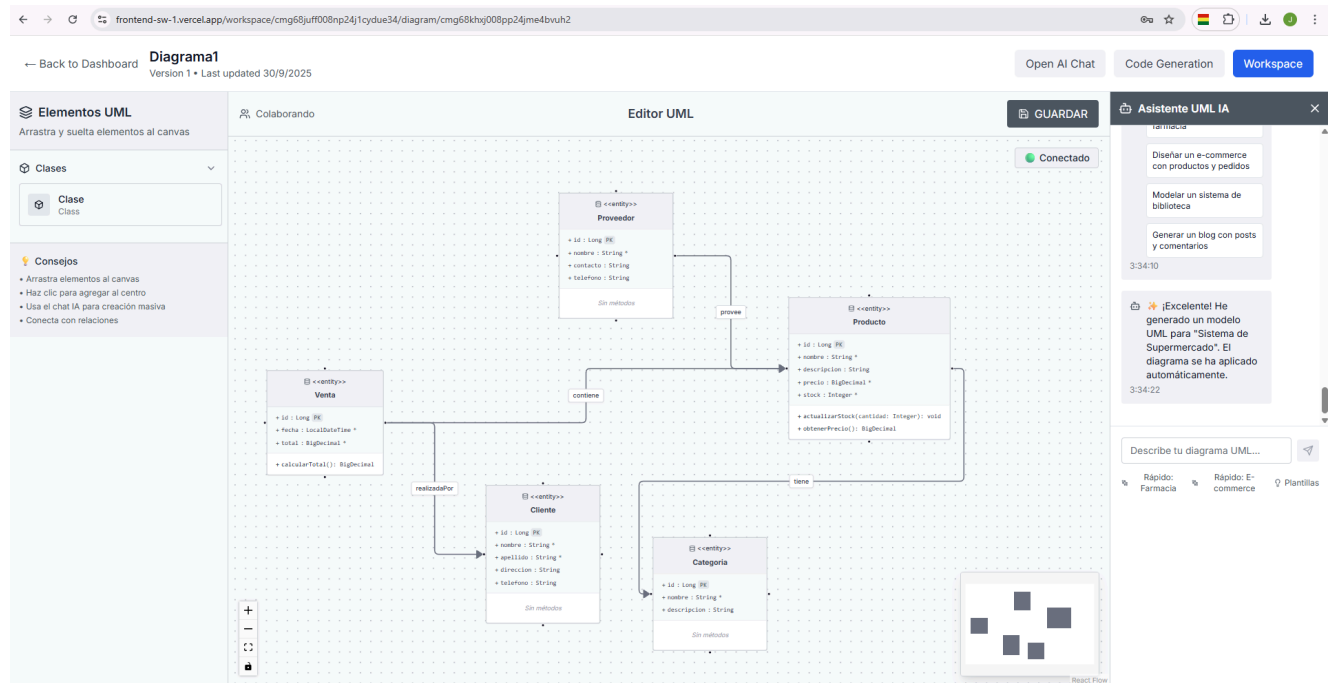
Crear relación (click sobre el punto y mantener)



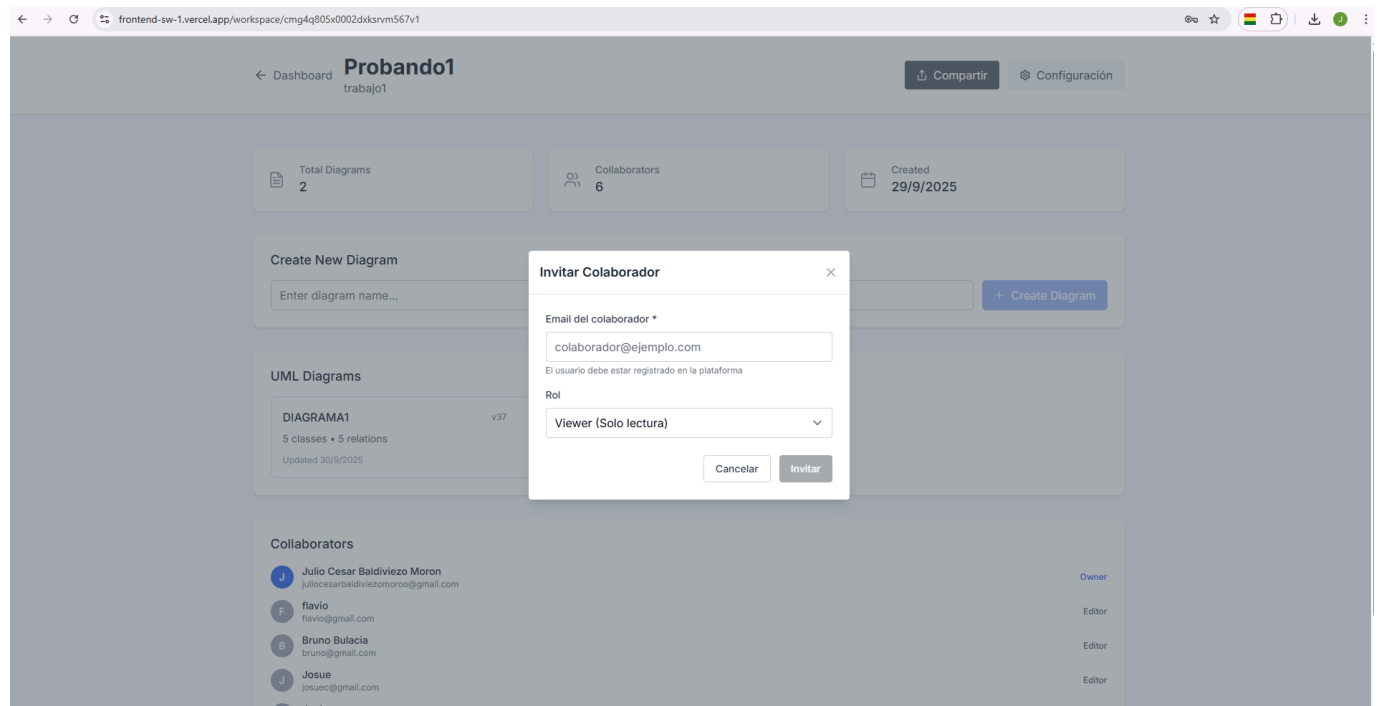
Añadir Atributos

The screenshot shows the 'Editor Clase' dialog box for the 'Usuario' class. It has a 'Class Name' field with 'Usuario'. Below, there are two sections: 'Attributes' and 'Methods'. The 'Attributes' section has a '+ Add' button and lists three attributes: 'id' (Long, Multiplicity, Not Null, Unique, Primary Key, Foreign Key), 'Nombre' (String, Multiplicity, Not Null, Unique, Primary Key, Foreign Key), and 'Carnet' (Integer, Multiplicity, Not Null, Unique, Primary Key, Foreign Key). The 'Methods' section has a '+ Add' button and a message 'No methods. Click "Add" to create one.' At the bottom, there are 'Cancel' and 'Save Class' buttons.

Interactuar Chat



Invitar Colaboradores



Generar Backend

← Back to Dashboard

DIAGRAMA1

Version 37 • Last updated 30/9/2025

Open AI Chat

Close Code Gen

Workspace

Elementos UML

Arrastra y suelta elementos al canvas

Clases

Clase

Consejos

- Arrastra elementos al canvas
- Haz clic para agregar al centro
- Usa el chat IA para creación masiva
- Conecta con relaciones

Colaborando

Editor UML

```
classDiagram
    class Medico {
        +id : Long ID
        +nombre : String
        +descripcion : String
        +previo : Boolean
        +stock : Integer
        +actualizarStock() void
    }
    class Visita {
        +id : Long ID
        +fecha : LocalDateTime
        +total : Integer
        +calcularTotal() Integer
    }
    class Cliente {
        +id : Long ID
        +nombre : String
        +apellidos : String
        +direccion : String
        +telefono : String
    }
    class Receta {
        +id : Long ID
        +nombre : String
        +descripcion : String
        +medicamento : String
    }
    Medico --> Visita
    Medico --> Receta
    Visita --> Cliente
    Receta --> Cliente
```

Code Generation

Generate a complete Spring Boot project from your UML diagram.

Generate Spring Boot Project

Generated Project Includes:

- JPA Entity classes with annotations
- Repository interfaces (Spring Data JPA)
- Service layer with CRUD operations
- REST Controllers with endpoints
- Spring Boot application configuration
- Maven build configuration (pom.xml)
- PostgreSQL database configuration

React Flow

4. BIBLIOGRAFÍA

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Schach, S. R. (2011). *Object-Oriented and Classical Software Engineering* (8th ed.). McGraw-Hill Education.
- Spring. (2024). *Spring Boot Reference Documentation*. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Microsoft. (2024). *TypeScript Documentation*. Recuperado de <https://www.typescriptlang.org/docs/>
- PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. Recuperado de <https://www.postgresql.org/docs/>
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- The Linux Foundation. (2023). *Kubernetes Documentation*. Recuperado de <https://kubernetes.io/docs/>
- Vercel. (2024). *Next.js Documentation*. Recuperado de <https://nextjs.org/docs>

5. ANEXOS

5.1 Repositorios del sistema base

Los códigos QR de la página siguiente permiten acceder a los repositorios del frontend y backend utilizados como base para la ampliación del proyecto. Las nuevas funcionalidades se integrarán progresivamente sobre esta arquitectura.

Anexo actualizado para el Primer Parcial.

Backend:



Frontend:

